Ministère de l'enseignement supérieur
et de la recherche scientifique
-----*-----
Université de la Manouba
-----*-----
ÉCOLE NATIONALE DES SCIENCES
DE L'INFORMATIQUE

MÉMOIRE

Présenté en vue de l'obtention du

Diplôme de Mastère de Recherche

en Informatique

Parcours : Data Science

Sujet :

Utilisation des LLM et du DRL dans une approche fondée sur la recherche locale pour résoudre le problème du sac à dos multi-objectif

---

Élaboré par :

Aziz DHAOUI

Laboratoire de Recherche en Intelligence Artificielle
LARIA

Soutenu le 21 octobre 2026 devant le jury d'examen composé de :

Présidente : Pr. Leila BEN AYED

Rapporteure : Dr. Amal TARIFA

Encadrante : Dr. Imen BEN MANSOUR

Année universitaire : 2025/2026

Dédicaces

C'est avec joie que je dédie ce travail :

À mes parents, dont l'amour, le soutien et les sacrifices ont rendu possible chaque étape de mon parcours,

À mes frères et sœurs, pour leur encouragement constant et leur présence bienveillante,

À mes amis et collègues de l'ENSI, pour les moments partagés et les discussions qui ont enrichi mon expérience académique,

À tous ceux qui croient que la recherche scientifique est avant tout une aventure collective et humaine.

Aziz DHAOUI

Remerciements

Ces quelques lignes ne sont pas une formalité : elles s'adressent à celles et ceux qui ont rendu ce travail possible.

Je tiens tout particulièrement à exprimer ma gratitude à **Dr. Imen BEN MANSOUR**, mon encadrante au laboratoire **LARIA** de l'**École Nationale des Sciences de l'Informatique**. Sa rigueur scientifique, sa disponibilité et ses conseils éclairés ont été déterminants. Travailler sur un sujet à la croisée de ses propres travaux, l'algorithme W-CMOLS, et des avancées récentes en intelligence artificielle a constitué une expérience formatrice exceptionnelle, et je lui suis reconnaissant de la confiance qu'elle m'a accordée comme de la liberté qu'elle m'a laissée d'explorer des pistes parfois ambitieuses.

J'adresse mes remerciements respectueux aux membres du jury : **Pr. Leila BEN AYED**, qui me fait l'honneur de le présider, et **Dr. Amal TARIFA**, qui a accepté d'évaluer ce travail en tant que rapporteure. Je remercie également le laboratoire LARIA et le corps enseignant de l'ENSI, dont la formation constitue le socle sur lequel s'appuie ce mémoire.

Je remercie enfin mes camarades de promotion, pour les discussions et l'entraide de cette année, et les communautés open-source, Python, Cython, NumPy, SciPy, sans lesquelles conduire des expériences rigoureuses sur du matériel grand public n'aurait pas été possible.

À tous, mes sincères remerciements.

Résumé

Le Problème du Sac à Dos Multi-Objectif (MOKP) est un problème classique d'optimisation combinatoire NP-difficile, dont la résolution efficace constitue un défi méthodologique majeur pour les instances de grande taille et de forte dimensionnalité. Les métaheuristiques classiques reposent généralement sur une initialisation aléatoire et sur des hyperparamètres fixes, deux choix qui pèsent fortement sur le résultat mais restent souvent fixés à la main.

Ce mémoire prend pour point de départ un solveur éprouvé, *W-CMOLS* (Ben Mansour, Basseur et Saubion, 2018), et lui apporte deux contributions. La première remplace l'archive initialement vide par un *cloud* de solutions candidates, construit soit par une heuristique gloutonne, soit à partir de directions réparties dont les graines centrales sont améliorées par un modèle de langage (LLM), soit par leur union. Les trois variantes améliorent l'hypervolume sur les neuf instances, avec un gain moyen de +48,0 % à +52,0 % et jusqu'à +130,4 % sur 500.4 (variante union). La variante LLM dépasse la variante gloutonne sur 7 instances sur 9, et la variante union la dépasse sur les neuf. À dispositif identique, les décisions du modèle, qui exploitent le sac réellement saturé, apportent un gain propre faible mais mesurable (+0,34 % en moyenne).

La deuxième contribution confie le choix des quatre hyperparamètres du solveur, pour chaque instance, à un agent d'apprentissage par renforcement profond (DQN). L'agent prédit le temps de chaque configuration et écarte celles qui dépasseraient un budget fixé sous le temps de *W-CMOLS*. Sur un même cloud, la configuration apprise conserve presque l'hypervolume et réduit le temps d'exécution de 44 à 54 %. Le système final *SW-CMOLS-UQ* garde la qualité de l'union (+52,1 %) tout en étant plus rapide que le solveur de référence sur les neuf instances (−35 %). Une campagne de validation externe confronte enfin les systèmes obtenus à treize algorithmes de référence, avec leurs opérateurs conservés et un protocole de budget explicitement documenté. Les deux systèmes finaux obtiennent un hypervolume moyen supérieur dans les 117 comparaisons, toutes significatives au seuil nominal, et ce classement est confirmé par l'indicateur epsilon et par un contrôle de cardinalité.

Une analyse approfondie du solveur met enfin en évidence une propriété de sa configuration de référence : le balayage des vecteurs de poids employé par *W-CMOLS* y est tronqué, au point de ne jamais solliciter directement le premier objectif sur les instances à quatre objectifs. Cette propriété fournit un mécanisme explicatif cohérent avec la croissance observée du gain lorsque le nombre d'objectifs augmente, sans constituer à elle seule une preuve causale.

Mots-clés : optimisation multi-objectif, problème du sac à dos, amorçage informé, modèles de langage, apprentissage par renforcement, réseau Q profond (DQN), métaheuristiques, hypervolume, W-CMOLS.

Abstract

The Multi-Objective Knapsack Problem (MOKP), an NP-hard combinatorial optimization problem, poses a persistent methodological challenge as instances grow in size and dimensionality. Conventional metaheuristics typically rely on random initialization and on fixed hyperparameters, two choices that strongly affect the result but are often set by hand.

This work builds on the established *W-CMOLS* solver (Ben Mansour, Basseur and Saubion, 2018) and proposes two extensions. The first replaces the initially empty archive with a cloud of candidate solutions, built by a greedy heuristic, from evenly spread search directions whose central seeds are improved by a language model (LLM), or by their union. All three variants improve the hypervolume on all nine instances, with mean gains of +48.0% to +52.0% and up to +130.4% on 500.4 (union variant). The LLM variant outperforms the greedy one on 7 of 9 instances, and the union variant outperforms it on all nine. With everything else identical, the model's decisions, which exploit the knapsack that is actually saturated, bring a small but measurable gain of their own (+0.34% on average).

The second extension delegates the choice of the solver's four hyperparameters, for each instance, to a deep reinforcement learning agent (DQN). The agent predicts the runtime of each configuration and discards those that would exceed a budget set below the runtime of *W-CMOLS*. On the same cloud, the learned configuration nearly keeps the hypervolume and cuts runtime by 44 to 54%. The final system, *SW-CMOLS-UQ*, keeps the quality of the union cloud (+52.1%) while running faster than the reference solver on all nine instances (−35%). A separate campaign compares the resulting systems with thirteen reference algorithms under an explicitly documented protocol. Both final systems achieve a higher mean hypervolume in all 117 comparisons, all significant at the nominal level, and this ranking is confirmed by the epsilon indicator and by a cardinality control.

A close analysis of the solver finally highlights a property of its reference configuration: the weight-vector sweep stops before directly engaging the first objective on four-objective instances. This property provides a mechanism consistent with the observed increase in gains as the number of objectives grows, without constituting causal proof by itself.

Keywords: multi-objective optimization, multi-objective knapsack problem, informed seeding, language models, reinforcement learning, Deep Q-Network, metaheuristics, hypervolume, W-CMOLS.

Table des matières

Introduction générale 1
Chapitre 1 : Optimisation multi-objectif 4
1.1 Introduction 4
1.2 Du mono-objectif au multi-objectif 4
1.3 Dominance et optimalité de Pareto 5
1.4 Évaluer une approximation 6
1.4.1 Convergence et diversité 6
1.4.2 Les propriétés d'un indicateur 6
1.4.3 L'hypervolume 6
1.4.4 L'indicateur epsilon 7
1.4.5 Les autres familles d'indicateurs 7
1.5 Conclusion 8
Chapitre 2 : État de l'art 9
2.1 Introduction 9
2.2 Métaheuristiques : principe et classes 9
2.3 La recherche locale et la méthode de descente 10
2.3.1 Point de départ, voisinage et critère d'acceptation 10
2.3.2 Itération et arrêt de la descente 10
2.3.3 Le voisinage : les variantes de parcours 11
2.3.4 Sortir de l'optimum local : critère relâché ou départ varié 11
2.3.5 La descente appliquée à une population 12
2.4 Les métaheuristiques multi-objectif : trois familles 13
2.5 De l'initialisation aléatoire à l'amorçage informé 15
2.6 Apprentissage profond 15
2.7 Modèles de langage 17
2.7.1 Architecture, entraînement et génération 17
2.7.2 Apprentissage en contexte et propriétés critiques 17
2.7.3 Modèles de langage et optimisation combinatoire 18
2.8 Apprentissage par renforcement 18
2.8.1 Du processus de décision markovien au Q-learning 18
2.8.2 Deep Q-Network et ses extensions 20
2.8.3 Réglage automatique des paramètres d'une métaheuristique 20
2.9 Positionnement 21
2.10 Conclusion 22
Chapitre 3 : Le problème du sac à dos multi-objectif et le solveur W-CMOLS 23
3.1 Introduction 23
3.2 Le sac à dos multidimensionnel multi-objectif 23
3.3 Espace réalisable et optimalité de Pareto 24
3.4 Le solveur de référence : W-CMOLS 25
3.4.1 Origine du solveur 25
3.4.2 Vue d'ensemble de l'algorithme 26
3.4.3 Représentation, décodage et poids tournant 27
3.4.4 Fitness par indicateur, voisinage et archive 28
3.5 Analyse critique de la configuration de référence : le balayage tronqué des poids 29
3.6 Cadre expérimental commun 30
3.6.1 Instances 30
3.6.2 Mesure de qualité : l'hypervolume 31
3.6.3 Protocole statistique 32
3.6.4 Environnement 32

3.7 Conclusion . . . 32
Chapitre 4 : Amorçage de la recherche par modèle de langage (LLM) et heuristique gloutonne . . . 33
4.1 Introduction . . . 33
4.2 Le concept de cloud de solutions . . . 33
4.2.1 Définition . . . 33
4.2.2 Propriétés mesurables d'un cloud . . . 34
4.2.3 Diversité : espacement et étendue . . . 34
4.3 Approche proposée : SW-CMOLS . . . 35
4.3.1 Motivations . . . 35
4.3.2 SW-CMOLS-L . . . 35
4.3.3 SW-CMOLS-G . . . 40
4.3.4 SW-CMOLS-U et intégration au solveur . . . 40
4.4 Expérimentations . . . 41
4.4.1 Protocole expérimental . . . 41
4.4.2 Résultats, cardinalité et temps d'exécution . . . 42
4.5 Conclusion . . . 48
Chapitre 5 : Configuration par instance des hyperparamètres via un agent DQN . . . 49
5.1 Introduction . . . 49
5.2 Formulation : ce que l'agent observe, choisit et comment il est noté . . . 49
5.3 Approche proposée : SW-CMOLS-Q . . . 50
5.3.1 Prédire le temps d'une configuration . . . 51
5.3.2 Budget et configurations autorisées . . . 51
5.3.3 Architecture du réseau . . . 51
5.3.4 Entraînement . . . 52
5.4 Expérimentations . . . 54
5.4.1 Protocole expérimental . . . 54
5.4.2 Résultats, cardinalité et temps d'exécution . . . 54
5.5 Validation de l'agent . . . 59
5.5.1 Écart à un oracle empirique . . . 59
5.5.2 Généralisation à une instance nouvelle . . . 60
5.5.3 Apport de chaque mécanisme et comparaison à SMAC3 . . . 60
5.6 Conclusion . . . 61
Chapitre 6 : Comparaison avec les algorithmes de l'état de l'art . . . 62
6.1 Introduction . . . 62
6.2 Protocole expérimental . . . 62
6.3 Algorithmes comparés . . . 63
6.3.1 Décomposition et scalarisation . . . 63
6.3.2 Contraintes et apprentissage par renforcement . . . 64
6.3.3 Recherche locale dédiée : mobkp/PLS . . . 64
6.3.4 Références classiques : NSGA-II, SPEA2 et IBEA . . . 64
6.4 Résultats . . . 65
6.4.1 Comparaison avec SW-CMOLS-U . . . 65
6.4.2 Comparaison avec SW-CMOLS-UQ . . . 66
6.4.3 Fronts de MOEA/D et MOEA-D-2WA . . . 71
6.4.4 Résultats selon le nombre d'objectifs . . . 75
6.4.5 Contrôle de la taille des fronts . . . 76
6.4.6 Validation avec l'indicateur epsilon . . . 76
6.4.7 Temps d'exécution indicatifs . . . 77
6.5 Conclusion . . . 78
Conclusion générale . . . 79
Bibliographie . . . 81

Liste des figures

1.1 Dominance de Pareto : partition de l'espace des objectifs et front non dominé. 5
1.2 L'hypervolume : aire dominée par l'approximation, bornée par le point de référence. 7
2.1 La méthode de descente : optima locaux, bassin d'attraction et trajectoire depuis un point de départ. 11
2.2 Un réseau de neurones : couche d'entrée, couches cachées, couche de sortie. 16
2.3 Effet de la température sur la distribution d'échantillonnage du modèle de langage. 17
2.4 La boucle d'interaction d'un processus de décision markovien (définition 2.1). 19
3.1 Une itération de la boucle principale du solveur *W-CMOLS* (algorithme 3.1). 26
3.2 Le balayage tronqué des poids, relevé sur les fichiers que consomme le solveur. 30
4.1 Espacement et étendue : illustration géométrique sur nuages de points synthétiques. 35
4.2 Instance 250.2 (2 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 43
4.3 Instance 250.3 (3 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 43
4.4 Instance 250.4 (4 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 44
4.5 Instance 500.2 (2 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 44
4.6 Instance 500.3 (3 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 45
4.7 Instance 500.4 (4 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 45
4.8 Instance 750.2 (2 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 46
4.9 Instance 750.3 (3 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 46
4.10 Instance 750.4 (4 objectifs) : *W-CMOLS* et les trois variantes d'amorçage. 47
5.1 Architecture du réseau de l'agent : un tronc commun, puis quatre têtes *Dueling*, une par paramètre, qui… 52
5.2 Instance 250.2 (2 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 55
5.3 Instance 250.3 (3 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 55
5.4 Instance 250.4 (4 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 56
5.5 Instance 500.2 (2 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 56
5.6 Instance 500.3 (3 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 56
5.7 Instance 500.4 (4 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 57
5.8 Instance 750.2 (2 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 57
5.9 Instance 750.3 (3 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 57
5.10 Instance 750.4 (4 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. 58
6.1 Instance 250.2 (2 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 67
6.2 Instance 250.3 (3 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 67
6.3 Instance 250.4 (4 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 68
6.4 Instance 500.2 (2 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 68
6.5 Instance 500.3 (3 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 69
6.6 Instance 500.4 (4 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 69
6.7 Instance 750.2 (2 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 70
6.8 Instance 750.3 (3 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 70
6.9 Instance 750.4 (4 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes. 71
6.10 Instance 250.2 (2 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 71
6.11 Instance 250.3 (3 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 72
6.12 Instance 250.4 (4 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 72
6.13 Instance 500.2 (2 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 73
6.14 Instance 500.3 (3 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 73
6.15 Instance 500.4 (4 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 74
6.16 Instance 750.2 (2 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 74
6.17 Instance 750.3 (3 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 75
6.18 Instance 750.4 (4 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition. 75

Liste des tableaux

2.1 Les trois variantes de parcours du voisinage 11
2.2 Positionnement des travaux les plus proches par rapport aux deux contributions de ce mémoire 21
3.1 Configuration de référence du solveur *W-CMOLS* 27
3.2 Couverture du simplexe des poids effectivement atteinte en 100 itérations, par nombre d'objectifs 29
3.3 Les neuf instances de Zitzler et Thiele employées dans tout le mémoire 31
4.1 Criblage du nombre de directions, sans modèle de langage (écart d'hypervolume moyen au cloud glouton, 20… 36
4.2 Hypervolume moyen sur 50 répétitions, *W-CMOLS* et variantes de *SW-CMOLS* 42
4.3 Distribution des hypervolumes sur les trois instances à deux objectifs, 50 répétitions 42
4.4 Effet propre des décisions du modèle de langage : même dispositif avec et sans les 45 décisions, 50… 43
4.5 Cardinalité moyenne du front rendu par exécution et temps moyen par exécution, 50 répétitions 48
5.1 Hypervolume moyen sur 50 répétitions 54
5.2 Cardinalité moyenne du front rendu par exécution et temps moyen par exécution, 50 répétitions 58
5.3 Écart entre la configuration choisie par *SW-CMOLS-UQ* et l'oracle empirique issu du criblage des 432… 59
5.4 Validation croisée par exclusion d'une instance : neuf agents réentraînés depuis zéro sur huit… 60
5.5 Retrait, un par un, des deux extensions architecturales 60
6.1 Configuration expérimentale des treize concurrents 63
6.2 Hypervolume moyen (*SW-CMOLS-U*, *W-CMOLS* et treize concurrents : 20 répétitions, sans exception) 65
6.3 Hypervolume moyen (*SW-CMOLS-UQ*, *W-CMOLS* et treize concurrents : 20 répétitions, sans exception) 66
6.4 Gain moyen d'hypervolume sur *W-CMOLS*, regroupé par nombre d'objectifs (moyenne des trois instances de… 75
6.5 Hypervolume après réduction de nos fronts au nombre moyen de points de *MOEA-D-2WA* 76
6.6 Indicateur ε additif moyen sur les neuf instances, plus petit étant meilleur 77
6.7 Temps moyen par exécution, en secondes 78

Liste des algorithmes

2.1 Descente locale générique 10
2.2 NSGA-II, une génération 13
2.3 IBEA, sélection environnementale 14
2.4 IBMOLS, un cycle 14
2.5 MOEA/D, une génération 14
2.6 Deep Q-Network, cycle d'entraînement 20
3.1 W-CMOLS, boucle principale 26
3.2 Décodage d'une permutation en solution réalisable 27
3.3 Génération d'un voisin 28
4.1 Construction du cloud SW-CMOLS-L 38
4.2 Génération du cloud glouton 40
4.3 Construction du cloud union 40
4.4 Intégration d'un cloud au cycle de recherche du solveur 41
5.1 Mise à jour autorégressive avec PER 53
5.2 Choix de la configuration d'une instance par l'agent entraîné 53

Acronymes

W-CMOLS	*Cooperative Multi-Objective Local Search based on Weighted Epsilon*, noté W_ε-CMOLS par ses auteurs. Le solveur de référence (chapitre 3)
SW-CMOLS	*Seeded* W-CMOLS (S : *seeded*, amorcé), la famille de systèmes proposée, dont l'archive initiale est amorcée par un cloud de solutions
SW-CMOLS-G	W-CMOLS amorcé par le cloud glouton (G : glouton)
SW-CMOLS-L	W-CMOLS amorcé par le cloud LLM (L : LLM) : directions réparties, graines centrales améliorées par le modèle de langage
SW-CMOLS-U	W-CMOLS amorcé par l'union des deux clouds (U : union) : 50 % cloud glouton, 50 % cloud LLM
SW-CMOLS-Q	la famille des variantes dont les quatre hyperparamètres sont choisis par un agent DQN (Q : réseau Q profond). La deuxième contribution
SW-CMOLS-GQ	SW-CMOLS-G dont les quatre hyperparamètres sont choisis par un agent DQN entraîné avec le cloud glouton (Q : réseau Q profond)
SW-CMOLS-LQ	SW-CMOLS-L dont les quatre hyperparamètres sont choisis par l'agent de SW-CMOLS-UQ, sans nouvel entraînement
SW-CMOLS-UQ	SW-CMOLS-U dont les quatre hyperparamètres sont choisis par un agent DQN entraîné avec le cloud union. Le système complet
MOKP	*Multi-Objective Knapsack Problem*, problème du sac à dos multi-objectif
NP	*Non-deterministic Polynomial*. Un problème NP-difficile est au moins aussi difficile que tout problème de la classe NP
HV	Hypervolume, volume de l'espace des objectifs dominé par une approximation du front
$I_{\varepsilon+}$	Indicateur epsilon additif binaire [1]
GD	*Generational Distance*, distance générationnelle
IGD	*Inverted Generational Distance*, distance générationnelle inversée
LLM	*Large Language Model*, modèle de langage de grande taille
DQN	*Deep Q-Network*, réseau de neurones approximant la fonction de valeur Q
ReLU	*Rectified Linear Unit*, fonction d'activation $\max(0, x)$
PER	*Prioritized Experience Replay*, rejeu d'expérience priorisé
MDP	*Markov Decision Process*, processus de décision markovien
MOEA	*Multi-Objective Evolutionary Algorithm*
IBEA	*Indicator-Based Evolutionary Algorithm*
NSGA-II	*Non-dominated Sorting Genetic Algorithm II*
SPEA	*Strength Pareto Evolutionary Algorithm*
SPEA2	*Strength Pareto Evolutionary Algorithm 2*
VEGA	*Vector Evaluated Genetic Algorithm*
MOGA	*Multi-Objective Genetic Algorithm*
NPGA	*Niched Pareto Genetic Algorithm*
MOEA/D	*Multi-Objective Evolutionary Algorithm based on Decomposition*
MOEA/D-GR	*MOEA/D with Global Replacement*, MOEA/D à remplacement global
EAG-MOEA/D	*External Archive Guided MOEA/D*, MOEA/D guidé par une archive externe
C-MOEA-D	*Constrained MOEA/D*, MOEA/D avec traitement des contraintes
MOEA-D-2WA	*MOEA/D with Two-type Weight Adjustments*, MOEA/D à deux types d'ajustement des poids
GWASF-GA	*Global Weighting Achievement Scalarizing Function Genetic Algorithm*
I-DBEA	*Improved Decomposition-Based Evolutionary Algorithm*
DCNSGA-III	*NSGA-III* à transformation dynamique des contraintes
CMODRL	algorithme évolutionnaire contraint configuré par apprentissage par renforcement profond
IBMOLS	*Indicator-Based Multi-Objective Local Search*
GRASP	*Greedy Randomized Adaptive Search Procedure*
ILS	*Iterated Local Search*, recherche locale itérée
VNS	*Variable Neighborhood Search*, recherche à voisinage variable
GLS	*Guided Local Search*, recherche locale guidée
SMAC	*Sequential Model-based Algorithm Configuration*
SMAC3	troisième version de l'outil SMAC, employée au chapitre 5
LLMOA	algorithme d'optimisation hyper-heuristique assisté par un grand modèle de langage
ReEvo	*Reflective Evolution*, évolution d'heuristiques guidée par la réflexion d'un modèle de langage
DRLOS-EMCMO	sélection d'opérateur par apprentissage par renforcement profond (*DRL-assisted Operator Selection*) appliquée à l'algorithme EMCMO
PLS	*Pareto Local Search*, recherche locale de Pareto
NBL	nombre de vecteurs de poids parcourus (*number of weight lines*)
PlatEMO	*Platform for Evolutionary Multi-Objective Optimization*, plateforme MATLAB d'algorithmes multi-objectifs
PISA	*Platform and Programming Language Independent Interface for Search Algorithms*
FIFO	*First In, First Out*, file où le premier élément entré est le premier traité

Introduction générale

Un grand nombre de décisions, en ingénierie comme en gestion, reviennent à une même question : quel sous-ensemble d'éléments retenir, sous des ressources limitées, lorsque plusieurs critères s'opposent ? Sélectionner des projets sous contrainte de budget et de main-d'œuvre, ou, dans sa version à choix binaires, composer un portefeuille en arbitrant rendement et risque [2], relève de la même structure : le sac à dos multi-objectif (*Multi-Objective Knapsack Problem*, MOKP).

Sa formulation est simple à énoncer et difficile à résoudre : la version décisionnelle du cas mono-objectif est NP-complète, et son optimisation est NP-difficile [3]. Le passage au multi-objectif ajoute une difficulté d'une autre nature. Il n'existe plus de solution optimale unique, mais un ensemble de compromis incomparables, le front de Pareto, dont le cardinal peut croître exponentiellement avec la taille de l'instance. Résoudre signifie donc approcher *un ensemble* de réponses, aussi proche et représentatif que possible.

Face à cette double difficulté, la communauté s'est tournée vers les métaheuristiques, qui renoncent à la garantie d'optimalité au profit d'une approximation de bonne qualité en temps raisonnable. C'est dans cette lignée que s'inscrit le solveur *W-CMOLS* de Ben Mansour, Basseur et Saubion [4], qui combine une recherche locale à base d'indicateur avec un balayage de vecteurs de poids, et qui sert de point de départ à l'ensemble de ce travail.

L'attention portée à ces algorithmes s'est très majoritairement concentrée sur leur mécanique de recherche : opérateurs de variation, mécanismes de sélection, préservation de la diversité. Deux décisions les précèdent pourtant, et restent souvent fixées à la main.

La première est le point de départ. La plupart des métaheuristiques initialisent leur population par un tirage aléatoire uniforme, choix standard et commode, mais qui n'exploite aucune information sur l'instance traitée, ni les profits, ni les poids, ni leur rapport. Une propriété structurelle du MOKP donne des raisons d'en douter. Deux solutions voisines sur le front de Pareto peuvent différer par un grand nombre de variables. Une recherche locale partie d'une mauvaise région peut donc y rester confinée, non par manque d'itérations, mais parce qu'aucun mouvement améliorant ne l'en fait sortir.

La seconde est le réglage des hyperparamètres. Taille de population, nombre d'itérations, profondeur de voisinage sont fixés une fois pour toutes, identiques pour toutes les instances, alors même que les instances de référence vont de 250 à 750 objets et de deux à quatre objectifs. Un réglage adéquat pour l'une n'a aucune raison de l'être pour une autre.

Ces deux questions ont pris un relief nouveau avec l'émergence, depuis 2023, des modèles de langage (*Large Language Models*, LLM) comme générateurs de solutions candidates pour des problèmes combinatoires [5]. Un modèle capable de suivre une instruction en langage naturel pourrait, en principe, produire des solutions initiales informées sans réentraînement ni données étiquetées. Cette promesse appelle une vérification que la littérature récente fournit rarement : un modèle de langage apporte-t-il quelque chose qu'une heuristique classique n'apporterait pas déjà, plus vite et plus sûrement ?

Ce mémoire se propose donc de répondre à trois questions :

1. L'amorçage informé de la recherche améliore-t-il la qualité du front obtenu, et dans quelle mesure la source de cet amorçage importe-t-elle ?
2. La configuration par instance des hyperparamètres, apprise par renforcement, permet-elle de réduire le temps d'exécution du solveur sans perdre la qualité apportée par l'amorçage ?
3. Les systèmes qui en résultent tiennent-ils la comparaison face à des algorithmes dédiés de la littérature, sous un protocole documenté ?

Ces questions déclinent les objectifs du sujet. La génération de solutions candidates par un modèle de langage, affinées ensuite par une recherche locale, fait l'objet du chapitre 4, où le cloud amorce *W-CMOLS*. La comparaison

à des méthodes de l'état de l'art sur le banc de référence, en qualité comme en temps de calcul, fait l'objet du chapitre 6. Le comportement de l'approche lorsque la taille et le nombre d'objectifs croissent, de 250 à 750 objets et de deux à quatre objectifs, est étudié aux chapitres 4 à 6. La configuration par apprentissage par renforcement du chapitre 5 prolonge le sujet : elle vise le temps de calcul de la recherche locale.

Ce mémoire y répond par deux contributions méthodologiques, complétées par une validation externe et par un résultat de mécanisme non prévu au départ.

Un mécanisme d'amorçage par cloud de solutions (chapitre 4) remplace l'initialisation aléatoire du solveur par un ensemble de solutions candidates. Celui-ci est produit soit par une heuristique gloutonne balayant un ensemble structuré de vecteurs de poids, soit par un ensemble de directions réparties dont les graines centrales sont améliorées par un modèle de langage, soit par l'union des deux. L'étude qui l'accompagne isole chacune des trois sources, et mesure séparément la contribution propre du modèle de langage à dispositif identique.

Elle montre que l'amorçage informé est associé à la plus grande part du gain, et que la couverture du front par des directions en nombre suffisant en est le premier levier. Le modèle de langage y ajoute un gain propre, faible mais mesurable : en raisonnant sur le sac réellement saturé, il améliore des graines que le tri glouton ne peut pas améliorer.

Une configuration par instance des hyperparamètres (chapitre 5) confie le réglage des quatre paramètres du solveur à un agent d'apprentissage par renforcement profond (DQN). Un masque de décision fondé sur un modèle prédictif du coût exclut les configurations estimées hors budget, sans garantir le temps mesuré. Sur un même cloud, la configuration apprise conserve presque l'hypervolume des configurations statiques et réduit le temps d'exécution d'environ moitié, jusqu'à passer sous le temps du solveur de référence, sans réglage manuel ni balayage exhaustif au moment du déploiement, pour un coût de décision de l'ordre de la milliseconde, une fois les références de l'instance mesurées.

Une validation face à treize algorithmes concurrents (chapitre 6) confronte l'ensemble à des implémentations publiées. Leurs opérateurs et mécanismes natifs sont conservés, les adaptations se limitant aux interfaces, formats et, lorsque nécessaire, au critère d'arrêt. *SW-CMOLS-U* et *SW-CMOLS-UQ* obtiennent un hypervolume moyen supérieur dans les 117 comparaisons instance-concurrent. Le chapitre met en évidence une régularité sur le banc étudié : les six concurrents dont la moyenne dépasse celle du solveur de référence relèvent de la décomposition ou de la scalarisation, avec un traitement des contraintes adapté au problème. Cette association ne constitue pas une règle générale de performance.

L'étude de sa configuration de référence met enfin en évidence une propriété du solveur dans cette configuration (chapitre 3). Le balayage des vecteurs de poids y est tronqué dans sa configuration de référence, au point que le premier objectif ne reçoit jamais le moindre poids sur les instances à quatre objectifs. Plusieurs mesures convergentes montrent que cette propriété constitue un mécanisme explicatif cohérent avec deux résultats observés : la croissance du gain avec le nombre d'objectifs, et le gain obtenu lorsque le nombre d'itérations passe à 130. Elle n'établit pas à elle seule une relation causale.

Le chapitre 1 pose le cadre de l'optimisation multi-objectif : la dominance, le front de Pareto, et les indicateurs qui permettent de juger une approximation sans disposer de la solution exacte.

Le chapitre 2 dresse l'état de l'art des méthodes mobilisées. Il traite successivement des métaheuristiques et de la recherche locale, des familles multi-objectif auxquelles appartiennent les concurrents du chapitre 6, de l'amorçage informé d'une recherche, de l'apprentissage profond, des modèles de langage et de l'apprentissage par renforcement.

Le chapitre 3 formule le problème traité, décrit en détail le solveur de référence, analyse son implémentation et fixe le cadre expérimental commun à tout le mémoire.

Les chapitres 4 et 5 développent les deux contributions méthodologiques. Le chapitre 6 les valide face à l'état de l'art. Une conclusion générale reprend l'ensemble, énonce les limites et ouvre sur les perspectives.

Chapitre 1

Optimisation multi-objectif

1.1 Introduction

Ce mémoire étudie le problème du sac à dos multi-objectif (MOKP), défini au chapitre 3. Nous le résolvons par une recherche locale, à laquelle nous ajoutons un meilleur point de départ et un réglage automatique de ses paramètres. Ce chapitre présente les notions nécessaires pour évaluer les résultats.

En mono-objectif, deux valeurs sont directement comparables. En multi-objectif, une solution peut améliorer un objectif et en dégrader un autre : l'optimum unique est remplacé par un ensemble de compromis de Pareto. Comparer deux ensembles exige donc des indicateurs adaptés.

Le chapitre procède en trois temps : la formulation d'un problème multi-objectif (section 1.2), la dominance de Pareto (section 1.3), puis les indicateurs qui permettent de comparer deux approximations (section 1.4).

1.2 Du mono-objectif au multi-objectif

Un problème mono-objectif consiste à maximiser une fonction $f : X \to \mathbb{R}$ sur l'ensemble réalisable X. Comme $\mathbb{R}$ est totalement ordonné, deux valeurs de l'objectif sont toujours comparables.

Cette propriété disparaît dès que l'on poursuit plusieurs objectifs simultanément. Un problème d'optimisation multi-objectif s'écrit :

$$\text{maximiser} \quad f(x) = \Big(f_1(x), f_2(x), \dots, f_m(x)\Big), \qquad x \in X \tag{1.1}$$

avec $m \geq 2$. L'image d'une solution n'est plus un scalaire mais un vecteur de $\mathbb{R}^m$, et $\mathbb{R}^m$ ne possède pas d'ordre total naturel. Deux solutions dont l'une est meilleure sur le premier objectif et l'autre sur le second sont tout simplement incomparables : aucune information tirée du problème lui-même ne permet de les départager.

Toutes les définitions de ce mémoire suivent une convention de maximisation. Un objectif à minimiser peut être ramené à cette convention en changeant son signe.

Le moment où interviennent les préférences du décideur partage les méthodes du domaine en trois familles [6] :

- les méthodes *a priori* demandent au décideur d'exprimer ses préférences avant la résolution, qui se ramène alors à un problème mono-objectif,
- les méthodes *interactives* alternent phases de calcul et consultations du décideur,
- les méthodes *a posteriori*, auxquelles ce mémoire appartient, produisent d'abord l'éventail des compromis et laissent le choix pour la fin.

Le choix entre un portefeuille plus rentable et un autre moins risqué [2] illustre ce dernier cas : il relève d'un arbitrage que seul le décideur peut trancher, une fois les compromis connus. Un algorithme *a posteriori* n'est donc pas jugé sur un point, mais sur la qualité de l'ensemble qu'il rend.

Une approche classique ramène le cas multi-objectif au cas mono-objectif par agrégation pondérée :

$$\tilde{f}(x) = \sum_{k=1}^{m} \lambda_k f_k(x), \qquad \lambda_k \geq 0, \quad \sum_{k=1}^{m} \lambda_k = 1 \tag{1.2}$$

Elle est légitime lorsque les poids λ sont connus à l'avance, mais elle présente deux limites :

- une limite pratique : elle exige de fixer les préférences avant de connaître les compromis disponibles,
- une limite structurelle : quel que soit le vecteur de poids employé, la maximisation de $\tilde{f}$ ne peut atteindre que des solutions supportées, c'est-à-dire optimales pour au moins un vecteur de poids, situées sur l'enveloppe convexe supérieure de l'image de X. Les compromis non supportés, situés à l'intérieur de cette enveloppe, restent inatteignables par cette seule scalarisation. Ils sont fréquents dans les problèmes combinatoires, dont l'image est discrète.

La pondération reste utile pour orienter la recherche. Dans le solveur étudié, l'archive est cependant filtrée par dominance sur les objectifs bruts. Elle n'est donc pas limitée aux seules solutions optimales d'une somme pondérée (chapitre 3, section 3.4).

1.3 Dominance et optimalité de Pareto

La dominance de Pareto fournit l'ordre partiel utilisé en multi-objectif.

Définition 1.1 (Dominance)

Soient $x, y \in X$. La solution x domine y, noté $x \succ y$, si :

$$\forall k \in \{1, \dots, m\} : f_k(x) \geq f_k(y) \quad \text{et} \quad \exists k : f_k(x) > f_k(y) \tag{1.3}$$

c'est-à-dire si x fait au moins aussi bien que y partout, et strictement mieux quelque part. Lorsque ni $x \succ y$ ni $y \succ x$, les deux solutions sont mutuellement non dominées. La solution x domine faiblement y, noté $x \succeq y$, si $f_k(x) \geq f_k(y)$ pour tout k : elle fait au moins aussi bien partout, sans exiger d'amélioration stricte.

La dominance ne permet pas de comparer toutes les paires de solutions : c'est un ordre partiel. C'est ce qui rend l'optimisation multi-objectif difficile.

Définition 1.2 (Ensemble et front de Pareto)

L'ensemble Pareto-optimal rassemble les solutions que rien ne domine :

$$X^* = \{x \in X \mid \nexists\, y \in X : y \succ x\} \tag{1.4}$$

Son image $F^* = f(X^*) \subseteq \mathbb{R}^m$ est le front de Pareto. Une solution de X^* est dite *efficace*. Son image est dite *non dominée*. La figure 1.1 illustre ces notions.

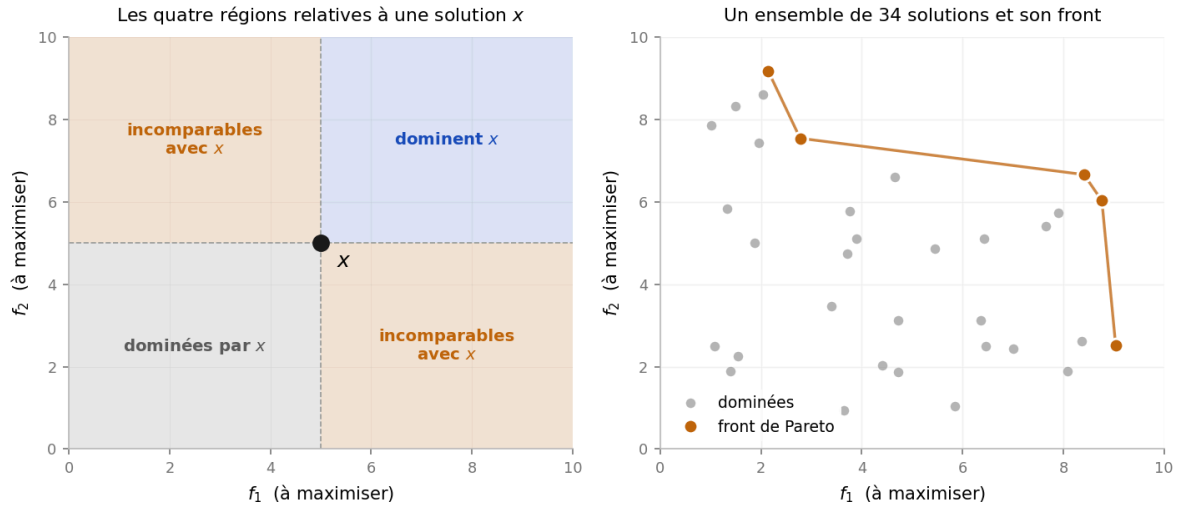


Fig. 1.1 : Dominance de Pareto : partition de l'espace des objectifs et front non dominé.

Les deux quadrants orange rassemblent les solutions que l'on ne peut ni préférer ni écarter. Points synthétiques à deux objectifs. Les fronts réels des chapitres 4 à 6 vont jusqu'à quatre objectifs.

Deux points décrivent théoriquement l'étendue du front. Le point idéal z^{id} rassemble les meilleures valeurs atteignables sur chaque objectif pris isolément, et le point de nadir z^{nad} les pires valeurs atteintes sur le front :

$$z_k^{\mathrm{id}} = \max_{x \in X} f_k(x), \qquad z_k^{\mathrm{nad}} = \min_{x \in X^*} f_k(x) \tag{1.5}$$

Le point idéal n'est généralement réalisé par aucune solution. En pratique, ces deux points exacts sont souvent inconnus. Le protocole du chapitre 3 utilise donc des bornes empiriques pré-calculées à partir de fronts de référence, élargies d'une marge puis figées pour tous les systèmes.

Une résolution exacte devrait produire X^*, dont la taille peut être exponentielle. On recherche donc une approximation $A \subseteq X$, proche du front et suffisamment étendue.

1.4 Évaluer une approximation

1.4.1 Convergence et diversité

Deux exigences définissent la qualité d'une approximation [7, 8].

La convergence mesure la proximité au front réel : les points de A sont-ils proches de F^* ? Une approximation peut être excellente sur ce critère tout en étant inutile, si tous ses points sont concentrés dans une même petite région.

La diversité mesure l'étendue et la répartition des points sur le front. Une approximation concentrée offre peu de choix au décideur, même si ses points sont proches de F^*.

Ces deux exigences sont en tension. Concentrer l'effort de recherche sur une région peut améliorer la convergence locale au détriment de la couverture. Disperser l'effort peut produire l'effet inverse. Les mécanismes présentés au chapitre 2 cherchent notamment à équilibrer ces deux exigences. En pratique, le front réel est inconnu sur nos instances : les indicateurs retenus (sections 1.4.3 à 1.4.5) évaluent ces deux exigences sans y faire appel.

1.4.2 Les propriétés d'un indicateur

Les indicateurs résument quantitativement la qualité des approximations [1].

Un indicateur est unaire s'il attribue une valeur à une approximation, et binaire s'il en compare deux. Pour deux approximations, A est dite meilleure que B si chaque point de B est faiblement dominé par au moins un point de A et que $A \neq B$. Un indicateur compatible avec la dominance doit alors favoriser A.

Zitzler et al. [1] montrent la limite des indicateurs unaires : aucun scalaire, ni aucune combinaison finie de scalaires, ne restitue toutes les relations de dominance possibles entre deux approximations. Deux fronts qui se croisent peuvent donc recevoir des valeurs proches sans être équivalents.

L'hypervolume est strictement compatible avec la dominance de Pareto. Il combine convergence et couverture, mais d'autres indicateurs restent utiles lorsque les fronts se croisent.

1.4.3 L'hypervolume

L'hypervolume mesure le volume de la région de l'espace des objectifs dominée par l'approximation, bornée par un point de référence r dominé par tous les points considérés :

$$HV(A, r) = \mathrm{Vol}\Big(\bigcup_{a \in A} [r, f(a)] \Big) \tag{1.6}$$

Avec le même point de référence, si A est meilleure que B (section 1.4.2), alors $HV(A, r) > HV(B, r)$. Améliorer un point qui reste dominé par un autre ne change pas l'hypervolume. Seule une extension de la région dominée l'augmente.

Un front éloigné perd du volume, tandis qu'un front laissant de grandes régions vides couvre moins d'espace. L'hypervolume est donc largement employé comme mesure conjointe de convergence et d'étendue.

Ses deux défauts sont connus :

- un défaut algorithmique : le coût de son calcul exact augmente rapidement avec le nombre de points et d'objectifs [9]. Ce mémoire utilise `pymoo`. Au-delà de 1 000 points, un amincissement déterministe précède le calcul, selon le protocole de la section 3.6. Les approximations stochastiques deviennent particulièrement utiles en dimension plus élevée [10],
- un défaut méthodologique : sa valeur dépend du point de référence choisi, de sorte que deux hypervolumes ne sont comparables que s'ils ont été calculés avec le même point et les mêmes bornes de normalisation.

C'est la raison pour laquelle le chapitre 3, section 3.6, fige ces bornes une fois pour toutes et les partage entre tous les systèmes comparés.

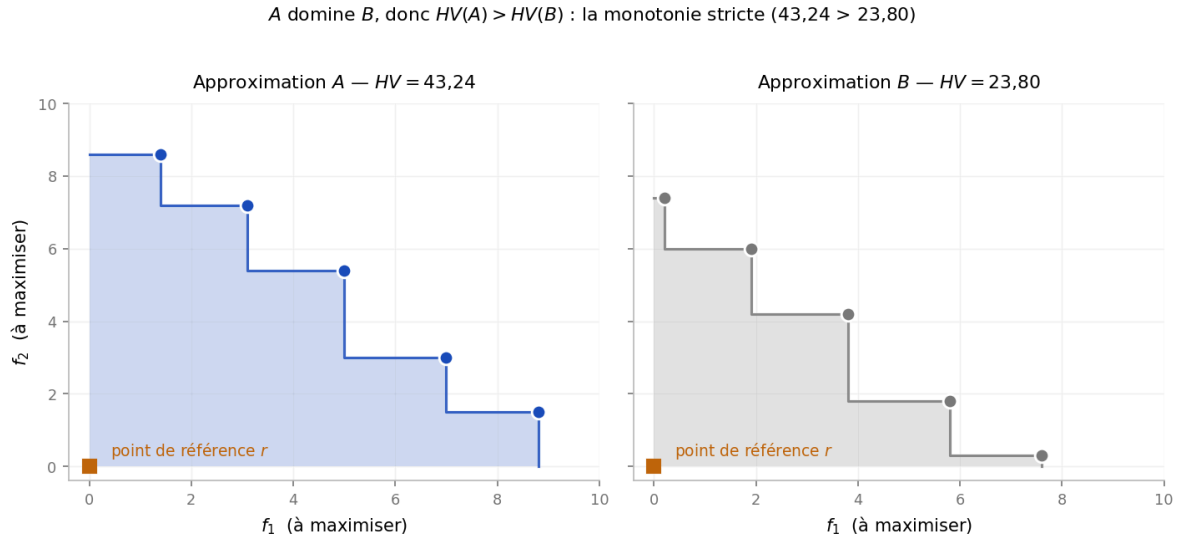


Fig. 1.2 : L'hypervolume : aire dominée par l'approximation, bornée par le point de référence.

La figure 1.2 illustre cette propriété : chaque point de B est dominé par au moins un point de A. Avec le même point de référence, l'hypervolume de B est donc inférieur à celui de A (23,80 contre 43,24).

1.4.4 L'indicateur epsilon

L'indicateur ε [1] mesure le facteur, additif ou multiplicatif, dont il faudrait améliorer une approximation pour qu'elle domine faiblement l'autre. Sa version binaire additive :

$$I_{\varepsilon+}(A, B) = \min_{\varepsilon} \{\, \forall b \in B, \exists a \in A : f_k(a) + \varepsilon \geq f_k(b), \forall k \,\} \qquad (1.7)$$

La lecture est directe : $I_{\varepsilon+}(A, B) \leq 0$ signifie que A domine faiblement B tout entier. Une valeur positive mesure la translation minimale encore nécessaire pour que A couvre B. Contrairement à l'hypervolume, qui agrège un volume, cet indicateur est déterminé par la région la plus mal couverte. Deux fronts de volume comparable peuvent donc obtenir des valeurs très différentes si l'un laisse un vide que l'autre couvre, ce qui en fait un complément et non un doublon de l'hypervolume.

Cet indicateur a deux rôles dans ce mémoire. Il sert d'abord de mécanisme : sous une forme pondérée, appliquée à deux solutions, il calcule la fitness des solutions dans la population du solveur, comme dans *IBEA* [11], dont s'inspire le solveur du chapitre 3 (section 3.4.4). Il sert ensuite de mesure : au chapitre 6, il compare le front de chaque système à un front de référence, en complément de l'hypervolume.

1.4.5 Les autres familles d'indicateurs

Les indicateurs de répartition mesurent la régularité de l'espacement entre points voisins, et l'étendue mesure l'ampleur de la région couverte. Une mesure inspirée du *spacing* de Schott [12], avec l'étendue, est employée au chapitre 4 pour décrire la répartition des archives finales, en complément de l'hypervolume et non comme mesure autonome de qualité.

La cardinalité compte les solutions non dominées mais ne mesure pas leur qualité. Elle indique le nombre de compromis offerts au décideur.

Les indicateurs de distance, comme la distance générationnelle (GD) et sa version inversée (IGD), mesurent l'écart entre une approximation et le front de Pareto exact. Ce front est inconnu pour nos instances. Ces indicateurs ne sont donc pas employés.

1.5 Conclusion

Ce chapitre a posé le cadre de mesure : une méthode *a posteriori* produit des compromis évalués par leur convergence et leur diversité. Aucun scalaire ne restitue toutes les relations entre fronts. L'hypervolume sert d'indicateur principal, l'indicateur ε complète l'analyse au chapitre 6, les mesures de répartition décrivent la régularité et l'étendue des fronts obtenus, et la cardinalité rend compte du nombre de compromis offerts au décideur.

Reste à savoir comment produire cette approximation. C'est l'objet du chapitre 2, qui présente les métaheuristiques, la recherche locale sur laquelle repose ce travail, et les deux familles de techniques d'apprentissage que nous lui associons.

Chapitre 2

État de l'art

2.1 Introduction

Le chapitre 1 a présenté l'optimalité de Pareto et les indicateurs de qualité. Sur les instances de grande taille étudiées ici, énumérer le front exact est impraticable : il faut produire une approximation, évaluée principalement par son hypervolume.

Ce chapitre parcourt, du général au particulier, les travaux dont ce mémoire hérite : les métaheuristiques et la recherche locale par descente (sections 2.2 à 2.4), l'amorçage informé (section 2.5), l'apprentissage profond (section 2.6), puis les deux familles de techniques d'apprentissage mobilisées, modèles de langage et apprentissage par renforcement, ainsi que le réglage automatique des paramètres (sections 2.7 et 2.8). La section 2.9 situe enfin les deux contributions par rapport aux travaux les plus proches.

2.2 Métaheuristiques : principe et classes

Un problème d'optimisation combinatoire possède un espace fini, mais sa taille croît rapidement : une sélection binaire sur n objets comporte 2^n possibilités, soit environ 10^{75} pour $n = 250$.

De nombreux problèmes combinatoires, dont le sac à dos, sont NP-difficiles [3, 13]. Les méthodes exactes garantissent l'optimalité. La méthode de séparation et évaluation (*branch and bound*) découpe l'espace des solutions en sous-problèmes, organisés en arbre, et calcule pour chacun une borne de la meilleure valeur atteignable : un sous-problème dont la borne est moins bonne que la meilleure solution déjà connue est abandonné sans être exploré. La programmation dynamique [14] résout le problème par étapes, objet par objet, en réutilisant les meilleures valeurs déjà calculées pour chaque capacité restante. Pour le sac à dos mono-objectif, son coût est proportionnel au nombre d'objets multiplié par la capacité [15]. Ces méthodes sont efficaces sur le sac à dos mono-objectif, mais le cas multi-objectif cumule les difficultés : il faut produire tout le front de Pareto, dont la taille peut croître exponentiellement, et chaque contrainte supplémentaire multiplie le coût de la programmation dynamique, qui doit suivre la capacité restante de chaque sac. Aux tailles considérées ici, de 250 à 750 objets avec deux à quatre objectifs et autant de contraintes, leur coût devient prohibitif.

Les métaheuristiques recherchent alors de bonnes solutions sans garantie d'optimalité. Leur structure est générale, mais la représentation, le voisinage et les opérateurs restent spécifiques au problème.

Les métaheuristiques se répartissent en deux grandes classes selon le nombre de solutions qu'elles maintiennent [16].

- Les méthodes à trajectoire font évoluer une solution unique par déplacements successifs. La descente sert de socle au solveur étudié. Le recuit simulé, la recherche tabou et *GRASP* ajoutent différents mécanismes d'échappement ou de redémarrage [17–19].
- Les méthodes à population font évoluer simultanément plusieurs solutions. Les familles représentatives sont les algorithmes évolutionnaires, les colonies de fourmis [20] et les essaims particulaires [21].

Le recuit simulé illustre les méthodes à trajectoire. Il est décrit avec la recherche tabou à la section 2.3.4. Les algorithmes évolutionnaires [22] illustrent les méthodes à population : sélection, croisement et mutation transforment simultanément plusieurs solutions.

Cette seconde classe présente un avantage naturel en multi-objectif : une population peut porter simultanément plusieurs compromis, là où une trajectoire unique doit être répétée ou complétée par une archive. Cela explique en partie la prédominance des approches évolutionnaires. Le solveur du chapitre 3 occupe une position intermédiaire : une descente appliquée à une population et adossée à une archive.

Toutes ces méthodes arbitrent entre intensification des régions prometteuses et diversification vers de nouvelles régions. Le chapitre 4 enrichit les points de départ. Le chapitre 5 adapte les paramètres qui contrôlent la recherche.

2.3 La recherche locale et la méthode de descente

La descente est une forme élémentaire de recherche locale à trajectoire. Elle est au cœur du solveur du chapitre 3. Cette section présente les trois choix qui la définissent (2.3.1), son déroulement (2.3.2), les façons de parcourir le voisinage (2.3.3), les moyens de sortir d'un optimum local (2.3.4) et son application à une population (2.3.5).

2.3.1 Point de départ, voisinage et critère d'acceptation

Une descente est entièrement déterminée par trois choix, et la qualité du résultat dépend de chacun d'eux.

- Le point de départ $x_0 \in X$, obtenu par tirage aléatoire ou par une heuristique constructive. Ce choix est celui sur lequel porte la première contribution de ce mémoire, et la section 2.5 lui est entièrement consacrée.
- Le voisinage $N : X \to 2^X$, qui associe à toute solution x l'ensemble $N(x)$ des solutions atteignables depuis x par une modification élémentaire. Le voisinage n'est pas donné avec le problème : c'est un choix de conception, et il détermine à lui seul la topologie de l'espace parcouru. Il est repris en détail à la section 2.3.3.
- Le critère d'acceptation décide si un voisin candidat remplace la solution courante. Dans la descente stricte, ce critère n'accepte que les améliorations. Le relâcher est l'une des deux façons d'échapper à un optimum local, développée à la section 2.3.4.

2.3.2 Itération et arrêt de la descente

La descente itère le même geste : engendrer un voisin, l'accepter s'il améliore, recommencer.

$$x_{t+1} = \begin{cases} y & \text{si } y \in N(x_t) \text{ et } F(y) > F(x_t) \\ x_t & \text{sinon} \end{cases} \qquad (2.1)$$

où F désigne la fonction évaluée, à maximiser. La recherche s'arrête lorsqu'aucun voisin de x_t ne l'améliore. La solution obtenue est alors un optimum local relativement à N :

$$x^\circ \text{ est un optimum local de } N \iff \forall y \in N(x^\circ) : F(y) \leq F(x^\circ) \qquad (2.2)$$

L'algorithme 2.1 formalise ce mécanisme.

Algorithme 2.1 (Descente locale générique)

```
Entrées : solution initiale x0 ∈ X, voisinage N
Sortie  : optimum local x°

x ← x0
tant qu'il existe y ∈ N(x) tel que F(y) > F(x) faire
    choisir un tel y, selon la variante de parcours retenue (section 2.3.3)
    x ← y
fin tant que
retourner x                        // aucun voisin de N(x) n'améliore x, cf. équation (2.2)
```

Être un optimum local n'est pas une propriété de la solution seule, mais du couple formé par la solution et le voisinage : élargir le voisinage, lorsque le nouveau voisinage contient l'ancien, ne peut pas augmenter le nombre d'optima locaux, mais augmente le coût de chaque itération.

Le bassin d'attraction d'un optimum local rassemble les points de départ qui y conduisent. Varier le voisinage ou le départ permet donc d'atteindre d'autres bassins. La figure 2.1 illustre ces notions.

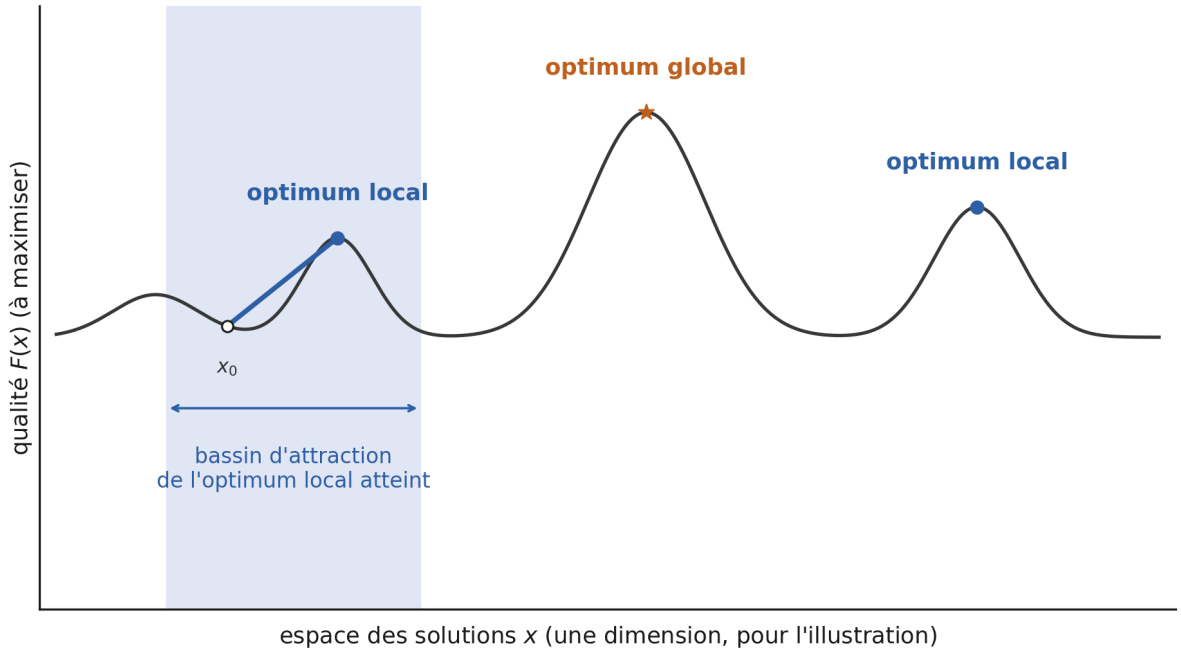


Fig. 2.1 : La méthode de descente : optima locaux, bassin d'attraction et trajectoire depuis un point de départ.

Le point de départ x_0 détermine le bassin exploré. La descente seule ne fournit aucune information sur la position de l'optimum global.

2.3.3 Le voisinage : les variantes de parcours

Deux variantes classiques se distinguent par la façon dont le voisinage est parcouru, et le choix entre elles n'est pas neutre. Une troisième s'impose lorsque l'énumération devient impraticable. Le tableau 2.1 les compare.

Variante	Parcours de $N(x)$	Coût par itération	Trajectoire	S'applique
Plus forte pente	$N(x)$ entier	$\lvert N(x)\rvert$ évaluations	déterministe	voisinage énumérable
Premier améliorant	1er améliorant rencontré	variable, moindre	dépend du parcours	voisinage énumérable
Échantillonnage aléatoire	sous-ensemble aléatoire	fixe (taille de l'échantillon)	stochastique	voisinage trop vaste

Tab. 2.1 : Les trois variantes de parcours du voisinage.

À budget égal, le premier améliorant effectue en général davantage de pas que la plus forte pente, pour une qualité souvent comparable.

Sur les instances de ce mémoire, le solveur du chapitre 3 échantillonne le voisinage, et l'ordre de grandeur explique ce choix. Retirer un objet parmi la moitié qui est sélectionnée, puis en insérer jusqu'à cinq parmi l'autre moitié, définit un voisinage d'environ 3×10^{10} éléments sur la plus petite instance et 2×10^{13} sur la plus grande. L'énumération exhaustive est donc impraticable. Le solveur ne génère qu'un nombre limité de mouvements aléatoires.

2.3.4 Sortir de l'optimum local : critère relâché ou départ varié

Une descente stricte ne peut sortir d'un optimum local lorsqu'elle refuse toute dégradation. Sa qualité finale dépend notamment de la structure du voisinage, du point de départ, du critère d'acceptation, du budget et du critère d'arrêt.

Une première voie relâche l'acceptation pour autoriser des dégradations contrôlées. Deux méthodes classiques la représentent.

Le recuit simulé [17] s'inspire du refroidissement lent d'un métal. Un voisin meilleur que la solution courante est toujours accepté. Un voisin moins bon est accepté avec la probabilité $e^{-\Delta/\theta}$, où $\Delta > 0$ mesure la dégradation et θ est un paramètre appelé température. Au début, la température est élevée : la méthode accepte souvent des dégradations et peut quitter un optimum local. La température baisse ensuite progressivement, et la méthode se comporte de plus en plus comme une descente.

La recherche tabou [18] passe à chaque itération au meilleur voisin, même s'il est moins bon que la solution courante. Pour ne pas revenir aussitôt à l'optimum qu'elle vient de quitter, elle garde en mémoire les derniers mouvements effectués, dans une liste dite tabou, et interdit de les annuler pendant quelques itérations. Cette interdiction est levée lorsqu'un mouvement tabou mène à une solution meilleure que toutes celles déjà rencontrées.

Une seconde voie conserve la descente stricte mais varie le départ. La descente à départs multiples lance plusieurs descentes depuis des solutions initiales différentes et garde le meilleur résultat. *GRASP* [19] répète deux phases. La phase de construction bâtit une solution élément par élément : à chaque pas, les candidats sont classés par un critère glouton, et l'élément ajouté est tiré au hasard parmi les mieux classés. La phase d'amélioration applique ensuite une descente à cette solution. Le critère glouton donne de bons départs, et le tirage au hasard les rend différents d'une répétition à l'autre (section 2.5).

Ce mémoire privilégie cette seconde voie : la mécanique centrale de recherche locale est conservée, tandis que le chapitre 4 modifie l'amorçage. Le chapitre 5 adapte ensuite les hyperparamètres sans remplacer les opérateurs de voisinage du solveur.

Trois autres méthodes combinent ces mécanismes. Elles situent le solveur étudié, mais ne sont pas utilisées dans les expériences.

La recherche locale itérée, ILS [23], enchaîne trois étapes. Une perturbation modifie le dernier optimum local assez fortement pour sortir de sa zone d'attraction. Une descente conduit ensuite à un nouvel optimum local. Un critère d'acceptation choisit enfin lequel des deux sert de point de départ suivant. La force de la perturbation est le réglage central : trop faible, la descente revient au même optimum. Trop forte, la méthode revient à des départs au hasard.

La recherche à voisinage variable, VNS [24], utilise plusieurs voisinages de portée croissante. Elle tire une solution dans le voisinage courant, lui applique une descente et compare le résultat à la solution courante. En cas d'amélioration, elle garde ce résultat et revient au voisinage le plus petit. Sinon, elle passe au voisinage suivant, plus large. L'idée est qu'un optimum local pour un voisinage ne l'est pas forcément pour un autre.

La recherche locale guidée, GLS [25], modifie la fonction évaluée plutôt que le voisinage. Chaque solution est décrite par des caractéristiques, par exemple la présence d'un objet donné dans le sac. À chaque optimum local, la méthode pénalise certaines caractéristiques de cet optimum, puis relance la descente sur la fonction pénalisée. L'optimum atteint devient moins attractif, et la recherche se déplace vers d'autres régions.

W-CMOLS emprunte à ces idées sans les reprendre telles quelles : son vecteur de poids, qui change à chaque itération, modifie la fonction évaluée, comme le fait *GLS*. Chaque itération repart d'une nouvelle population tirée de l'archive, comme une descente à départs multiples (chapitre 3).

2.3.5 La descente appliquée à une population

Une dernière remarque prépare la lecture du chapitre 3. Rien n'oblige une descente à porter sur une solution unique. Appliquée à une population de α solutions, elle conserve son schéma, mais le critère d'acceptation change de nature : un voisin n'est plus jugé sur sa propre qualité mais sur ce qu'il apporte à l'ensemble.

La recherche locale de Pareto, PLS [26], est la forme la plus directe de cette idée. Elle maintient une archive de solutions mutuellement non dominées et explore le voisinage de chacune. Tout voisin qu'aucune solution de l'archive ne domine y est ajouté, et les solutions qu'il domine sont retirées. La recherche s'arrête lorsque le voisinage de toutes les solutions de l'archive a été exploré sans ajout. PLS juge donc un voisin par la dominance. Elle figure parmi les concurrents du chapitre 6, dans une implémentation dédiée au sac à dos.

Une autre voie juge un voisin par un indicateur de qualité. C'est celle d'*IBMOLS* [27], dont s'inspire directement le solveur employé ici. Dans *W-CMOLS*, un voisin entre ainsi dans la population dès qu'il n'y est pas le moins bon selon cette fitness, même s'il est moins bon que la solution dont il est issu (chapitre 3). Elle combine la simplicité de la descente et la capacité d'une population à porter plusieurs compromis simultanément, ce qui la rend directement adaptée au cadre multi-objectif, objet de la section suivante.

2.4 Les métaheuristiques multi-objectif : trois familles

Transposer une métaheuristique au cas multi-objectif exige un mécanisme de sélection compatible avec un ordre partiel. La littérature distingue traditionnellement trois réponses, reprises notamment par Zhou et al. [28]. Le chapitre 6 utilise cette classification : sur nos instances, les concurrents fondés sur la décomposition ou la scalarisation obtiennent les meilleurs résultats, sans que cela constitue une règle générale.

- La famille dominance ordonne la population par rangs de dominance, puis la départage par un critère de diversité.
- La famille indicateur substitue à la dominance une quantité scalaire dérivée d'un indicateur de qualité.
- La famille décomposition décompose le problème en sous-problèmes scalaires, résolus conjointement en exploitant leur voisinage dans l'espace des poids.

La famille fondée sur la dominance s'est structurée à partir de plusieurs algorithmes fondateurs :

- *VEGA* [29] sélectionne des sous-populations selon un objectif chacune,
- *MOGA* [30] introduit le rang fondé sur le nombre de solutions dominantes,
- *NPGA* [31] lui préfère la sélection par tournoi entre solutions non dominées,
- *NSGA-II* [32] combine tri non dominé rapide et distance de surpeuplement,
- *SPEA2* [33] emploie une force calculée à partir du nombre de solutions dominées et une archive externe.

Leur limite est connue : le rang de dominance perd son pouvoir discriminant quand m croît, puisque la proportion de paires incomparables tend vers 1 (chapitre 3, section 3.3).

NSGA-II [32] regroupe parents et descendants, les répartit en fronts par rang de dominance, puis départage le dernier front admis par distance de surpeuplement. L'algorithme 2.2 en donne le déroulement.

Algorithme 2.2 (NSGA-II, une génération)

```
Entrées : population parent P_t de taille N
Sortie  : population suivante P_{t+1}

Q_t ← Croisement_Mutation(P_t)                  // descendance, taille N
R_t ← P_t ∪ Q_t                                  // taille 2N
(F_1, F_2, ...) ← Tri_non_dominé_rapide(R_t)       // fronts par rang de dominance
P_{t+1} ← ∅
i ← 1
tant que |P_{t+1}| + |F_i| ≤ N faire
    P_{t+1} ← P_{t+1} ∪ F_i
    i ← i + 1
fin tant que
si |P_{t+1}| < N alors
    calculer la distance de surpeuplement de chaque individu de F_i
    trier F_i par distance de surpeuplement décroissante
    P_{t+1} ← P_{t+1} ∪ (les N - |P_{t+1}| premiers de F_i)
fin si
retourner P_{t+1}
```

La famille fondée sur les indicateurs s'appuie sur *IBEA* [11], qui attribue une fitness scalaire calculée à partir d'un indicateur binaire, tel que ε. Cette fitness reste discriminante lorsque la dominance l'est peu. L'algorithme 2.3 en résume la sélection environnementale. Le chapitre 3 en reprend la formule.

Algorithme 2.3 (IBEA, sélection environnementale)

```
Entrées : parents P de taille N, descendants Q de taille N,
          indicateur binaire I, facteur d'échelle κ
Sortie  : population suivante P'

R ← P ∪ Q
pour chaque x ∈ R faire
    F(x) ← - Σ_{y∈R, y≠x} exp(- I(y,x) / κ)
fin pour
tant que |R| > N faire
    x* ← argmin_{x∈R} F(x)                          // retirer la plus petite valeur, donc la moins favorable
    retirer x* de R
    mettre à jour F des individus restants          // l'indicateur dépend de la population
fin tant que
P' ← R
retourner P'
```

Basseur et Burke [27] transfèrent cette fitness à une recherche locale sur population. *IBMOLS* explore le voisinage de chaque individu sans croisement ni sélection de parents. Il constitue l'ancêtre direct du solveur du chapitre 3. L'algorithme 2.4 en donne le principe.

Algorithme 2.4 (IBMOLS, un cycle)

```
Entrées : population P de taille N, indicateur binaire I, facteur d'échelle κ,
          fonction de voisinage N(·)
Sortie  : population suivante

Q ← ∅
pour chaque x ∈ P faire
    Q ← Q ∪ {un voisin de x tiré dans N(x)}
fin pour
R ← P ∪ Q
calculer F(x) pour tout x ∈ R
tant que |R| > N faire
    x* ← argmin_{x∈R} F(x)                          // retirer la plus petite valeur, donc la moins favorable
    retirer x* de R
    mettre à jour F des individus restants          // l'indicateur dépend de la population
fin tant que
retourner R
```

La famille décomposition trouve dans *MOEA/D* [34] son représentant canonique. Des vecteurs de poids définissent des sous-problèmes scalaires voisins, sans garantir une couverture uniforme du front. L'algorithme 2.5 en donne le principe. Le chapitre 3 reprend le balayage de poids, appliqué à un indicateur plutôt qu'à une scalarisation.

Algorithme 2.5 (MOEA/D, une génération)

```
Entrées : N sous-problèmes, vecteurs de poids λ_1,...,λ_N, voisinages B(1),...,B(N),
          fonction de scalarisation g (ex. Tchebychev pondérée),
          population initiale x_1,...,x_N, point idéal initial z
Sortie  : population x_1,...,x_N (une solution par sous-problème)

pour i = 1 à N faire
    tirer k, l dans B(i)
    y ← Croisement_Mutation(x_k, x_l)
    pour chaque objectif h faire
        si f_h(y) meilleur que z_h alors
            z_h ← f_h(y)                    // mise à jour du point idéal
        fin si
    fin pour
    pour chaque j ∈ B(i) faire
        si g(y | λ_j, z) ≤ g(x_j | λ_j, z) alors
            x_j ← y
        fin si
    fin pour
fin pour
retourner x_1,...,x_N
```

Ces familles ne sont pas exclusives, et le solveur du chapitre 3 en est l'illustration : il emploie la fitness par indicateur de la deuxième et le balayage de vecteurs de poids de la troisième.

Travaux dédiés au MOKP. Zitzler et Thiele [35] ont proposé les instances aujourd'hui utilisées comme référence, et les ont résolues avec l'algorithme évolutionnaire SPEA. L'article fondateur de *MOEA/D* [34] évalue aussi la décomposition sur ces instances. Du côté de la recherche locale, *W-CMOLS* [4] combine une fitness fondée sur un indicateur et un balayage de vecteurs de poids. C'est le solveur de ce mémoire (chapitre 3). Ben Mansour [36] propose une colonie de fourmis hybride pour le sac à dos, fondée sur une recherche multi-directionnelle, et la bibliothèque *mobkp* [37] regroupe des recherches locales de Pareto dédiées au sac à dos binaire multi-objectif. *MOEA/D*, *mobkp* et *W-CMOLS* figurent dans la comparaison du chapitre 6.

2.5 De l'initialisation aléatoire à l'amorçage informé

Le point de départ détermine le bassin d'attraction exploré par une descente (section 2.3.2). Un départ aléatoire n'exploite pourtant aucune information sur l'instance. Le chapitre 4 remplace donc l'initialisation aléatoire par un ensemble de solutions informées. Quatre familles d'approches situent ce choix :

1. Les heuristiques constructives déterministes trient les objets, souvent par rapport profit/poids, puis les insèrent sous contrainte de capacité [15]. En multi-objectif, les profits peuvent être agrégés par pondération. Varier les poids produit plusieurs compromis.
2. Les méthodes constructives probabilistes, notamment les colonies de fourmis [20], combinent information heuristique et mémoire collective. Elles produisent des solutions diverses mais sont plus coûteuses qu'une construction gloutonne simple. Elles ont été appliquées au MOKP [36].
3. Les constructions gloutonnes randomisées, telles que *GRASP* [19], tirent parmi une liste de bons candidats. La variante du chapitre 4 diversifie plutôt le critère de tri par un balayage de poids.
4. L'initialisation par apprentissage entraîne un modèle constructeur. Les réseaux à pointeurs ont appris à générer des tournées [38], puis l'apprentissage par renforcement a supprimé le besoin de solutions de référence [39]. Les architectures à attention ont ensuite amélioré cette approche [40].

Cette dernière famille déplace le coût de la résolution vers une phase d'entraînement préalable, et suppose soit un corpus d'instances résolues, soit un budget d'entraînement propre à chaque distribution d'instances.

Un modèle de langage, présenté à la section 2.7, offre une autre voie : il peut proposer ou modifier des solutions sans entraînement dédié, à partir d'une simple description du problème dans le prompt. Le chapitre 4 associe ainsi la première famille, un balayage glouton sur plusieurs directions, à des décisions d'un modèle de langage sur les graines centrales.

2.6 Apprentissage profond

Les deux sections suivantes présentent des techniques qui reposent l'une et l'autre sur le même instrument : un réseau de neurones profond. Le modèle de langage de la section 2.7 en est un, de très grande taille. L'agent de la section 2.8 en emploie un, beaucoup plus petit. Cette section présente les notions communes aux deux.

Un réseau de neurones [41] est formé de couches successives. Chaque neurone reçoit les sorties $x_1, \dots, x_d$ de la couche précédente et calcule :

$$y = \mathrm{ReLU}\Big(\sum_{i=1}^{d} w_i\, x_i + b\Big), \qquad \mathrm{ReLU}(z) = \max(0, z) \tag{2.3}$$

Les poids w_i et le biais b sont les paramètres appris. La fonction ReLU est une non-linéarité : sans elle, plusieurs couches successives se réduiraient à une seule transformation linéaire. La profondeur désigne le nombre de couches, qui apprennent des représentations intermédiaires des données (figure 2.2).

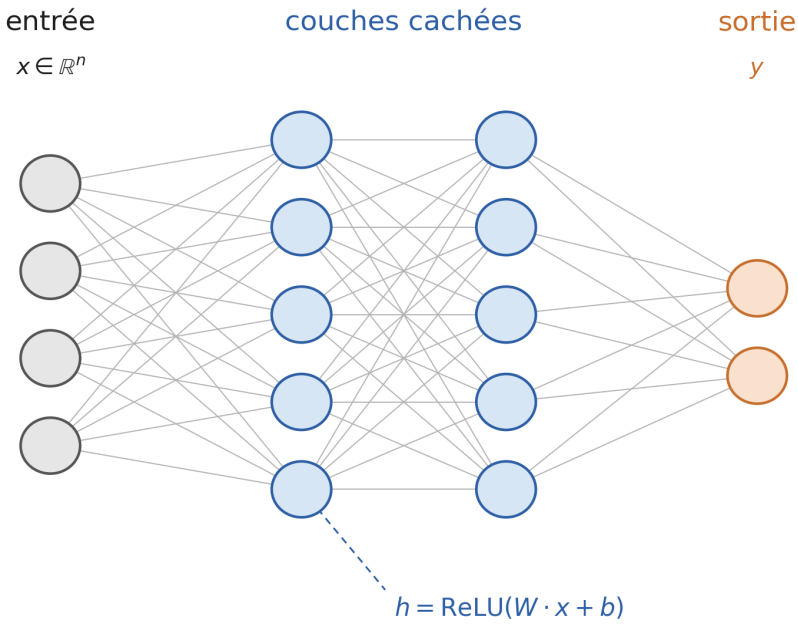


Fig. 2.2 : Un réseau de neurones : couche d'entrée, couches cachées, couche de sortie.

Chaque connexion porte un poids ajustable, et la couche de sortie fournit la prédiction du réseau. On note θ l'ensemble des poids du réseau. Une fonction de perte $\mathcal{L}(\theta)$ mesure l'écart entre les prédictions et les valeurs attendues, par exemple la moyenne des carrés des erreurs. La descente de gradient la diminue pas à pas :

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta) \qquad (2.4)$$

où η est le taux d'apprentissage. La rétropropagation calcule ce gradient couche par couche, de la sortie vers l'entrée. La perte est en général calculée sur un mini-lot, c'est-à-dire un petit nombre d'exemples tirés au hasard, plutôt que sur toutes les données à la fois. L'optimiseur *Adam* [42], employé au chapitre 5, ajuste en plus le pas de chaque poids à partir de la moyenne de ses gradients récents et de celle de leurs carrés.

La perte de Huber [43], employée elle aussi au chapitre 5, limite l'effet des erreurs isolées très fortes. Pour une erreur e et un seuil $\delta > 0$:

$$\mathcal{L}_\delta(e) = \begin{cases} \frac{1}{2} e^2 & \text{si } |e| \le \delta \\ \delta\left(|e| - \frac{1}{2}\delta\right) & \text{sinon} \end{cases} \qquad (2.5)$$

Elle est quadratique pour les petites erreurs et seulement linéaire pour les grandes. Le chapitre 5 l'utilise avec un seuil égal à 1.

La perte n'étant généralement pas convexe, la descente de gradient ne garantit pas l'obtention d'un optimum global. La qualité du modèle doit être vérifiée expérimentalement.

Un réseau ne reçoit que des nombres. Un choix parmi k valeurs est donc codé en *one-hot* : un vecteur de k composantes, toutes nulles sauf celle de la valeur choisie, qui vaut 1. Le chapitre 5 code ainsi les paramètres déjà choisis par l'agent.

Un réseau peut approximer une fonction sur un espace continu, contrairement à une table exhaustive. Cette propriété est utilisée par l'agent de la section 2.8.

Le mémoire emploie un Transformer préentraîné au chapitre 4 et entraîne une architecture *Dueling* plus petite au chapitre 5.

2.7 Modèles de langage

2.7.1 Architecture, entraînement et génération

Un modèle de langage est un modèle probabiliste sur des suites d'unités lexicales, ou *tokens*, entraîné à prédire l'unité suivante à partir des précédentes. Les modèles actuels reposent sur l'architecture Transformer [44], dont le mécanisme central est l'attention : chaque position de la séquence calcule une somme pondérée des représentations de toutes les autres, les poids étant appris et dépendant du contenu.

L'attention permet de relier directement des positions éloignées, par exemple une contrainte placée au début du prompt et un identifiant produit plus loin. Le calcul parallèle pendant l'entraînement facilite aussi le passage à l'échelle.

Le pré-entraînement apprend à prédire le symbole suivant sur un grand corpus. L'ajustement par instruction [45] adapte ensuite le modèle au suivi de consignes, ce qui permet de demander une solution dans un format imposé.

La génération est stochastique : à chaque étape le modèle produit une distribution sur le symbole suivant, dans laquelle on échantillonne. La température τ contrôle la dispersion de cette distribution : $\tau \to 0$ rend la génération quasi déterministe et répétitive, alors qu'une température élevée accroît la variété au prix de la cohérence. Au chapitre 4, cette variabilité est neutralisée : chaque graine reçoit une seule décision, consignée puis rejouée, si bien que le cloud ne dépend pas d'un nouvel échantillonnage.

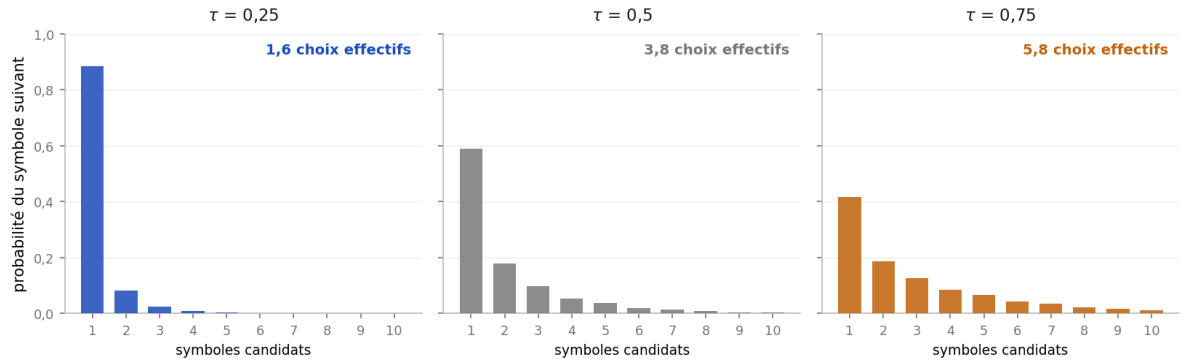


Fig. 2.3 : Effet de la température sur la distribution d'échantillonnage du modèle de langage.

La figure 2.3 applique trois températures à un même vecteur illustratif de dix scores. Le nombre effectif de choix, 2^H où H est l'entropie, augmente lorsque la distribution s'aplatit. Il s'agit d'une illustration du mécanisme, et non d'une mesure sur les sorties expérimentales.

2.7.2 Apprentissage en contexte et propriétés critiques

Une capacité importante des grands modèles est l'apprentissage en contexte [46] : présenter quelques exemples dans le prompt peut orienter le comportement sans mise à jour des paramètres. La réponse est alors conditionnée par le contenu du prompt, et non par un nouvel entraînement. Cette capacité fonde l'usage des modèles de langage comme composants d'optimisation sans réentraînement dédié : on décrit le problème, on montre le format attendu, puis on demande une solution.

Trois propriétés deviennent critiques dès lors que la sortie du modèle alimente une chaîne automatisée :

- L'hallucination est la production d'un contenu syntaxiquement plausible mais factuellement faux. Dans notre protocole, chaque identifiant renvoyé est vérifié (existence de l'objet, appartenance à la graine pour un retrait), puis la faisabilité est contrôlée avant intégration. Toute sortie reste donc une proposition à valider.
- Le non-déterminisme interdit de conclure d'une exécution unique : il impose soit de répéter les appels, soit de consigner les réponses et de les rejouer. Ce mémoire retient la seconde solution (chapitre 4).
- La conformité de format conditionne l'exploitation automatique de la réponse. Une réponse mal formée ou inexploitable doit être détectée. Dans notre protocole, la graine gloutonne est alors conservée.

Deux modes de déploiement s'offrent enfin : l'exécution locale d'un modèle à poids ouverts, ou l'appel à un modèle distant. Le premier est figé et identifiable par son empreinte. Le second peut être mis à jour ou retiré sans

préavis, mais offre souvent des capacités supérieures. Ce mémoire utilise un modèle distant, Claude [47], au chapitre 4. Pour préserver la reproductibilité, chaque prompt et chaque réponse sont consignés : la chaîne rejoue les décisions enregistrées et reconstruit le même cloud sans nouvel appel au modèle.

2.7.3 Modèles de langage et optimisation combinatoire

L'utilisation des modèles de langage en optimisation combinatoire s'est développée récemment selon deux directions.

Le modèle comme optimiseur itératif. Yang et al. [5] placent dans le prompt une liste de solutions déjà évaluées, triées par score, et demandent au modèle d'en proposer de nouvelles. Les nouvelles solutions sont évaluées, ajoutées à la liste, et le cycle recommence. Sur de petites instances du voyageur de commerce, le modèle améliore ainsi ses propositions, mais ses résultats se dégradent lorsque la taille de l'instance augmente. Le chapitre 4 limite au contraire le modèle à la construction des graines initiales.

Le modèle comme opérateur. Une seconde direction remplace ou guide un opérateur de génération ou de variation, à l'intérieur d'une métaheuristique.

LLMOA [48] est une hyper-heuristique : le modèle ne produit pas de solutions, il choisit la suite d'opérateurs de recherche à appliquer, parmi une bibliothèque d'opérateurs existants, en s'appuyant sur les résultats déjà obtenus.

ReEvo [49] fait écrire au modèle des heuristiques sous forme de code, dans une boucle évolutionnaire. Après chaque génération, le modèle compare les heuristiques performantes et les moins performantes, rédige une courte réflexion sur leurs différences, et s'en sert pour produire les heuristiques suivantes.

Mind Evolution [50] fait évoluer une population de solutions rédigées en langage naturel. Un évaluateur note chaque solution, et le modèle joue le rôle des opérateurs de croisement et de mutation : il combine et corrige les meilleures solutions. La méthode est appliquée à des problèmes de planification.

La contribution du chapitre 4 se distingue en limitant le modèle à l'amorçage, puis en le comparant à un amorçage glouton non génératif.

2.8 Apprentissage par renforcement

2.8.1 Du processus de décision markovien au Q-learning

L'apprentissage par renforcement traite d'un agent interagissant avec un environnement afin de maximiser une récompense cumulée [51]. À chaque instant t, l'agent observe un état, choisit une action, reçoit une récompense et se retrouve dans un nouvel état.

Définition 2.1 (Processus de décision markovien)

Un processus de décision markovien (MDP) est un quintuplet (S, A, R, P, γ) où :

- S est l'ensemble des états,
- A est l'ensemble des actions,
- $R : S \times A \to \mathbb{R}$ est la fonction de récompense,
- $P(s' \mid s, a)$ est la probabilité de transition de s vers s' sous l'action a,
- $\gamma \in [0, 1]$ est le facteur d'actualisation, qui pondère les récompenses futures relativement aux immédiates.

Le qualificatif markovien désigne l'hypothèse fondamentale : la transition ne dépend que de l'état et de l'action courants, non de l'historique. L'état doit donc résumer toute l'information utile à la décision, condition qui, dans une application donnée, relève d'un choix de modélisation à justifier (chapitre 5).

L'agent suit une politique $\pi(a \mid s)$, distribution sur les actions conditionnée à l'état. Son objectif est de maximiser le retour actualisé $G_t = \sum_{i \geq 0} \gamma^i R_{t+i}$. La figure 2.4 résume cette boucle.

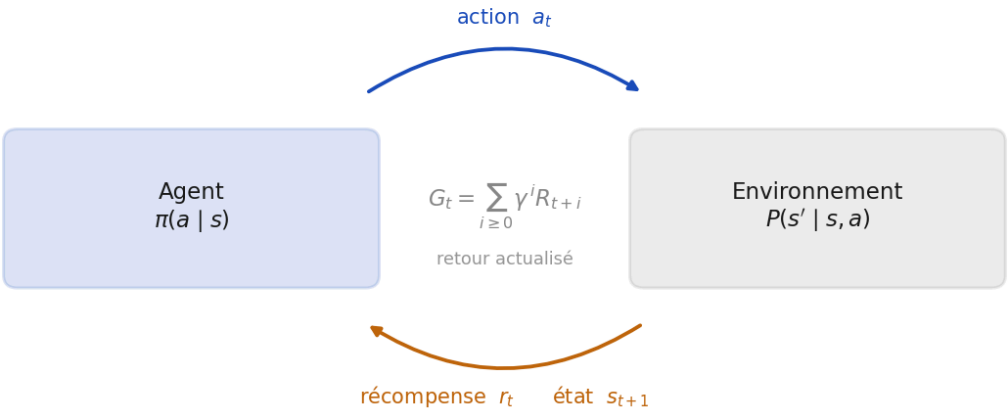


Fig. 2.4 : La boucle d'interaction d'un processus de décision markovien (définition 2.1).

Le facteur d'actualisation γ n'est pas un composant de la boucle mais le poids des récompenses futures dans le retour G_t.

Deux fonctions résument la qualité d'une politique. La fonction de valeur d'état $V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ donne le retour espéré depuis s. La fonction de valeur d'action, ou fonction Q, est celle qui nous intéresse :

$$Q^\pi(s,a) = \mathbb{E}_\pi\big[G_t \mid S_t = s,\ A_t = a\big] \qquad (2.6)$$

soit le retour espéré en exécutant a dans s puis en suivant π. Son intérêt pratique est direct : connaissant Q, agir de façon optimale se réduit à choisir $\arg\max_a Q(s,a)$, sans avoir à modéliser les transitions.

La fonction Q optimale satisfait l'équation d'optimalité de Bellman [14] :

$$Q^*(s,a) = \mathbb{E}\Big[R(s,a) + \gamma \max_{a'} Q^*(s',a')\Big] \qquad (2.7)$$

qui exprime une décomposition récursive : la valeur d'une action est sa récompense immédiate augmentée de la meilleure valeur atteignable depuis l'état suivant, actualisée. C'est le principe d'optimalité de la programmation dynamique.

Le *Q-learning* [52] estime Q^* par mise à jour incrémentale, à partir des seules transitions observées et sans modèle de l'environnement. Après chaque transition (s, a, r, s') :

$$Q(s,a) \leftarrow Q(s,a) + \eta\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{cible}} - Q(s,a)\Big] \qquad (2.8)$$

où η est le taux d'apprentissage, noté ainsi pour le distinguer de la taille de population α du chapitre 3. La quantité entre crochets est l'erreur de différence temporelle : l'écart entre l'estimation courante et une estimation améliorée par une observation réelle.

L'algorithme est hors politique : la cible emploie $\max_{a'}$, donc la politique optimale, quelle que soit la politique réellement suivie pour explorer. Cette propriété autorise l'apprentissage à partir de transitions passées, ce qu'exploite le rejeu d'expérience présenté plus bas.

Reste à choisir les actions pendant l'apprentissage. Toujours prendre la meilleure action connue enferme l'agent dans ses estimations initiales. Toujours explorer empêche de tirer parti de ce qui est appris. Ce compromis exploration-exploitation se traite classiquement par la stratégie ε-greedy : action aléatoire avec probabilité ε, meilleure action connue sinon, ε décroissant au fil de l'apprentissage. Ce ε est une probabilité d'exploration, sans lien avec l'indicateur ε du chapitre 1.

2.8.2 Deep Q-Network et ses extensions

Le *Q-learning* tabulaire stocke une valeur par paire (s, a), ce qui suppose des espaces discrets et petits. Dès que l'état comporte des composantes continues, cette représentation devient impraticable, et il faut approcher Q par une fonction paramétrée $Q(s, a; \theta)$.

Employer un réseau de neurones comme approximateur pose deux difficultés. Les transitions successives sont fortement corrélées, ce qui viole l'hypothèse d'indépendance des données sur laquelle repose la descente de gradient stochastique. De plus, la cible de régression dépend elle-même des paramètres courants, de sorte que le réseau poursuit une cible mouvante, configuration notoirement instable.

Le Deep Q-Network [53] emploie deux mécanismes. Le rejeu d'expérience entraîne le réseau sur des transitions échantillonnées dans une mémoire tampon, ce qui réduit leur corrélation temporelle. Un réseau cible, mis à jour moins fréquemment que le réseau principal, stabilise la cible

$$y = r + (1 - d)\,\gamma \max_{a'} Q(s', a'; \theta^{-}) \qquad (2.9)$$

où $d = 1$ si la transition est terminale et 0 sinon. L'algorithme 2.6 assemble ces mécanismes.

Algorithme 2.6 (Deep Q-Network, cycle d'entraînement)

```
Entrées : environnement, réseau principal Q(·,·;θ), réseau cible Q(·,·;θ⁻), tampon D
Sortie  : politique apprise (agir par argmax_a Q(s,a;θ))

θ⁻ ← θ
pour chaque épisode faire
    observer l'état initial s
    tant que l'épisode n'est pas terminé faire
        a ← ε-greedy(Q(s,·;θ))
        exécuter a, observer la récompense r et l'état suivant s'
        stocker (s,a,r,s',d) dans D
        (s_i,a_i,r_i,s'_i,d_i) ← lot tiré aléatoirement dans D
        cible_i ← r_i + (1-d_i)·γ·max_a' Q(s'_i,a';θ⁻)
        mettre à jour θ par descente de gradient sur Σ_i (cible_i - Q(s_i,a_i;θ))²
        si le nombre de pas est un multiple de C alors
            θ⁻ ← θ
        fin si
        s ← s'
    fin tant que
fin pour
retourner θ
```

Deux extensions de ce socle sont utilisées au chapitre 5 : l'architecture *Dueling* et le rejeu priorisé.

L'architecture *Dueling* [54] décompose la fonction de valeur d'action en une valeur d'état $V(s)$, commune à toutes les actions, et un avantage $A(s, a)$ propre à chacune, recombinées par :

$$Q(s, a) = V(s) + \left(A(s, a) - \text{moy}_{a'}\, A(s, a')\right) \qquad (2.10)$$

Cette décomposition améliore l'apprentissage lorsque la valeur d'un état est largement indépendante du choix précis de l'action.

Le rejeu d'expérience priorisé [55] échantillonne plus souvent les transitions associées à une forte erreur de différence temporelle. Des poids de correction compensent le biais d'échantillonnage.

Double DQN [56] réduit le biais de surestimation en utilisant le réseau principal pour sélectionner l'action et le réseau cible pour l'évaluer. Il est sans effet au chapitre 5, où la décision est unique.

Ces extensions ne traitent pas la structure de l'espace d'action. La factorisation autorégressive conditionne chaque composante d'une action sur les précédentes et évite d'énumérer toutes leurs combinaisons [57].

2.8.3 Réglage automatique des paramètres d'une métaheuristique

Cette section présente trois façons de régler automatiquement les paramètres d'une métaheuristique : les règles de contrôle classiques, la configuration hors ligne et l'apprentissage par renforcement. Elle situe le chapitre 5, qui relève de la troisième.

Eiben et al. [58] distinguent le réglage, fait avant l'exécution, et le contrôle, fait pendant l'exécution. Le contrôle est déterministe lorsque les paramètres changent selon une règle fixée à l'avance, par exemple en fonction du numéro d'itération. Il est adaptatif lorsqu'ils changent selon un retour d'information sur la recherche. Il est auto-adaptatif lorsqu'ils sont codés dans les solutions et évoluent avec elles. Ces méthodes restent souvent spécifiques à un algorithme et à un problème [59].

La configuration hors ligne traite les paramètres comme des variables d'un problème d'optimisation. SMAC [60] construit un modèle de substitution, qui estime la performance d'une configuration à partir des configurations déjà essayées. À chaque étape, il exécute l'algorithme avec la configuration que ce modèle juge la plus prometteuse, puis met à jour le modèle avec le résultat. La recherche doit donc être refaite pour chaque nouvelle instance. Le chapitre 5 le retient comme point de comparaison.

L'apprentissage par renforcement contrôle souvent le choix d'opérateur plutôt que des valeurs globales de paramètres. Pour le voyageur de commerce, Zheng et al. [61] emploient le *Q-learning* pour estimer la qualité d'arêtes candidates dans un algorithme génétique hybride. La recherche locale essaie alors en priorité les arêtes que l'agent juge utiles.

Deux travaux récents appliquent l'apprentissage par renforcement profond à l'optimisation multi-objectif sous contraintes. DRLOS-EMCMO [62] emploie un agent qui choisit, au fil des générations, l'opérateur de variation à appliquer selon l'état de la population. CMODRL [63] ajuste pendant l'exécution les hyperparamètres d'un algorithme évolutionnaire contraint. Il figure parmi les concurrents du chapitre 6. Dans les deux cas, l'agent agit pendant la recherche.

Le chapitre 5 choisit au contraire une configuration globale avant chaque exécution. L'épisode s'arrête après cette décision : la cible de l'équation (2.9) se réduit à la récompense ($d = 1$). Le problème est donc formellement un bandit contextuel, même si l'implémentation reprend l'architecture du DQN.

2.9 Positionnement

Le tableau 2.2 situe les travaux les plus proches par rapport aux deux contributions de ce mémoire. Il compare le problème traité, le rôle confié à l'apprentissage et le moment où celui-ci intervient.

Travail	Problème	Rôle de l'apprentissage	Multi-objectif	Moment d'intervention
Yang et al. [5]	Voyageur de commerce, petites instances	Le modèle de langage propose de nouvelles solutions	non	Pendant toute la recherche
LLMOA [48]	Optimisation mono-objectif	Le modèle de langage choisit la suite d'opérateurs	non	Pendant la recherche
ReEvo [49]	Problèmes combinatoires mono-objectif	Le modèle de langage écrit des heuristiques	non	Conception de l'heuristique
Mind Evolution [50]	Planification	Le modèle de langage croise et corrige les solutions	non	Pendant la recherche
SMAC [60]	Configuration d'algorithmes	Modèle de substitution, sans renforcement	selon l'usage	Avant l'exécution, recherche refaite par instance
Zheng et al. [61]	Voyageur de commerce	*Q-learning* : qualité des arêtes candidates	non	Pendant la recherche
DRLOS-EMCMO [62]	Multi-objectif sous contraintes	Renforcement profond : choix de l'opérateur	oui	Pendant la recherche
CMODRL [63]	Multi-objectif sous contraintes	Renforcement profond : réglage des hyperparamètres	oui	Pendant la recherche
Ce mémoire, chapitre 4	MOKP, deux à quatre objectifs	Le modèle de langage améliore les graines de l'amorçage	oui	Avant la recherche
Ce mémoire, chapitre 5	MOKP, deux à quatre objectifs	DQN : quatre hyperparamètres par instance	oui	Avant l'exécution, une seule décision

Tab. 2.2 : Positionnement des travaux les plus proches par rapport aux deux contributions de ce mémoire.

Parmi les travaux recensés ici, aucun ne limite le modèle de langage à l'amorçage d'un solveur multi-objectif, et aucun ne choisit la configuration d'un solveur multi-objectif une seule fois, avant l'exécution, à partir de caractéristiques de l'instance.

2.10 Conclusion

La descente constitue le mécanisme central du solveur employé ici. Ce travail agit principalement sur ses points de départ plutôt que sur son critère d'acceptation. Dans le cadre multi-objectif, les méthodes étudiées relèvent de trois familles (dominance, indicateur et décomposition), dont le solveur mobilise les deux dernières.

L'amorçage informé par apprentissage exige généralement un entraînement dédié. Les modèles de langage peuvent s'en dispenser par apprentissage en contexte, au prix de trois risques : hallucination, non-déterminisme et conformité de format. L'apprentissage par renforcement fournit ensuite le cadre de la configuration sous récompense, et le chapitre 5 applique les mécanismes du DQN au choix global des hyperparamètres.

Aucun des travaux recensés ne limite un modèle de langage à l'amorçage d'un solveur multi-objectif, ni ne choisit sa configuration une seule fois, avant l'exécution (section 2.9) : ce sont les deux contributions de ce mémoire. Le chapitre 3 présente d'abord le problème et le solveur *W-CMOLS*.

Chapitre 3

Le problème du sac à dos multi-objectif et le solveur W-CMOLS

3.1 Introduction

Les trois chapitres qui suivent partagent le même socle : un problème d'optimisation combinatoire précis, un solveur de référence unique et un protocole de mesure commun. Ce chapitre l'établit. La section 3.4 décrit le solveur tel qu'il est utilisé dans nos expériences, et distingue sa configuration de référence de l'amorçage introduit au chapitre 4.

3.2 Le sac à dos multidimensionnel multi-objectif

Dans le problème du sac à dos (*knapsack problem*), n objets possèdent chacun un poids et un profit. Il faut sélectionner le sous-ensemble qui maximise le profit total sans dépasser la capacité du sac. La version décisionnelle figure parmi les vingt-et-un problèmes dont Karp a établi la NP-complétude. La version d'optimisation est NP-difficile et constitue un modèle classique d'optimisation combinatoire [3, 15].

Deux généralisations successives conduisent au problème traité dans ce mémoire.

La première généralisation est multidimensionnelle : chaque objet consomme plusieurs ressources et la sélection doit respecter simultanément toutes les capacités.

La seconde est multi-objectif : m profits sont maximisés simultanément. Il n'existe alors généralement plus de solution optimale unique, mais un ensemble de compromis de Pareto (section 1.2).

Ces deux généralisations sont combinées dans la formulation dite « à sacs multiples », retenue par Zitzler et Thiele [35] et devenue depuis le jeu d'essai de référence du domaine. Chaque objectif k y est associé à son propre sac, doté de sa propre capacité c_k, et chaque objet i possède un poids $w_{k,i}$ et un profit $p_{k,i}$ spécifiques à ce sac. Le nombre de contraintes est donc égal au nombre d'objectifs, et une même décision d'inclusion agit simultanément sur les m objectifs et les m contraintes. C'est cette formulation, et elle seule, qui est utilisée dans tout ce mémoire.

Soit une instance définie par un nombre d'objets n, un nombre d'objectifs m, une matrice de profits $P = (p_{k,i})$ et une matrice de poids $W = (w_{k,i})$, avec $k \in \{1, \dots, m\}$ et $i \in \{1, \dots, n\}$.

Une solution est décrite par un vecteur de décision binaire :

$$x = (x_1, x_2, \dots, x_n) \in \{0, 1\}^n \qquad (3.1)$$

où $x_i = 1$ signifie que l'objet i est sélectionné et $x_i = 0$ qu'il ne l'est pas. Le caractère binaire des variables, un objet est pris entier ou pas du tout, distingue ce problème de sa relaxation continue, laquelle se résout en temps polynomial et ne présente pas la difficulté combinatoire qui nous intéresse.

Les m fonctions objectif sont linéaires et toutes à maximiser :

$$f_k(x) = \sum_{i=1}^{n} p_{k,i}\, x_i, \qquad k = 1, \dots, m \tag{3.2}$$

L'image d'une solution dans l'espace des objectifs est le vecteur $f(x) = \big(f_1(x), \dots, f_m(x)\big) \in \mathbb{R}^m$. C'est sur cet espace image, et non sur l'espace de décision, que sont définies la relation de dominance (définition 1.1, reprise à la section 3.3) et la mesure de qualité employée dans tout le mémoire (section 3.6).

À chaque objectif k est associée une contrainte de ressource, dite contrainte de capacité :

$$g_k(x) = \sum_{i=1}^{n} w_{k,i}\, x_i \;\le\; c_k, \qquad k = 1, \dots, m \tag{3.3}$$

où $c_k > 0$ désigne la capacité du sac k. Les m contraintes doivent être satisfaites simultanément : une solution qui respecte $m-1$ capacités et viole la dernière est irréalisable au même titre qu'une solution qui les viole toutes.

Deux propriétés de ces contraintes sont importantes pour la suite :

- Elles sont couplées par les variables. Les m contraintes portent sur les mêmes variables x_i : inclure un objet consomme simultanément de la capacité dans tous les sacs. Le problème ne se décompose donc pas en m sacs indépendants.
- La faisabilité est stricte. Toute capacité doit être respectée. Un algorithme doit soit ne produire que des candidats réalisables, comme *W-CMOLS* (section 3.4.3), soit réparer les candidats irréalisables, comme au chapitre 4.

Dans les fichiers employés, chaque capacité est fixée au voisinage de la moitié du poids total du sac :

$$c_k \simeq 0{,}5 \sum_{i=1}^{n} w_{k,i} \tag{3.4}$$

Ce réglage évite des sélections presque vides ou, à l'inverse, des contraintes peu actives. Sur les 27 capacités des neuf instances, 25 valent exactement la moitié du poids total et deux s'en écartent très légèrement (rapport entre 0,4998 et 0,5012).

En rassemblant les éléments précédents, le MOKP s'écrit :

$$\begin{aligned} \text{maximiser} \quad & f(x) = \Big(\sum_{i=1}^{n} p_{1,i} x_i,\ \dots,\ \sum_{i=1}^{n} p_{m,i} x_i \Big) \\ \text{sous contraintes} \quad & \sum_{i=1}^{n} w_{k,i}\, x_i \le c_k, \qquad k = 1, \dots, m \\ & x_i \in \{0,1\}, \qquad i = 1, \dots, n \end{aligned} \tag{3.5}$$

Complexité. Le MOKP contient le sac à dos 0/1 classique comme cas particulier ($m = 1$) et est donc NP-difficile [13]. Son espace comprend 2^n vecteurs binaires, soit environ $1{,}8 \times 10^{75}$ possibilités pour $n = 250$ et $5{,}9 \times 10^{225}$ pour $n = 750$. De plus, le nombre de solutions de Pareto peut croître exponentiellement. Ces propriétés motivent l'emploi de métaheuristiques et l'évaluation quantitative des fronts approchés (section 3.6).

3.3 Espace réalisable et optimalité de Pareto

Définition 3.1 (Espace des solutions réalisables)

L'espace des solutions réalisables d'une instance du MOKP est l'ensemble des vecteurs binaires satisfaisant l'intégralité des contraintes de capacité :

$$X = \Big\{ x \in \{0,1\}^n \;\Big|\; \sum_{i=1}^{n} w_{k,i}\, x_i \le c_k,\ \forall k \in \{1, \dots, m\} \Big\} \tag{3.6}$$

On note $\mathcal{B} = \{0,1\}^n \supseteq X$ l'espace binaire non contraint. L'inclusion est stricte lorsque les capacités sont actives : la sélection de tous les objets viole notamment les m contraintes, puisque chaque capacité vaut environ la moitié du poids total.

Cette distinction est reprise au chapitre 4 (définition 4.1) pour caractériser la validité d'un cloud. Un générateur non contraint, tel qu'un modèle de langage, produit des éléments de $\mathcal{B}$ sans garantir leur appartenance à X.

L'ensemble X n'est jamais vide : le vecteur nul $x = (0, \dots, 0)$ y appartient toujours, les capacités étant strictement positives. Il est en revanche impossible à énumérer en pratique.

Les définitions 1.1 et 1.2 (dominance, front de Pareto) s'appliquent ici sans modification, X désignant désormais l'espace réalisable de la définition 3.1 : la contrainte de capacité restreint le domaine sur lequel la relation opère, non la relation elle-même.

De cette relation découle une opération dont dépend l'ensemble du mémoire : filtrer un ensemble quelconque de solutions en n'en retenant que les éléments non dominés, sans jamais perdre d'information utile. Notée $\mathrm{extract_non_dominées}(\cdot)$ dans toute la suite, elle est le mécanisme par lequel le solveur maintient son archive (section 3.4.4), et c'est elle qui fonde la propriété de non-régression établie au chapitre 4 (propriété 4.1).

Propriété 3.1 (Idempotence et monotonie de l'extraction non dominée)

Soit A un ensemble fini de solutions et $\mathrm{ND}(A) = \mathrm{extract_non_dominées}(A)$ l'ensemble de ses éléments non dominés au sein de A. Alors $\mathrm{ND}(\mathrm{ND}(A)) = \mathrm{ND}(A)$, et pour tout ensemble B, $\mathrm{ND}(A \cup B)$ contient toute solution de $\mathrm{ND}(A)$ qui n'est dominée par aucun élément de B. Une solution de $\mathrm{ND}(A)$ n'est donc retirée après l'ajout de B que si un élément de B la domine.

Deux propriétés éclairent les résultats des chapitres suivants :

- La proximité dans l'espace des objectifs n'implique pas la proximité dans l'espace de décision. Deux solutions dont les images sont voisines dans $\mathbb{R}^m$ peuvent différer par de nombreux bits. Selon le voisinage retenu, passer de l'une à l'autre peut alors exiger plusieurs mouvements, dont certains sont refusés par une recherche strictement améliorante. Le point de départ peut ainsi conditionner la région du front atteinte, motivation centrale du chapitre 4.
- La dimension de l'espace des objectifs dégrade la sélectivité de la dominance. La probabilité que deux solutions tirées au hasard soient mutuellement non dominées croît rapidement avec m. Pour deux solutions aux objectifs indépendants, elle vaut $1 - 2^{1-m}$: 50 % pour $m = 2$, 75 % pour $m = 3$ et 87,5 % pour $m = 4$.

C'est pourquoi les méthodes à base d'indicateur conservent une information scalaire supplémentaire pour départager les solutions.

3.4 Le solveur de référence : W-CMOLS

3.4.1 Origine du solveur

Le solveur de référence est W_ε-CMOLS (*Cooperative Multi-Objective Local Search based on Weighted Epsilon*), proposé par Ben Mansour, Basseur et Saubion [4]. Le symbole W_ε désigne l'indicateur epsilon pondéré, C le cadre coopératif et MOLS la recherche locale multi-objectif. Le mémoire emploie l'écriture abrégée *W-CMOLS*.

Le solveur combine trois idées. Il reprend d'*IBEA* une fitness scalaire fondée sur un indicateur binaire [11], puis d'*IBMOLS* son emploi dans une recherche locale à population fixe [27]. Enfin, un vecteur de poids tournant modifie les objectifs avant le calcul de l'indicateur. Comme dans *MOEA/D* [34], plusieurs directions sont ainsi parcourues, mais chaque poids déforme ici l'espace des objectifs au lieu de définir un sous-problème scalaire distinct.

L'algorithme publié fait coopérer D sous-populations *IBMOLS*, chacune avec son poids et son archive locale. Leurs solutions non dominées alimentent une archive globale qui sert ensuite à les réinitialiser. Les auteurs emploient $D = 5$ dans leurs expériences.

L'implémentation du mémoire fixe $D = 1$. Une seule population de α individus est maintenue et le vecteur de poids change à chaque itération en parcourant une liste pré-générée (section 3.4.3). La coopération inter-populations disparaît donc et l'archive globale devient l'archive unique de la recherche.

Le nom *W-CMOLS* est conservé conformément au code hérité. Les mesures portent donc sur une recherche locale séquentielle à indicateur pondéré, et non sur le modèle coopératif en îlots.

3.4.2 Vue d'ensemble de l'algorithme

Avant d'entrer dans le détail de chaque composant, il est utile de disposer de la boucle complète. L'algorithme 3.1 en donne le schéma, et la figure 3.1 en illustre une itération. Les sections suivantes reprennent une à une les fonctions qu'il appelle.

Algorithme 3.1 (W-CMOLS, boucle principale)

```
Entrées : instance (n, m, P, W, c), liste de vecteurs de poids Λ,
          nombre d'itérations T, taille de population α, profondeur L,
          facteur d'échelle κ
Sortie  : archive finale A (approximation du front de Pareto)

A ← ∅
pour t = 1 à T faire
    λ ← Λ[(t-1) mod |Λ|]                    // balayage séquentiel cyclique (§3.4.3)
    P ← α individus tirés de A              // complétés aléatoirement si |A| < α
    A ← extract_non_dominées(A ∪ P)
    v ← pondération de P par λ              // v_k(x) = λ_k · f_k(x)
    calculer F(x) pour tout x ∈ P           // fitness indicateur ε+ (§3.4.4)
    E ← ∅                                   // archive de l'itération

    répéter                                 // recherche locale (§3.4.4)
        pour chaque x ∈ P non marqué « exploré » faire
            générer au plus n_pris(x) mouvements aléatoires (1, L)
            pour chaque candidat qui surclasse un membre de P selon la fitness faire
                remplacer le membre le moins favorable
            fin pour
            marquer la position traitée « explorée » à la fin du parcours
        fin pour
        (E, nouveaux) ← extract_non_dominées(E ∪ P)
    jusqu'à ce que nouveaux = 0
    A ← extract_non_dominées(A ∪ E)

fin pour
retourner A
```

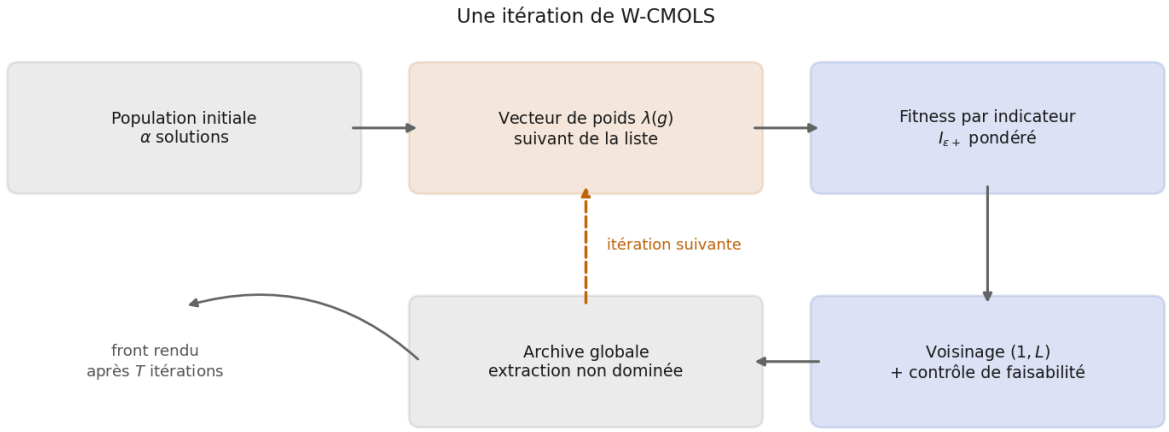


Fig. 3.1 : Une itération de la boucle principale du solveur *W-CMOLS* (algorithme 3.1).

À chaque itération, le poids oriente la fitness, le voisinage $(1, L)$ produit les candidats et l'archive conserve les solutions non dominées selon les objectifs bruts. Pendant la recherche locale, les solutions non dominées sont d'abord rassemblées dans une archive propre à l'itération, E, fusionnée dans l'archive globale à la fin de l'itération. La configuration de référence désactive toute mutation. Toutes les variantes proposées dans ce mémoire utilisent cette même configuration.

Les valeurs de la configuration de référence, celles de l'implémentation d'origine, employées telles quelles pour *W-CMOLS* de tout le mémoire, sont rassemblées au tableau 3.1.

Paramètre	Symbole	Valeur	Rôle
Itérations de la boucle principale	T (NBL dans le code)	100	nombre de vecteurs de poids parcourus
Taille de la population de travail	α	10	support du calcul de fitness
Profondeur du voisinage	L	5	insertions tentées par mouvement
Facteur d'échelle de la fitness	κ	0,05	raideur de la pression de diversité
Capacité maximale de l'archive	-	28 000	jamais saturée sur nos instances

Tab. 3.1 : Configuration de référence du solveur *W-CMOLS*.

Les quatre premiers paramètres constituent l'espace d'action de l'agent d'apprentissage du chapitre 5. Le suivant reste fixe.

3.4.3 Représentation, décodage et poids tournant

W-CMOLS manipule deux représentations d'une même solution, et la distinction entre les deux n'est pas cosmétique.

La représentation directe est le vecteur binaire $x \in \{0,1\}^n$ de la section 3.2. C'est sur elle qu'opèrent l'évaluation des objectifs, le test de faisabilité et l'opérateur de voisinage.

La représentation indirecte est une permutation π des indices $\{1, \dots, n\}$, utilisée uniquement à la création d'une solution. Elle est convertie en vecteur binaire par le décodeur glouton de type « premier ajusté » (algorithme 3.2).

Algorithme 3.2 (Décodage d'une permutation en solution réalisable)

```
Entrées : permutation π des n objets, poids w, capacités c
Sortie  : solution réalisable x ∈ X

x ← (0, ..., 0)
charge ← (0, ..., 0)
pour t = 1 à n faire
    i ← π(t)                                  // objet examiné à la position t
    si charge_k + w_{k,i} ≤ c_k  pour tout k = 1..m  alors
        x_i ← 1
        charge_k ← charge_k + w_{k,i}   pour tout k
    fin si                                    // sinon l'objet est simplement ignoré
fin pour
retourner x
```

Le décodeur ne produit que des solutions réalisables : chaque objet est ajouté après vérification des m contraintes. L'initialisation tire une permutation aléatoire puis la décode, sans réparation ni rejet.

Le décodage tend aussi à saturer la solution : l'aléa porte sur l'*ordre d'examen* des objets, non sur un taux de remplissage. Cette limite motive l'amorçage étudié au chapitre 4.

À chaque itération de la boucle principale, le solveur sélectionne un vecteur de poids

$$\lambda = (\lambda_1, \dots, \lambda_m), \qquad \lambda_k \geq 0, \qquad \sum_{k=1}^{m} \lambda_k = 1 \tag{3.7}$$

dans une liste prédéfinie, lue depuis un fichier et parcourue séquentiellement et cycliquement : l'itération t emploie le t-ième vecteur de la liste, et le parcours reprend au premier une fois la liste épuisée. Il ne s'agit donc ni d'un tirage aléatoire ni d'une adaptation en ligne, mais d'un balayage déterministe.

Ce vecteur définit les valeurs pondérées d'une solution :

$$v_k(x) = \lambda_k \cdot f_k(x), \qquad k = 1, \dots, m \tag{3.8}$$

Ce sont ces valeurs v, et non les objectifs bruts f, qui alimentent l'indicateur de la section suivante. L'effet est une déformation temporaire de l'espace des objectifs : à l'itération courante, la pression de sélection favorise la direction indiquée par λ. Sur une exécution suffisamment longue, le balayage vise à diversifier ces directions sur le simplexe. La couverture réellement obtenue dépend toutefois du budget et de l'ordre des vecteurs.

Les listes employées comportent respectivement 51, 626 et 1001 vecteurs pour $m = 2$, 3 et 4 objectifs. Elles sont ordonnées lexicographiquement : les premiers vecteurs concentrent tout le poids sur les derniers objectifs, et

les composantes des premiers objectifs ne deviennent non nulles que tardivement dans la liste. Cette propriété d'ordonnancement, anodine en apparence, est reprise et analysée à la section 3.5.

3.4.4 Fitness par indicateur, voisinage et archive

Le cœur du solveur est l'affectation de fitness reprise d'*IBEA*, appliquée aux valeurs pondérées.

Définition 3.2 (Indicateur ε+ binaire pondéré)

Soient a, b deux solutions et λ le vecteur de poids courant. L'indicateur $\varepsilon+$ binaire pondéré est :

$$I_{\varepsilon+}(a,b) = \frac{1}{v_{\max}} \max_{k\in\{1,\dots,m\}} \Big(v_k(a) - v_k(b)\Big) \qquad (3.9)$$

où $v_{\max} = \max_{x\in P,\,k} v_k(x)$ est la plus grande valeur pondérée observée dans la population courante P, utilisée comme facteur de normalisation.

Interprétation : $I_{\varepsilon+}(a,b)$ mesure le plus grand écart pondéré de a relativement à b. Une valeur strictement négative implique que b est meilleur que a sur toutes les composantes pondérées. Une valeur positive indique qu'au moins une composante favorise a, sans suffire à conclure à une dominance lorsque les autres composantes favorisent b. Avec les notations de l'équation (1.7), cette quantité correspond, à la pondération et à la normalisation près, à $I_{\varepsilon+}(b,a)$: l'ordre des arguments est celui du code du solveur, ce qui explique qu'une fitness plus faible y soit considérée comme meilleure (définition 3.3).

Définition 3.3 (Fitness d'un individu)

La fitness d'une solution x au sein d'une population P est :

$$F(x) = -\sum_{\substack{a\in P\\ a\neq x}} \exp\left(\frac{-I_{\varepsilon+}(a,x)}{\kappa}\right) \qquad (3.10)$$

où $\kappa > 0$ est un facteur d'échelle. Avec l'orientation de l'indicateur utilisée par cette implémentation de maximisation, les valeurs sont négatives et une valeur plus faible est considérée comme meilleure lors de la sélection.

Cette formulation conserve une discrimination scalaire lorsque plusieurs solutions sont mutuellement non dominées. Elle peut aussi favoriser des solutions complémentaires, sans garantir à elle seule la diversité. Enfin, κ règle la sensibilité des termes exponentiels et fait donc partie des hyperparamètres contrôlés au chapitre 5.

L'opérateur de voisinage est un échange déséquilibré, que nous désignons par mouvement $(1, L)$ et que détaille l'algorithme 3.3.

Algorithme 3.3 (Génération d'un voisin)

```
Entrées : solution x ∈ X, profondeur L
Sortie  : voisin y ∈ X

i ← objet sélectionné dans x, tiré uniformément parmi les objets pris
y ← x privée de l'objet i                    // libère de la capacité dans les m sacs
ajoutés ← ∅
répéter L fois
    j ← objet non sélectionné, tiré uniformément
    si j ∉ ajoutés  et  charge_k(y) + w_{k,j} ≤ c_k pour tout k  alors
        y ← y augmentée de l'objet j
        ajoutés ← ajoutés ∪ {j}
    fin si                                   // sinon la tentative est perdue
fin répéter
retourner y
```

Le mouvement retire donc exactement un objet et tente d'en insérer jusqu'à L, chaque insertion étant subordonnée à la vérification des m contraintes de capacité. Deux conséquences en découlent :

- Le traitement des contraintes est structurel. Chaque insertion est contrôlée. Aucune pénalité ni réparation a posteriori n'est nécessaire.

- Le mouvement est orienté vers la densification. Retirer un objet puis tenter d'en ajouter jusqu'à L tend à augmenter le nombre d'objets sélectionnés jusqu'à saturation des capacités.

La sélection d'un voisin ne repose pas sur une comparaison directe avec la solution courante. Le candidat y est évalué avec la population. S'il surclasse au moins un membre selon la convention de fitness précédente, le membre identifié comme le moins favorable est remplacé. Le noyau génère au plus $n_{\text{pris}}(x)$ mouvements aléatoires pour la position traitée : les objets retirés sont échantillonnés avec remise entre les tentatives, il ne s'agit donc pas d'une énumération du voisinage.

La population de travail P compte α individus (dix par défaut). La fitness et $v_{\max}$ sont calculés uniquement sur cette population, ce qui limite le coût quadratique du calcul.

L'archive globale A conserve les solutions non dominées rencontrées, dans une limite de 28 000 solutions jamais atteinte sur ces instances. Elle est mise à jour par $\text{extract_non_dominées}(\cdot)$.

L'archive filtre par dominance sur les objectifs bruts f, tandis que la population sélectionne par fitness sur les objectifs pondérés v.

Le poids n'intervient donc pas directement dans l'admission à l'archive, mais il influence les candidats visités. L'archive peut préserver un extrême déjà rencontré sans compenser une direction jamais explorée.

Au début de chaque itération, α individus sont tirés sans remise dans l'archive. Si elle en contient moins, les places restantes sont initialisées aléatoirement. Le chapitre 4 intervient à ce point : l'archive est préremplie par un cloud de solutions informées avant la première itération.

3.5 Analyse critique de la configuration de référence : le balayage tronqué des poids

Deux éléments décrits plus haut font apparaître une propriété de la configuration de référence : l'ordonnancement des listes de vecteurs de poids (section 3.4.3) et le budget de référence $T = 100$ (tableau 3.1). Cette analyse repose sur la lecture directe des trois fichiers de poids utilisés par le solveur.

Le balayage est séquentiel : il commence toujours au premier vecteur de la liste. Une exécution de T itérations n'emploie donc que les T premiers vecteurs, jamais les suivants. Avec $T = 100$:

- à $m = 2$, la liste compte 51 vecteurs : elle est parcourue en entier, et même presque deux fois,
- à $m = 3$, la liste compte 626 vecteurs : seuls les 100 premiers sont employés,
- à $m = 4$, la liste compte 1001 vecteurs : seuls les 100 premiers sont employés.

Ce qui compte n'est pas le nombre de vecteurs manqués, mais la portion du simplexe qu'ils couvrent. Le tableau 3.2 et la figure 3.2 la mesurent, par lecture directe des trois fichiers de poids.

m	**Vecteurs disponibles**	**Employés ($T = 100$)**	**Plage de λ_1 sur les vecteurs employés**	**Premier indice où $\lambda_1 > 0$**
2	51	51 (cycle complet)	[0,000 ; 1,000]	1
3	626	100	[0,000 ; 0,269]	25
4	1001	100	{0,000}	100

Tab. 3.2 : Couverture du simplexe des poids effectivement atteinte en 100 itérations, par nombre d'objectifs.

Valeurs obtenues par lecture directe des fichiers de poids employés par le solveur. La dernière colonne utilise l'indexation à partir de zéro du code : l'indice 100 correspond à la 101e ligne du fichier.

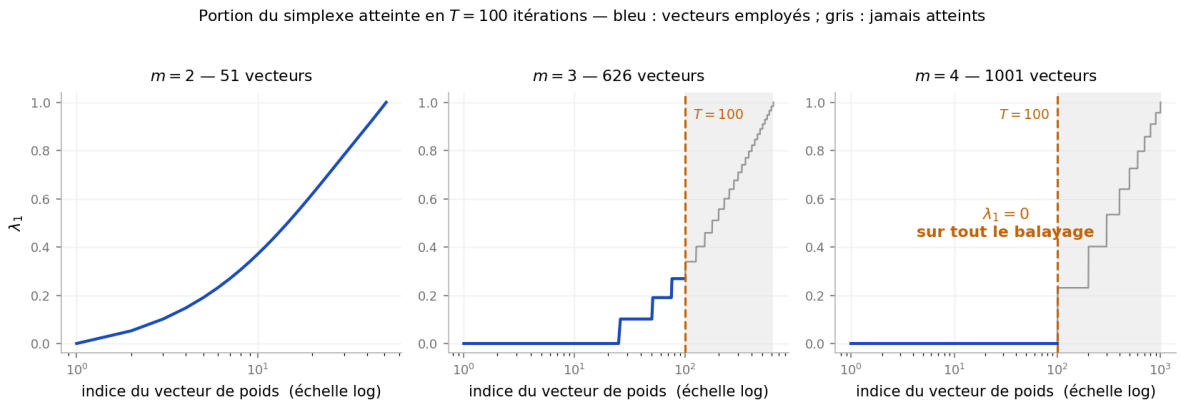


Fig. 3.2 : Le balayage tronqué des poids, relevé sur les fichiers que consomme le solveur.

En bleu, les vecteurs employés en $T = 100$ itérations. En gris, ceux jamais atteints. À $m = 4$, λ_1 reste nul sur toute l'exécution : son premier indice utile, 100, manque le budget d'une seule position.

Ces valeurs se déduisent de la méthode de génération des poids elle-même, dite des *poids graduels* (équation 3.11) :

$$\lambda_1(i) = \ln\left[\frac{4\,i\,\mathrm{e}}{FQ} + \cos\frac{2\pi i}{FQ}\right] \qquad (3.11)$$

FQ est le paramètre de fréquence du générateur de poids. Il vaut 200, 100 et 40 pour $m = 2$, 3 et 4 respectivement. Chaque boucle imbriquée parcourt $FQ/4$ valeurs avant l'ajout du vecteur extrême final. Comme λ_1 dépend de la boucle la plus externe, il reste nul pendant $(FQ/4)^{m-2}$ entrées. Les premiers indices où il devient strictement positif sont donc 1, 25 et 100, comme l'indique le tableau 3.2. Le solveur utilise une seule population et consomme ces poids un par un. C'est cette configuration, non la méthode de génération elle-même, qui crée la troncature. Un budget $T \geq |\Lambda|$ permettrait de parcourir toute la liste.

Conséquence directe : à $m = 4$, $v_1(x) = \lambda_1 f_1(x) = 0$ pour toute solution durant les 100 itérations de référence. L'objectif 1 ne contribue donc pas directement à la fitness pendant cette exécution, même si l'archive (section 3.4.4) peut conserver les bonnes solutions déjà rencontrées sur cet objectif. L'étude de sensibilité relève notamment un saut d'hypervolume de +41 % sur 250.4 entre $T = 100$ et $T = 130$, résultat cohérent avec l'entrée dans la portion où λ_1 devient non nul, sans constituer à lui seul une preuve causale.

Cette troncature fournit aussi une interprétation au fait que le gain de l'amorçage augmente avec le nombre d'objectifs (+12 %, +33 %, +113 %, résultats présentés au chapitre 6, tableau 6.4) : un cloud construit sur des directions réparties sur tout le simplexe apporte précisément les compromis que le balayage de référence n'atteint pas. Ces observations sont compatibles avec l'intérêt de couvrir davantage de directions. Elles n'isolent cependant pas ce mécanisme des autres effets de l'amorçage.

3.6 Cadre expérimental commun

3.6.1 Instances

L'ensemble du mémoire repose sur les instances de référence introduites par Zitzler et Thiele [35], largement employées dans les travaux sur le MOKP. Elles facilitent la comparaison avec la littérature lorsque les formulations, budgets et indicateurs coïncident. Le chapitre 6 précise ces conditions pour chaque concurrent.

Neuf instances sont retenues, croisant trois tailles et trois nombres d'objectifs (tableau 3.3). Cette grille permet d'examiner séparément les tendances associées à n et à m (section 6.4.4), sans prétendre à une attribution causale puisqu'une seule instance représente chaque combinaison.

Instance	Objets n	Objectifs m	Contraintes	Capacités c_k	Espace de décision
250.2	250	2	2	6 536 ; 6 489	2^{250}
250.3	250	3	3	7 184 ; 6 962,5 ; 6 635,5	2^{250}
250.4	250	4	4	6 819 ; 6 835,5 ; 7 098,5 ; 6 837	2^{250}
500.2	500	2	2	13 743,5 ; 13 357	2^{500}
500.3	500	3	3	13 320 ; 13 130 ; 13 956,5	2^{500}
500.4	500	4	4	13 499 ; 13 755 ; 13 454 ; 13 900	2^{500}
750.2	750	2	2	20 351,5 ; 20 460	2^{750}
750.3	750	3	3	20 945,5 ; 20 242 ; 21 312	2^{750}
750.4	750	4	4	20 622,5 ; 20 348,5 ; 20 450 ; 20 989	2^{750}

Tab. 3.3 : Les neuf instances de Zitzler et Thiele employées dans tout le mémoire.

Poids et profits sont des entiers tirés uniformément dans $[10, 100]$ et non corrélés. Les capacités suivent le rapport voisin de 0,5 vérifié à la section 3.2.

L'absence de corrélation empêche un rapport profit/poids unique de suffire. L'heuristique gloutonne du chapitre 4 balaie donc plusieurs directions.

3.6.2 Mesure de qualité : l'hypervolume

Comparer deux approximations du front de Pareto demande une mesure commune. L'hypervolume est retenu notamment pour sa monotonie stricte au sens de la dominance de Pareto : à point de référence fixé, si une approximation A domine strictement une approximation B, alors $HV(A) > HV(B)$ [1]. Cette propriété combine convergence et étendue dans une même mesure. Elle ne dispense toutefois ni de fixer une normalisation commune ni de documenter l'approximation numérique appliquée aux grands fronts.

L'hypervolume (section 1.4.3) est calculé dans tout le mémoire selon une procédure unique, appliquée à l'identique à tous les systèmes et à tous les concurrents externes.

Normalisation par des bornes unifiées. Chaque front est ramené dans $[0, 1]^m$ par des bornes (ℓ, h) propres à l'instance :

$$\hat{f}_k = \frac{f_k - \ell_k}{h_k - \ell_k}, \qquad \text{puis} \qquad \hat{f}_k \leftarrow \min\Big(\max(\hat{f}_k, 0), 1\Big) \tag{3.12}$$

Ces bornes sont dérivées une fois pour toutes de l'union de fronts de référence obtenus sur l'instance, élargie d'une marge de 1 %, puis figées et partagées par tous les systèmes comparés. Ce point est essentiel à l'équité de la comparaison : des bornes recalculées par système rendraient les hypervolumes incomparables entre eux, chaque système étant alors mesuré dans sa propre échelle. C'est en ce sens que le chapitre 6 parle de « bornes d'hypervolume unifiées ».

Point de référence. L'hypervolume est calculé sur le front normalisé, en maximisation, relativement au point de référence $r = (-0{,}01, \dots, -0{,}01)$, placé juste en dessous de la borne inférieure de chaque objectif. Ce décalage de 1 % au-delà des bornes garantit que les solutions situées exactement sur une borne contribuent malgré tout au volume, plutôt que d'être comptées pour zéro. L'hypervolume rapporté est ainsi une valeur adimensionnelle comprise entre 0 et $1{,}01^m$, comparable entre instances de même dimension.

Amincissement adaptatif. Le coût du calcul exact de l'hypervolume croît rapidement avec le nombre de points et d'objectifs, au point de devenir prohibitif sur certains fronts à quatre objectifs dépassant deux mille points. Au-delà de 1 000 points, le front est donc sous-échantillonné : les points atteignant le minimum ou le maximum de chaque objectif sont conservés, puis les places restantes sont tirées parmi les autres points. Le germe est dérivé du contenu et de la taille du front. Un même front produit ainsi le même sous-échantillon.

La conservation des minima et maxima limite la perte des points extrêmes. Le plafond de 1 000 points résulte d'un compromis mesuré sur un front à quatre objectifs d'environ 2 100 points : le calcul prend environ 4 s avec un écart relatif de 3,0 %, contre 12 s et 1,1 % à 1 500 points, puis 0,6 s et 6,3 % à 500 points. Comme ce calcul est répété pour de nombreux fronts, 1 000 points réduit nettement le coût tout en limitant l'erreur. Les hypervolumes amincis restent des estimations déterministes.

3.6.3 Protocole statistique

Les règles suivantes définissent le protocole principal. Les études exploratoires et les mesures de temps qui emploient un effectif différent l'indiquent explicitement dans leur chapitre.

Répétitions indépendantes. Chaque combinaison (système, instance) de l'évaluation principale des chapitres 4 et 5 est exécutée cinquante fois. Au chapitre 6, où nos systèmes sont confrontés aux concurrents externes, tous les hypervolumes reposent sur vingt répétitions. L'effet de ce passage de cinquante à vingt sur nos propres systèmes est vérifié par un contrôle dédié (section 6.2). Chaque exécution emploie un germe aléatoire distinct. Le criblage exploratoire du nombre de directions au chapitre 4 utilise vingt ou cinquante répétitions, tandis que certains temps d'exécution reposent sur des effectifs plus faibles, toujours signalés avec les valeurs concernées.

Test de Mann-Whitney U [64]. La comparaison de deux systèmes sur une instance repose sur un test de rangs unilatéral, au seuil de 0,05. Ce test non paramétrique est retenu parce que rien ne garantit la normalité des distributions d'hypervolume, et l'inspection montre des distributions dissymétriques, voire bimodales sur certaines instances à deux objectifs. Les chapitres 4 à 6 précisent les hypothèses testées et signalent explicitement l'absence de correction pour comparaisons multiples. Les tests doivent donc être lus instance par instance.

Significativité et ampleur. Sur cinquante répétitions à faible variance, un écart de quelques dixièmes de pour cent est statistiquement significatif sans être important : les résultats sont donc rapportés à la fois par le verdict du test et par le gain relatif.

3.6.4 Environnement

Le noyau du solveur est écrit en Cython [65] et compilé en code natif. L'orchestration, la génération des clouds et l'analyse utilisent Python 3.12. L'hypervolume est calculé avec pymoo [66], après l'amincissement décrit ci-dessus lorsque celui-ci est nécessaire. Les décisions du modèle de langage du chapitre 4 sont produites par Claude (Opus 5.5) [47]. Les prompts et les réponses sont conservés, puis rejoués lors de la construction du cloud.

Les concurrents du chapitre 6 s'exécutent dans leur environnement d'origine (PlatEMO sous MATLAB ou code source des auteurs en C et C++), sans modification de leurs opérateurs. Les adaptations de format et de budget, dont le remplacement du critère temporel de *MOEA/D* par son compteur d'évaluations, sont documentées à la section 6.2. Leurs temps sont indicatifs, car les environnements matériels et logiciels ne sont pas tous identiques.

Toutes les expérimentations internes ont été menées sur une machine de bureau unique sous Windows 11, ce qui rend les temps comparables entre systèmes internes au sein d'une même campagne. Les gains de temps du chapitre 5 ont en outre été vérifiés en remesurant tous les systèmes le même jour (section 5.4.2). Les temps des concurrents externes sont présentés séparément avec leur environnement d'exécution.

3.7 Conclusion

Ce chapitre a établi le socle commun aux deux contributions et à leur validation externe. Il a présenté le problème du sac à dos multidimensionnel multi-objectif dans la formulation de Zitzler et Thiele, ainsi que le solveur *W-CMOLS* qui sert de référence. Il a ensuite fixé le cadre de mesure commun : neuf instances, hypervolume à bornes unifiées et test de Mann-Whitney. Enfin, il a mis en évidence une propriété de la configuration de référence du solveur (section 3.5).

La description précédente fait apparaître trois limites qui structurent la suite du mémoire. Premièrement, l'archive est initialement vide et la première population est produite à partir de permutations aléatoires. Le décodeur de la section 3.4.3 en garantit la faisabilité, sans exploiter les profits ni les poids. Le chapitre 4 remplace cet amorçage non informé par un cloud de solutions candidates.

Deuxièmement, la configuration de référence fixe les mêmes hyperparamètres pour toutes les instances, alors que la section 3.5 montre que l'effet de T dépend notamment de m et de l'ordre des poids. Le chapitre 5 en confie le choix à un agent d'apprentissage par renforcement. Troisièmement, battre *W-CMOLS* ne suffit pas à se situer dans la littérature : le chapitre 6 confronte donc les variantes à des solveurs externes.

Chapitre 4

Amorçage de la recherche par modèle de langage (LLM) et heuristique gloutonne

4.1 Introduction

Ce chapitre présente la première contribution de ce mémoire : *SW-CMOLS*, une extension de *W-CMOLS* [4] amorcée par un ensemble de solutions candidates construit par un modèle de langage (LLM), par une heuristique gloutonne ou par leur union. Le système de référence démarre avec une archive vide et construit sa première population à partir de permutations aléatoires. Nous étudions si une initialisation informée améliore le front final sans modifier le cœur de la recherche locale.

Trois variantes sont évaluées séparément : un cloud glouton (*SW-CMOLS-G*), un cloud dont les graines centrales sont améliorées par un modèle de langage (*SW-CMOLS-L*) et leur union à parts égales (*SW-CMOLS-U*). Chaque expérience utilise un seul de ces clouds. Le système de référence n'en utilise aucun. Tester chaque cloud séparément permet d'attribuer un écart de résultat à sa source : ce qu'apporte le modèle de langage, ce qu'apporte le glouton, et ce que leur union ajoute.

L'étude compare les trois sources d'amorçage sur neuf instances. Elle mesure leur effet sur l'hypervolume, la stabilité, la diversité, la cardinalité du front et le temps d'exécution. Un contrôle dédié isole la contribution propre du modèle de langage, à dispositif identique.

4.2 Le concept de cloud de solutions

4.2.1 Définition

Le cloud de solutions prolonge le mécanisme d'archive de *W-CMOLS* (chapitre 3, section 3.4). Dans la configuration de référence, la première population de travail est issue de permutations aléatoires. Avec amorçage, le cloud préremplit au contraire l'archive avant la sélection de cette population.

Définition 4.1 (Cloud de solutions)

Soit X l'espace des solutions réalisables d'une instance du MOKP (défini au chapitre 3, section 3.3) et $\mathcal{B} \supseteq X$ l'espace binaire des vecteurs de sélection d'objets. Un cloud de solutions est une collection finie de solutions candidates. Sa représentation peut conserver des doublons :

$$C = (x_1, x_2, \dots, x_q) \in \mathcal{B}^q \qquad (4.1)$$

Un cloud est dit valide si $x_i \in X$ pour tout $i \in \{1, \dots, q\}$, c'est-à-dire si chacune de ses solutions satisfait les contraintes de capacité. Dans la chaîne étudiée, toute modification proposée par le modèle de langage est vérifiée et réparée avant son intégration au solveur. Cette définition reste indépendante du processus générateur.

Le cloud intervient une seule fois, avant la première itération. L'archive de départ est formée des solutions non dominées du cloud, soit $A_0 = \text{extract_non_dominées}(C)$. La population de travail de taille α est ensuite

sélectionnée dans cette archive. Sur les neuf instances étudiées, l'archive non dominée extraite du cloud contient au moins α solutions et fournit donc directement cette première population, sans complément aléatoire. Tout le reste du solveur est inchangé.

Propriété 4.1 (Non-régression de l'amorçage)

Soit P une archive et C un cloud de solutions valide. L'opérateur $\mathrm{extract_non_dominées}(\cdot)$ ne retient, parmi $P \cup C$, que les solutions qui ne sont dominées par aucune autre solution de cet ensemble. Pour l'hypervolume exact, calculé sur l'archive complète avec une normalisation et un point de référence fixes, on a :

$$HV\Big(\mathrm{extract_non_dominées}(P \cup C)\Big) \;\geq\; HV(P) \qquad (4.2)$$

Autrement dit, l'injection d'un cloud ne peut détériorer l'hypervolume exact de l'archive au moment où elle a lieu. Cette propriété ne dit rien de l'effet du cloud sur la *trajectoire* de la recherche locale qui suit. Savoir si un cloud améliore le front *final*, après toutes les itérations, relève donc de l'expérimentation (section 4.4.2).

4.2.2 Propriétés mesurables d'un cloud

Un cloud de solutions se caractérise selon quatre dimensions : sa cardinalité, son taux de faisabilité, sa diversité et son coût de génération. Les deux premières se définissent directement. La diversité demande une mesure précise.

Cardinalité. C'est le nombre $q = |C|$ de solutions du cloud, doublons compris.

Taux de faisabilité. C'est la proportion des solutions d'un cloud C satisfaisant les contraintes de capacité de l'instance (définies au chapitre 3, section 3.2) :

$$\tau(C) = \frac{1}{q} \sum_{i=1}^{q} \mathbf{1}[x_i \in X] \qquad (4.3)$$

où X est l'ensemble des solutions réalisables. Les trois clouds de ce chapitre ont un taux égal à 1 : le remplissage glouton n'insère jamais un objet qui violerait une capacité, et les modifications du modèle de langage sont réparées avant intégration.

4.2.3 Diversité : espacement et étendue

Pour quantifier la diversité d'un ensemble de solutions, nous reprenons deux mesures complémentaires de la littérature, d'ordinaire appliquées au front de Pareto rendu par un algorithme. C'est aussi notre usage à la section 4.4.2, sur l'archive finale de chaque variante.

Soit $C = \{y_1, \dots, y_q\}$ l'ensemble considéré, représenté dans l'espace des objectifs et normalisé dans $[0,1]^m$ par les bornes de l'instance, comme pour l'hypervolume (chapitre 3).

Espacement. Pour chaque point y_i, on définit sa distance à son plus proche voisin au sens de la norme L_1 :

$$d_i = \min_{j \neq i} \sum_{k=1}^{m} |y_i^k - y_j^k| \qquad (4.4)$$

L'espacement est alors l'écart moyen absolu de ces distances à leur moyenne $\bar{d}$:

$$\mathrm{Esp}(C) = \frac{1}{q} \sum_{i=1}^{q} |d_i - \bar{d}|, \qquad \bar{d} = \frac{1}{q} \sum_{i=1}^{q} d_i \qquad (4.5)$$

Cette mesure s'inspire de l'indicateur de spacing introduit par Schott [12]. Un espacement faible signifie que les solutions sont régulièrement réparties. Un espacement élevé signale des amas de points séparés par de grands vides.

Étendue. L'étendue mesure l'ampleur de la région de l'espace des objectifs couverte, comme la norme L_2 du vecteur des étendues par objectif :

$$\mathrm{Ete}(C) = \|e\|_2\,, \qquad e_k = \max_i y_i^k - \min_i y_i^k \quad (k = 1, \dots, m) \qquad (4.6)$$

Les deux mesures sont complémentaires : un ensemble peut être uniformément réparti tout en restant confiné à une petite région, ou couvrir une large région de façon irrégulière. La figure 4.1 les illustre sur des nuages synthétiques en 2 dimensions.

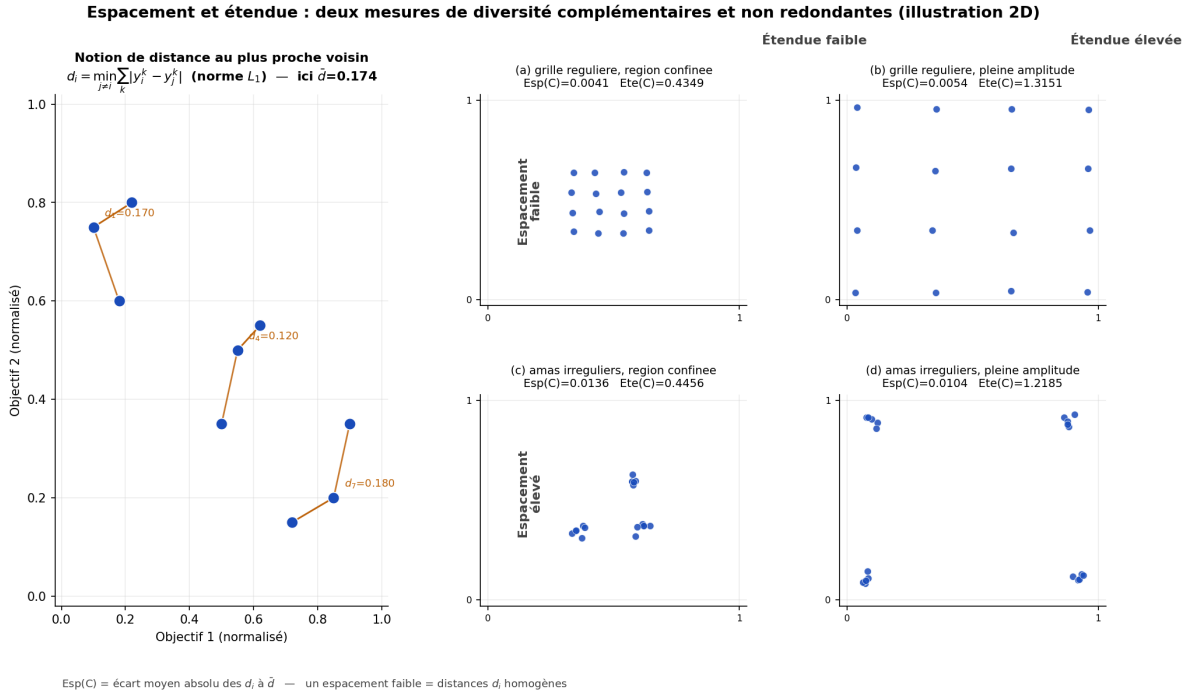


Fig. 4.1 : Espacement et étendue : illustration géométrique sur nuages de points synthétiques.

Gauche : distance au plus proche voisin d_i (norme L_1) sous-jacente à la mesure d'espacement. Droite : les quatre combinaisons espacement/étendue, avec Esp(C) et Ete(C) calculés directement sur les points affichés.

La dernière dimension, le coût de génération, ne demande pas de formalisation dédiée : c'est le temps nécessaire à la production du cloud.

4.3 Approche proposée : SW-CMOLS

4.3.1 Motivations

La conception suit quatre principes :

- couvrir tout le front : le cloud est construit autour de directions de recherche réparties sur le simplexe des poids,
- ne pas déléguer la faisabilité : le modèle de langage ne produit pas de solution complète, il propose des retraits et des ajouts sur une solution gloutonne valide, et le code vérifie le résultat,
- concentrer le modèle sur les graines de compromis, où tous les objectifs se disputent les capacités : les cinq graines centrales de chaque instance,
- plafonner tous les clouds à 1 510 solutions, pour ne pas confondre l'effet de la source avec celui d'un budget plus grand.

4.3.2 SW-CMOLS-L

La variante *SW-CMOLS-L* construit son cloud en sept étapes :

1. choisir N directions de recherche,
2. construire une graine gloutonne par direction,
3. choisir les cinq graines centrales,
4. soumettre chacune au modèle de langage par un prompt,
5. appliquer la décision du modèle,

6. vérifier le résultat,
7. entourer chaque graine de voisins.

Étape 1 : directions de recherche. Une direction est un vecteur de poids λ de somme 1. Pour m objectifs, on choisit N directions réparties sur le simplexe :

- deux objectifs : N directions régulièrement espacées de $(1,0)$ à $(0,1)$, $\lambda_j = \left(1 - \frac{j}{N-1},\ \frac{j}{N-1}\right)$ pour $j = 0, \dots, N-1$. Avec $N = 30$, $\lambda_{13} = (16/29\,;\,13/29) \approx (0{,}552\,;\,0{,}448)$,
- trois ou quatre objectifs : une grille régulière de pas $1/H$, formée de tous les vecteurs dont les composantes sont des multiples de $1/H$.

Une telle grille contient

$$N = \binom{H+m-1}{m-1} \qquad (4.7)$$

directions. Pour trois et quatre objectifs, les points autres que les coins sont rapprochés du vecteur équilibré : $\lambda \leftarrow 0{,}8\,\lambda + 0{,}2\,(1/m, \dots, 1/m)$, pour qu'aucun objectif ne reçoive un poids nul hors des coins.

Nous retenons 30 directions pour deux objectifs, 28 pour trois ($H = 6$) et 56 pour quatre ($H = 5$). Ces valeurs viennent d'un criblage sans modèle de langage (tableau 4.1) :

- trois et quatre objectifs : N est augmenté jusqu'à dépasser le cloud glouton de la section 4.3.3. Le budget étant fixe, plus de directions signifie moins de voisins par direction. La plus petite valeur qui dépasse le glouton est retenue,
- deux objectifs : aucune valeur testée n'atteint le cloud glouton, et les écarts restent du même ordre (de $-0{,}29$ à $-0{,}55$ %, sur des sous-ensembles d'instances différents). Les écarts entre 20 et 30 directions ne sont donc pas départageables. Nous retenons 30 directions, qui rapprochent les cinq graines centrales du vecteur équilibré, là où intervient le modèle.

Nombre d'objectifs	N testé	Écart moyen à *SW-CMOLS-G*	Instances
2	10	$-0{,}55$ %	250.2, 500.2, 750.2
2	20	$-0{,}29$ %	250.2, 500.2, 750.2
2	30 (retenu)	$-0{,}42$ %	500.2, 750.2
3	10	$-2{,}21$ %	250.3, 500.3
3	28 (retenu)	$+1{,}44$ %	250.3, 500.3, 750.3
4	10	$-32{,}2$ %	250.4, 500.4
4	35	$-2{,}82$ %	250.4
4	56 (retenu)	$+1{,}39$ %	250.4, 500.4, 750.4

Tab. 4.1 : Criblage du nombre de directions, sans modèle de langage (écart d'hypervolume moyen au cloud glouton, 20 ou 50 répétitions). Les valeurs retenues sont signalées par « retenu ».

Avec trois et quatre objectifs, une grille trop grossière laisse des régions du front sans graine : à quatre objectifs, l'écart passe de -32 % avec dix directions à $+1{,}4$ % avec 56. Avec deux objectifs, le front est une courbe que 20 à 30 directions couvrent déjà. L'écart restant n'est comblé que par l'union (section 4.4.2).

Étape 2 : graines gloutonnes. Pour chaque direction λ, chaque objet i reçoit un score :

$$\mathrm{ratio}_\lambda(i) = \frac{\sum_k \lambda_k\, p_{k,i}}{\sum_k w_{k,i}/c_k} \qquad (4.8)$$

- le numérateur est la *valeur pondérée* de l'objet, $v_\lambda(i) = \sum_k \lambda_k p_{k,i}$: ce qu'il rapporte dans la direction λ,
- le dénominateur est la part des capacités qu'il consomme,
- le ratio est donc la valeur rapportée par unité de place occupée.

Exemple sur 250.2, avec $c = (6\,536\,;\,6\,489)$ et $\lambda = (0{,}552\,;\,0{,}448)$. L'objet 228 a pour profits $(96\,;\,10)$ et pour poids $(88\,;\,34)$:

- $v = 0{,}552 \times 96 + 0{,}448 \times 10 = 57{,}45$,

- part des capacités : $88/6\,536 + 34/6\,489 = 0{,}0187$,
- ratio : $57{,}45/0{,}0187 = 3\,071{,}5$.

Les objets sont ensuite examinés par ratio décroissant. Un objet est ajouté s'il tient dans tous les sacs. Sinon, il est ignoré et l'examen continue jusqu'au dernier objet. Dans l'exemple, les 144 premiers objets entrent. Le sac 1 n'a ensuite plus que 11 unités libres : presque tous les objets suivants sont ignorés, sauf l'objet 89, de poids 10 dans ce sac. La graine compte 145 objets.

Trois grandeurs décrivent une graine :

- ses objectifs f, somme des profits des objets retenus : $f = (9\,334\,;\,9\,113)$,
- sa capacité libre, capacité moins charge dans chaque sac : $(1\,;\,140)$,
- son score S dans sa direction :

$$S = \sum_k \lambda_k f_k = \sum_{i \in \text{graine}} v_\lambda(i) \qquad (4.9)$$

Pour cette graine, $S = 0{,}552 \times 9\,334 + 0{,}448 \times 9\,113 = 9\,234{,}9$.

On obtient ainsi N graines, chacune une solution complète et réalisable, orientée vers sa direction.

Le score S sert uniquement à construire le cloud. Le solveur ne l'utilise pas : son vecteur de poids pondère chaque objectif séparément dans l'indicateur (section 3.4.3). Une solution qui maximise S pour des poids strictement positifs est Pareto-optimale. Augmenter S rapproche donc une graine du front dans sa direction, sans garantir que chaque objectif augmente.

Étape 3 : choix des graines centrales. Le code calcule la distance euclidienne de chaque direction au vecteur équilibré $(1/m, \ldots, 1/m)$ et retient les cinq plus proches. Sur 250.2, ce sont les directions $(0{,}552\,;\,0{,}448)$ à $(0{,}414\,;\,0{,}586)$. Le modèle ne participe pas à ce choix.

Le centre est un choix de conception :

- au centre, tous les objectifs se disputent les capacités, qui se remplissent de façon inégale,
- cinq graines par instance limitent le coût à 45 décisions pour les neuf instances.

Comparer ce choix à une autre sélection de graines reste une perspective.

Étape 4 : le prompt. Chaque graine centrale reçoit un prompt. Le modèle ne voit jamais l'instance complète ni les autres graines. Le prompt contient :

- un objectif unique : augmenter le score S sans dépasser aucune capacité, ou renvoyer des listes vides si aucun échange n'est avantageux,
- l'état de la graine : capacité libre de chaque sac, objectifs f et score S,
- dix candidats au retrait : les objets du sac de plus faible ratio, avec leurs poids w et leur valeur v,
- dix candidats à l'ajout : les objets hors du sac de plus fort ratio, avec w, v et la place manquante dans chaque sac, $\text{need}_k = \max(0,\ w_{k,i} - \text{libre}_k)$,
- le format de réponse : une ligne `{"remove": [...], "add": [...]}`.

Dans l'exemple 4.1, l'objet 57, de poids $(28\,;\,94)$, face à une capacité libre $(1\,;\,140)$, manque de 27 unités dans le sac 1 et d'aucune dans le sac 2.

Exemple 4.1 (Prompt réel, instance 250.2, graine de direction (0,552 ; 0,448))

```
GOAL: raise S = sum_k lambda_k*f_k, every knapsack within capacity.
      No gain -> empty lists.
Free capacity = [1, 140]   f = [9334, 9113]   S = 9234.9
REMOVE candidates (in bag, lowest ratio):
  89  w=[10, 80] v=30.7     97  w=[93, 55] v=69.0 ...
  228 w=[88, 34] v=57.4     ...
ADD candidates (outside, highest ratio), need = room missing:
  88  w=[72, 42] v=52.8 need=[71, 0]   57 w=[28, 94] v=56.3 need=[27, 0]
  179 w=[12, 76] v=40.0 need=[11, 0]   ...
Answer: {"remove": [...], "add": [...]}
```

Réponse : `{"remove": [228], "add": [179, 57]}`.

Étape 5 : la décision du modèle. Elle exploite une limite du score glouton : le dénominateur de l'équation (4.8) traite tous les sacs comme également rares. Or, une fois la graine remplie, un sac est souvent plein alors qu'un autre a encore de la place.

Dans l'exemple 4.1, le sac 1 est plein (1 unité libre) et le sac 2 dispose de 140 unités :

- le glouton classe l'objet 228 (ratio 3 072) au-dessus des objets 179 (2 955) et 57 (3 002) : il ne ferait jamais cet échange,
- or 228 occupe 88 unités du sac plein pour une valeur de 57,4, soit 0,65 par unité de ce sac,
- 179 et 57 n'en occupent que 40 pour une valeur de 96,3, et leur poids dans le sac 2 (170) tient dans la place libérée (174),
- le score passe de 9 234,9 à 9 273,9 (+38,9), et les objectifs de $(9334,\ 9113)$ à $(9290,\ 9254)$: c'est un nouveau compromis.

Sur 500.2, un échange de même nature retire l'objet 460 (ratio 6 339) et ajoute les objets 196, 236 et 255 (ratios de 6 033 à 6 126). Les deux objectifs augmentent, de $(18523,\ 19375)$ à $(18573,\ 19524)$: la nouvelle graine domine l'ancienne.

Étape 6 : vérification. Le code contrôle chaque réponse en trois temps :

- identifiants : chaque objet doit exister, chaque retrait doit être dans la graine et chaque ajout hors de la graine. Un identifiant invalide est écarté et consigné,
- capacités : si un sac déborde, les objets de plus faible ratio sont retirés, puis la solution est complétée dans le même ordre,
- filet de sécurité : si le résultat est dominé par la graine de départ ou lui est identique, la graine gloutonne est conservée.

Le modèle ne peut donc jamais remplacer une graine par une solution qu'elle domine. Sur les 45 réponses, aucun identifiant n'était invalide et aucune réparation n'a été nécessaire.

L'algorithme 4.1 résume les sept étapes. L'étape 7 est détaillée après lui.

Algorithme 4.1 (Construction du cloud SW-CMOLS-L)

```
Entrées : instance (m objectifs, n objets, poids w, profits p, capacités c),
          directions Λ = (λ_1, ..., λ_N), modèle de langage M, plafond Q = 1510
Sortie  : cloud C_L

pour j = 1 à N faire
    g_j ← SolutionGloutonne(λ_j, w, p, c)          // équation (4.8)
fin pour
Centre ← les 5 indices j de λ_j les plus proches de (1/m, ..., 1/m)
pour j ∈ Centre faire
    (REMOVE, ADD) ← M(Prompt(g_j, λ_j))            // une décision par graine
    g' ← Réparer((g_j \ REMOVE) ∪ ADD, λ_j)        // faisabilité, puis complétion
    si g' n'est ni dominée par g_j ni égale à g_j alors
        g_j ← g'
    fin si
fin pour
C_L ← liste vide
n_v ← ⌊Q / N⌋ − 1              // voisins par graine
pour j = 1 à N faire
    AjouterFin(C_L, g_j)
    AjouterFin(C_L, Voisinage(g_j, n_v))           // îles biaisées (voir ci-dessous)
fin pour
retourner C_L
```

Étape 7 : voisinages. Chaque graine, qui est une solution unique, reçoit $n_v = \lfloor 1510/N \rfloor - 1$ voisins, sans nouvel appel au modèle : 49 pour deux objectifs, 52 pour trois et 25 pour quatre. Chaque graine forme ainsi un groupe de 50, 53 ou 26 solutions, et le cloud compte 1 500, 1 484 ou 1 456 solutions, sous le plafond de 1 510.

Les voisins sont répartis en *îles*. Une île est un vecteur de poids cible qui oriente une partie des voisins.

Objectifs	Îles	Répartition des n_v voisins
2	3 : $(0{,}85\,;\,0{,}15)$, $(0{,}15\,;\,0{,}85)$, $(0{,}5\,;\,0{,}5)$	$\lfloor n_v/3 \rfloor$ vers chaque objectif, le reste équilibré : 16, 16, 17
3	7 : un objectif $(0{,}8\,;\,0{,}1\,;\,0{,}1)$, une paire $(0{,}475\,;\,0{,}475\,;\,0{,}05)$, équilibrée	$\lfloor n_v/5 \rfloor$ par objectif, $\lfloor n_v/10 \rfloor$ par paire, le reste équilibré : 3×10, 3×5, 7
4	8 : un objectif (0,85 et 0,05), trois paires (0,45 et 0,05) : celles qui contiennent l'objectif 1, équilibrée	$\lfloor n_v/7 \rfloor$ par objectif, $\lfloor n_v/15 \rfloor$ par paire, le reste équilibré : 4×3, 3×1, 10

Un voisin est construit en quatre opérations :

1. copier la graine,
2. retirer r objets tirés au hasard,
3. calculer pour chaque objet le ratio de l'équation (4.8) avec le vecteur de l'île, multiplié par un bruit aléatoire tiré pour chaque objet et chaque voisin,
4. remplir de nouveau la solution par score décroissant, en ajoutant chaque objet qui tient.

Le nombre r et l'amplitude du bruit dépendent de la taille de l'instance :

Instances	Objets retirés r	Bruit multiplicatif
250.2, 250.3, 250.4	2 à 6	0,92 à 1,08
500.3, 500.4	5 à 12	0,85 à 1,15
750.3, 750.4	10 à 25	0,70 à 1,30
500.2, 750.2	20 à 50	0,60 à 1,40

Sur les grandes instances, une perturbation faible reconstruirait le plus souvent la même solution. Le bruit ne compte que pour les objets hors du sac, y compris ceux qui viennent d'être retirés : ce sont les seuls que le remplissage examine.

L'exemple 4.2 applique ces quatre opérations à une graine réelle de 250.2, pour l'île orientée vers l'objectif 1.

Exemple 4.2 (Un voisin construit pas à pas, instance 250.2)

Graine de départ : direction $(0{,}552\,;\,0{,}448)$ après la décision du modèle, 146 objets, $f = (9\,290\,;\,9\,254)$, capacité libre $(49\,;\,4)$. Île : vecteur $(0{,}85\,;\,0{,}15)$, orienté vers l'objectif 1.

Opération	Ce qui se passe	Résultat
1. Copier la graine	Le voisin part de la graine ci-dessus	146 objets, capacité libre $(49\,;\,4)$
2. Retirer $r = 3$ objets	Tirés au hasard : 170, 65 et 127, de poids $(37\,;\,92)$, $(65\,;\,22)$ et $(21\,;\,66)$	Capacité libre $(172\,;\,184)$
3. Calculer les scores de l'île	Exemple de l'objet 65, de profits $(89\,;\,95)$: $v = 0{,}85 \times 89 + 0{,}15 \times 95 = 89{,}9$, ratio 6 742, bruit tiré 1,074	Score 7 237
4. Remplir par score décroissant	65 (7 237) et 127 (5 750) entrent de nouveau. 228 (4 643) ne tient pas. 11 (4 345), de poids $(84\,;\,47)$, entre. 170 (3 742) et les suivants ne tiennent plus	Capacité libre $(2\,;\,49)$

Bilan : l'objet 170, orienté vers l'objectif 2, est remplacé par l'objet 11, orienté vers l'objectif 1. Le voisin vaut $f = (9\,305\,;\,9\,177)$: il s'est déplacé vers l'objectif 1, comme le demande son île.

Une perturbation faible reconstruit parfois une solution déjà présente : sur 250.2, les 1 500 entrées du cloud comptent 1 047 solutions distinctes. Seule la partie non dominée, 106 solutions sur cette instance, forme l'archive de départ du solveur.

Coût. Les 45 prompts ont été soumis au modèle Claude Opus 5.5 [47]. Les décisions ont pris 4 min 43 s au total. Bilan :

- 39 décisions modifient leur graine, dont 29 la dominent,
- les six autres sont des réponses « aucun gain »,
- pour les décisions retenues, le score de la graine augmente de 0,44 % en moyenne (de 0,07 % à 1,17 %).

Les prompts et les réponses sont consignés. La chaîne les rejoue à l'identique : le cloud est reproductible sans nouvel appel au modèle.

4.3.3 SW-CMOLS-G

La variante *SW-CMOLS-G* construit un cloud reproductible sans appel au modèle de langage, par un balayage glouton sur un ensemble structuré de vecteurs de poids. Les vecteurs de Dirichlet utilisent un générateur pseudo-aléatoire de graine fixe (42).

Construction de l'ensemble de vecteurs de poids. Pour une instance à m objectifs, l'ensemble Λ_G de 302 vecteurs comprend, par ordre de priorité :

- les m coins du simplexe, chacun correspondant à la maximisation isolée d'un objectif,
- les $\binom{m}{2}$ milieux d'arêtes, vecteurs à deux coordonnées égales à $0{,}5$,
- un complément par balayage linéaire fin lorsque $m = 2$, et par tirage selon une loi de Dirichlet uniforme sur le simplexe lorsque $m \geq 3$.

L'algorithme 4.2 en donne le déroulement.

Algorithme 4.2 (Génération du cloud glouton)

```
Entrées : instance (m objectifs, n objets, poids w, profits p, capacités c)
Sortie  : cloud C_G

Λ_G ← ConstruireVecteursPoids(m, 302)          // coins + arêtes + balayage/Dirichlet
C_G ← liste vide
pour chaque λ ∈ Λ_G faire
    AjouterFin(C_G, SolutionGloutonne(λ, w, p, c))   // équation (4.8)
fin pour
retourner C_G
```

Des directions voisines conduisent souvent au même sac : selon l'instance, ce balayage donne 67 à 302 solutions distinctes. Le coût de génération est de l'ordre de quelques secondes par instance.

4.3.4 SW-CMOLS-U et intégration au solveur

L'union associe les deux sources précédentes. Le cloud glouton apporte des solutions précises réparties sur 302 directions. Le cloud *SW-CMOLS-L* apporte des voisinages denses et des graines centrales améliorées.

Construction par échantillonnage régulier. *SW-CMOLS-U* prélève 755 solutions dans chacun des deux clouds, par un échantillonnage à pas régulier plutôt qu'un tirage aléatoire :

$$\mathrm{ech_reg}(C, k) = \{c_{\lfloor 0 \cdot n/k \rfloor}, c_{\lfloor 1 \cdot n/k \rfloor}, \ldots, c_{\lfloor (k-1) \cdot n/k \rfloor}\}, \qquad n = |C| \tag{4.10}$$

Les solutions des deux clouds sont ordonnées par construction (vecteur de poids pour l'un, graine puis voisinage pour l'autre) : un prélèvement à pas régulier couvre tout cet ordonnancement.

Algorithme 4.3 (Construction du cloud union)

```
Entrées : cloud glouton C_G, cloud C_L de SW-CMOLS-L, taille prélevée k = 755
Sortie  : cloud union C_U (1510 entrées, 587 à 1054 distinctes)

C_U ← liste vide
pour i = 0 à k-1 faire
    AjouterFin(C_U, C_G[⌊i · |C_G| / k⌋])
fin pour
pour i = 0 à k-1 faire
    AjouterFin(C_U, C_L[⌊i · |C_L| / k⌋])
fin pour
retourner C_U
```

Intégration au solveur. Les trois clouds partagent, une fois construits, le même mécanisme d'intégration à *W-CMOLS* (algorithme 4.4) : l'archive est préremplie par les solutions non dominées du cloud, puis la recherche locale se déroule sans autre modification.

Algorithme 4.4 (Intégration d'un cloud au cycle de recherche du solveur)

```
Entrées : cloud C (une seule variante à la fois), nombre d'itérations T
Sortie  : archive finale A

A ← extract_non_dominées(C)              // préremplissage de l'archive
pour it = 0 à T-1 faire
    P ← Sélectionner(A, α)                // population de travail tirée de l'archive
    E ← RechercheLocaleW-CMOLS(P)         // une itération de recherche locale à
                                          // indicateur ε+ pondéré (section 3.4)
    A ← extract_non_dominées(A ∪ E)
fin pour
retourner A
```

Le tableau ci-dessous résume les choix de conception des trois variantes, indépendamment de tout chiffre d'hypervolume :

Variante	Construction	Atout principal	Coût de génération
SW-CMOLS-G	Balayage glouton sur 302 vecteurs de poids (section 4.3.3)	Solutions précises, faisabilité native	Quelques secondes par instance
SW-CMOLS-L	30, 28 ou 56 directions, graines centrales améliorées par le modèle, voisinages (section 4.3.2)	Couverture du front et graines centrales améliorées	Quelques secondes, plus 45 décisions du modèle au total (4 min 43 s)
SW-CMOLS-U	Échantillonnage régulier à parts égales des deux clouds (algorithme 4.3)	Association des deux sources dans le même plafond	Celui de *SW-CMOLS-L*

4.4 Expérimentations

4.4.1 Protocole expérimental

Instances. Les trois variantes de *SW-CMOLS* sont évaluées sur les neuf instances de référence décrites à la section 3.6.

Modèle de langage. Les 45 décisions de *SW-CMOLS-L* proviennent de Claude Opus 5.5 [47]. Elles sont obtenues une fois, consignées, puis rejouées : le cloud est identique d'une répétition à l'autre.

Configuration du solveur. Pour chaque combinaison (système, instance), le solveur tourne sur 100 itérations de recherche locale. Les variantes amorcées préremplissent leur archive avec un cloud. Le solveur de référence démarre avec une archive vide et construit sa première population à partir de permutations aléatoires. Aucun autre mécanisme n'est ajouté : tous les systèmes utilisent la même recherche locale, sans perturbation supplémentaire.

Paramètres. La taille de la population de travail ($\alpha = 10$), la profondeur du voisinage ($L = 5$), le facteur d'échelle de la fitness ($\kappa = 0{,}05$) et le nombre d'itérations ($T = 100$) sont communs à tous les systèmes (chapitre 3, tableau 3.1). Le seul élément qui varie est donc le point de départ.

Métriques. L'hypervolume $HV(A, r)$ (section 1.4.3) est l'indicateur principal. Il mesure le volume de l'espace des objectifs dominé par le front obtenu, borné par un point de référence : il augmente lorsque le front se rapproche du front optimal et lorsqu'il s'étend. Il est retenu parce qu'il résume ces deux qualités en un seul nombre et qu'il respecte la dominance de Pareto : un front meilleur obtient toujours un hypervolume plus grand. Son calcul suit la procédure unifiée du chapitre 3 (section 3.6) : chaque front est normalisé dans $[0, 1]^m$ par des bornes propres à l'instance, partagées par tous les systèmes, puis l'hypervolume est calculé relativement au point de référence $(-0{,}01, \ldots, -0{,}01)$. Pour les fronts de plus de 1000 points, le calcul utilise l'amincissement déterministe décrit au chapitre 3. S'y ajoutent l'espacement et l'étendue (section 4.2.3), qui décrivent la répartition du front, sa cardinalité, c'est-à-dire le nombre de solutions remises au décideur, et le temps d'exécution.

Répétitions. Chaque combinaison (système, instance) est exécutée 50 fois de manière indépendante, avec les mêmes germes aléatoires pour tous les systèmes. Les fichiers de clouds restent fixes d'une répétition à l'autre.

Tests statistiques. Chaque comparaison, instance par instance, repose sur un test de Mann-Whitney U unilatéral au seuil de 0,05. Les comparaisons globales entre deux variantes utilisent en complément le test exact de Wilcoxon apparié unilatéral sur les neuf moyennes d'instance. Aucune correction pour comparaisons multiples n'est appliquée aux tests par instance. Face à *W-CMOLS*, toutes les valeurs de p sont inférieures à 2×10^{-17} et le resteraient après correction.

4.4.2 Résultats, cardinalité et temps d'exécution

Le tableau 4.2 donne l'hypervolume moyen, sur 50 répétitions, de *W-CMOLS* et des trois variantes de *SW-CMOLS*. Un astérisque (*) signale une amélioration significative par rapport à *W-CMOLS* ($p < 0{,}05$).

Instance	W-CMOLS	SW-CMOLS-G	SW-CMOLS-L	SW-CMOLS-U
250.2	0,7895	0,8107*	**0,8144***	0,8143*
250.3	0,4311	0,5530*	0,5575*	**0,5609***
250.4	0,1818	0,3185*	0,3245*	**0,3292***
500.2	0,7261	0,7855*	0,7822*	**0,7862***
500.3	0,3847	0,5106*	**0,5260***	0,5251*
500.4	0,1330	0,2922*	0,2992*	**0,3065***
750.2	0,6418	0,7714*	0,7708*	**0,7728***
750.3	0,3871	0,4991*	0,5065*	**0,5091***
750.4	0,1280	0,2771*	0,2815*	**0,2898***
Moyenne	0,4226	0,5354	0,5403	**0,5438**

Tab. 4.2 : Hypervolume moyen sur 50 répétitions, *W-CMOLS* et variantes de *SW-CMOLS*. (*) : amélioration significative par rapport à *W-CMOLS*, test de Mann-Whitney U, $p < 0{,}05$. Toutes les valeurs de p sont inférieures à 2×10^{-17}.

Les trois variantes améliorent *W-CMOLS* sur les neuf instances, avec un test significatif dans chaque cas. En moyenne non pondérée des neuf gains relatifs, le gain est de +48,0 % pour *SW-CMOLS-G*, +50,1 % pour *SW-CMOLS-L* et +52,0 % pour *SW-CMOLS-U*. Il va de +3,1 % (250.2) à +130,4 % (500.4) pour l'union, et croît avec le nombre d'objectifs.

Les variantes se départagent ensuite entre elles. *SW-CMOLS-L* dépasse *SW-CMOLS-G* sur 7 instances sur 9, chaque fois de façon significative (Wilcoxon apparié sur les neuf moyennes, $p = 0{,}0098$). Il ne reste derrière que sur 500.2 ($-0{,}42$ %) et 750.2 ($-0{,}08$ %, différence non significative). *SW-CMOLS-U* dépasse *SW-CMOLS-G* sur les neuf instances, chaque fois de façon significative ($p = 0{,}0020$), y compris sur 500.2 et 750.2. Il dépasse aussi *SW-CMOLS-L* sur 7 instances ($p = 0{,}0098$). L'union obtient ainsi le meilleur hypervolume moyen.

Le tableau 4.3 détaille la stabilité sur les instances à deux objectifs.

Instance	Système	Moyenne	Médiane	Écart-type	Minimum	Effondrements
250.2	W-CMOLS	0,7895	0,7898	0,0043	0,7676	0 / 50
	SW-CMOLS-L	0,8144	0,8144	0,0003	0,8141	0 / 50
	SW-CMOLS-U	0,8143	0,8143	0,0005	0,8133	0 / 50
500.2	W-CMOLS	0,7261	0,7703	0,1529	0,1771	4 / 50
	SW-CMOLS-L	0,7822	0,7823	0,0009	0,7800	0 / 50
	SW-CMOLS-U	0,7862	0,7861	0,0007	0,7850	0 / 50
750.2	W-CMOLS	0,6418	0,7636	0,2478	0,1210	10 / 50
	SW-CMOLS-L	0,7708	0,7710	0,0017	0,7668	0 / 50
	SW-CMOLS-U	0,7728	0,7729	0,0019	0,7658	0 / 50

Tab. 4.3 : Distribution des hypervolumes sur les trois instances à deux objectifs, 50 répétitions.

Un « effondrement » est une répétition dont l'hypervolume tombe sous la moitié de la médiane de *W-CMOLS* sur la même instance. *W-CMOLS* présente deux groupes de résultats sur 500.2 et 750.2 : la majorité des répétitions atteint environ 0,77, mais 4 répétitions sur 500.2 et 10 sur 750.2 s'effondrent. Ces valeurs abaissent la moyenne sans déplacer fortement la médiane. Les variantes amorcées ne présentent aucun effondrement et relèvent aussi la médiane : sur 750.2, l'écart-type de *SW-CMOLS-L* est de 0,0017 contre 0,2478 pour *W-CMOLS*.

Contribution propre du modèle de langage. Le gain de *SW-CMOLS-L* sur *SW-CMOLS-G* mélange deux effets : la construction du cloud (directions, voisinages) et les décisions du modèle. Pour les séparer, le même dispositif a été exécuté sans les 45 décisions, les cinq graines centrales restant gloutonnes. Tout le reste est identique (directions, voisinages, solveur, germes). Le tableau 4.4 compare les deux.

Instance	Sans décisions du modèle	Avec décisions (*SW-CMOLS-L*)	Écart	p
250.2	0,8138	0,8144*	+0,07 %	< 0,001
250.3	0,5563	0,5575*	+0,22 %	< 0,001
250.4	0,3222	0,3245*	+0,71 %	< 0,001
500.2	0,7807	0,7822*	+0,20 %	< 0,001
500.3	0,5243	0,5260*	+0,33 %	< 0,001
500.4	0,2974	0,2992*	+0,59 %	< 0,001
750.2	0,7703	0,7708	+0,06 %	0,103
750.3	0,5032	0,5065*	+0,65 %	< 0,001
750.4	0,2809	0,2815*	+0,19 %	0,048
Moyenne	0,5388	0,5403	+0,34 %	

Tab. 4.4 : Effet propre des décisions du modèle de langage : même dispositif avec et sans les 45 décisions, 50 répétitions. (*) : amélioration significative, test de Mann-Whitney U unilatéral, $p < 0{,}05$.

Les décisions du modèle améliorent l'hypervolume moyen sur les neuf instances, de façon significative sur huit. L'exception est 750.2 (+0,06 %, $p = 0{,}10$). Le gain moyen est de +0,34 % (Wilcoxon apparié unilatéral sur les neuf moyennes, $p = 0{,}0020$). Il est faible devant l'écart qui sépare les variantes amorcées de *W-CMOLS* : l'essentiel du gain vient de la couverture du front. Mais il est systématique, alors que les décisions ne modifient que cinq graines sur 28 à 56 par instance.

Les figures 4.2 à 4.10 présentent les fronts obtenus sur les neuf instances, dans l'ordre du tableau 4.2.

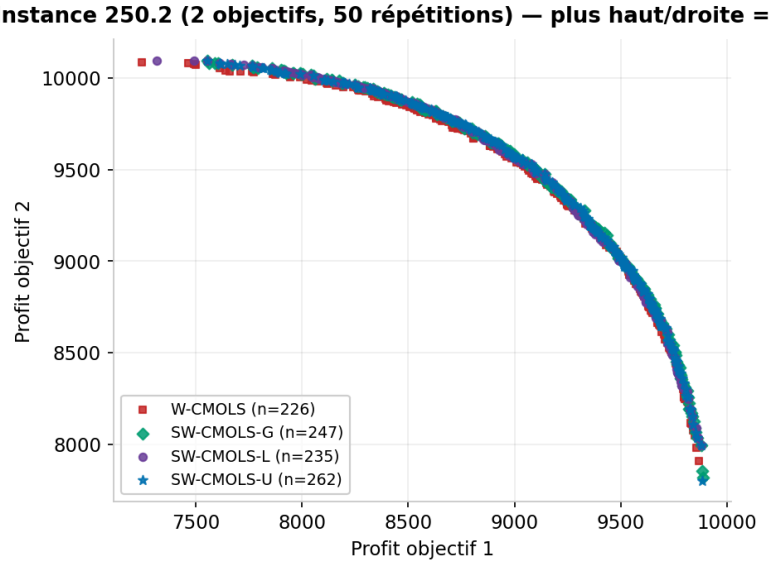


Fig. 4.2 : Instance 250.2 (2 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

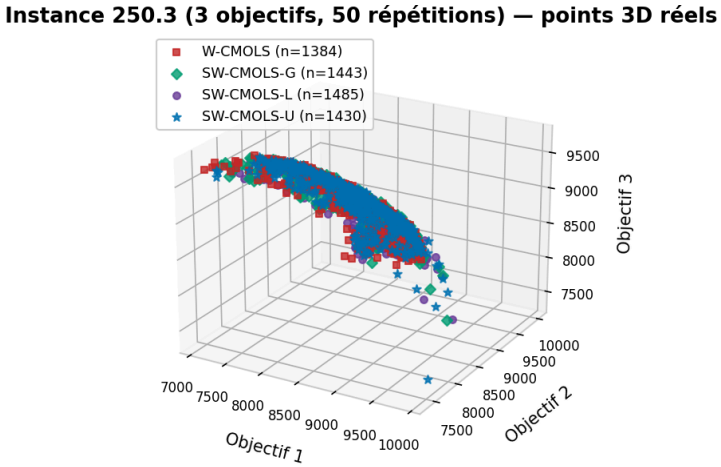


Fig. 4.3 : Instance 250.3 (3 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

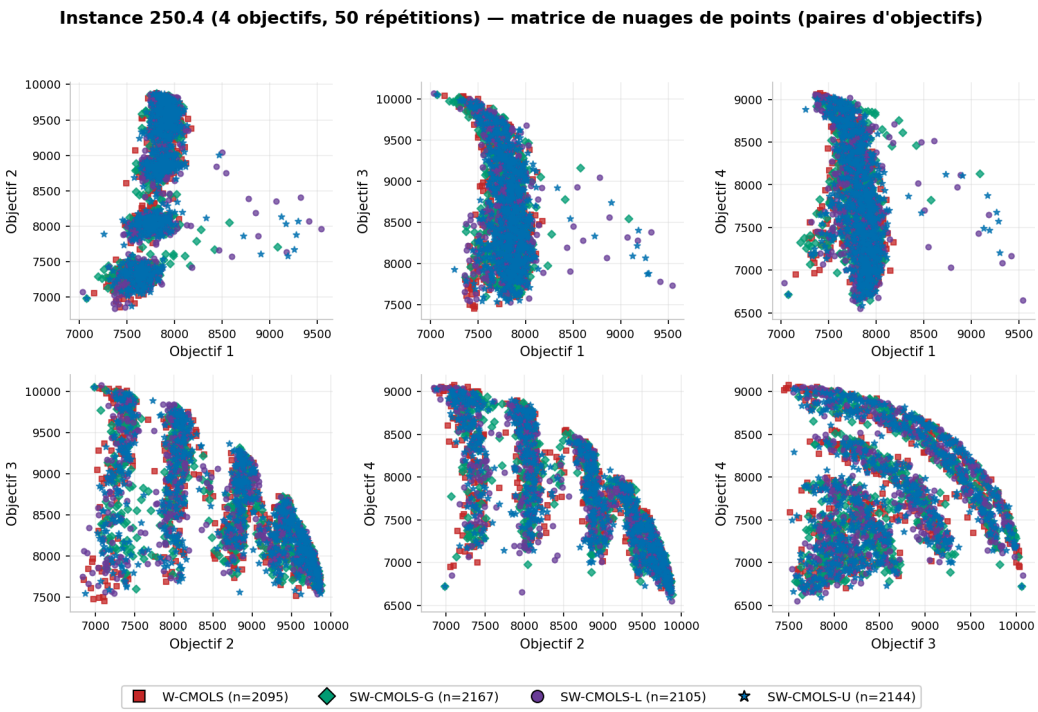


Fig. 4.4 : Instance 250.4 (4 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

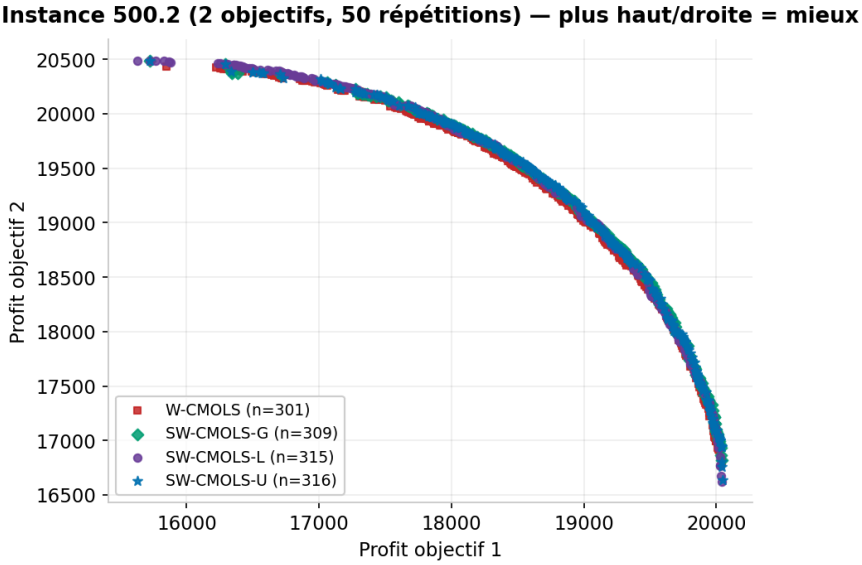


Fig. 4.5 : Instance 500.2 (2 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

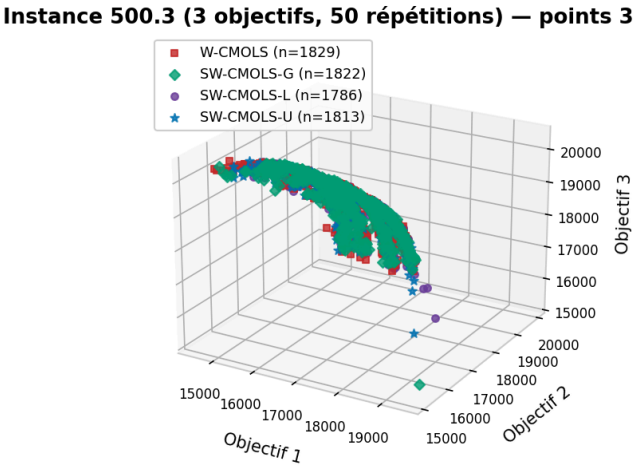


Fig. 4.6 : Instance 500.3 (3 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

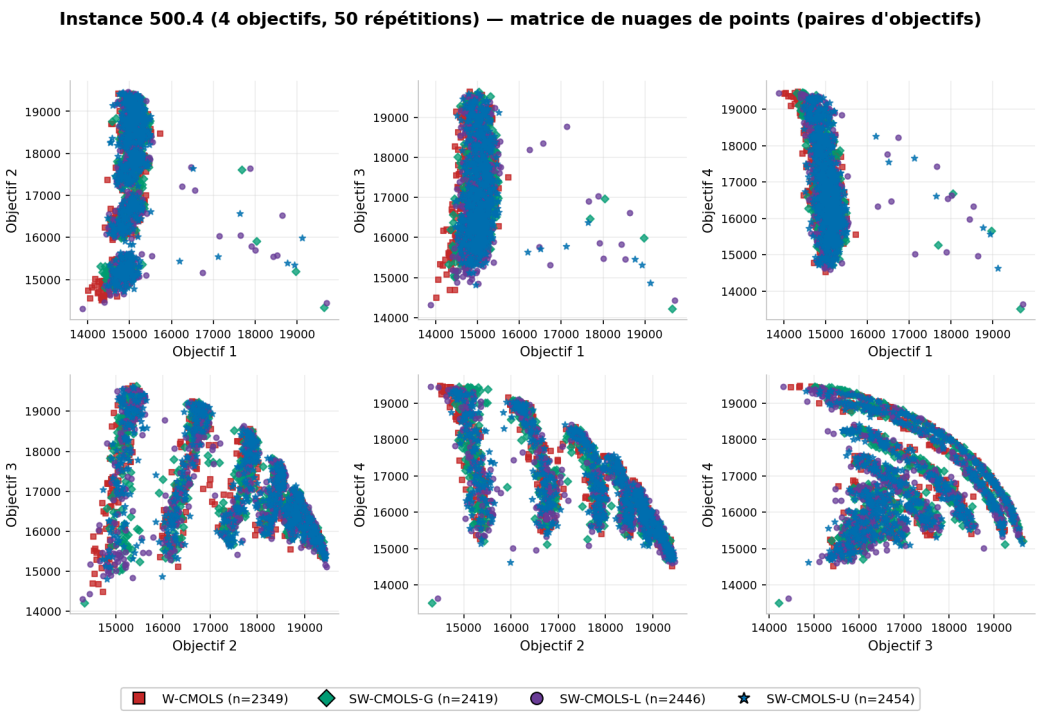


Fig. 4.7 : Instance 500.4 (4 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

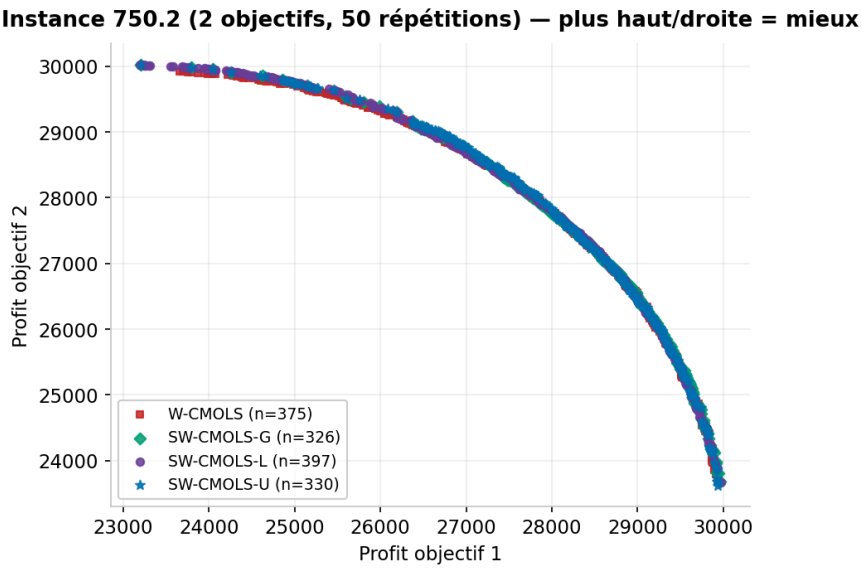


Fig. 4.8 : Instance 750.2 (2 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

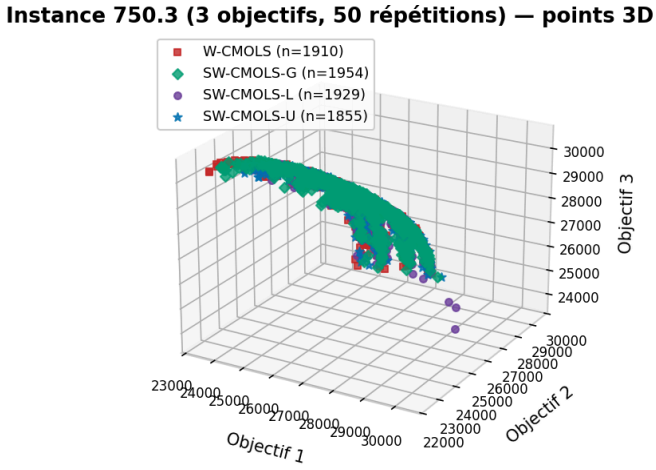


Fig. 4.9 : Instance 750.3 (3 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

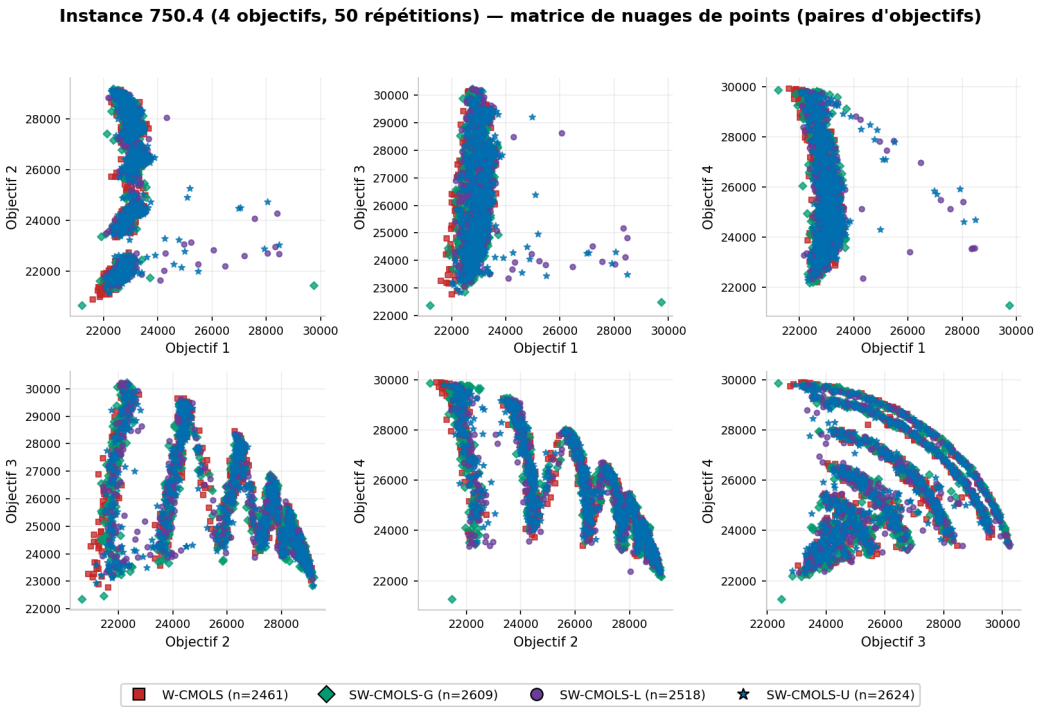


Fig. 4.10 : Instance 750.4 (4 objectifs) : *W-CMOLS* et les trois variantes d'amorçage.

Diversité des archives finales. L'espacement et l'étendue (section 4.2.3) sont mesurés sur l'archive finale des 20 premières répétitions, après les 100 itérations. Au-delà de 400 points, l'archive est amincie par la procédure du chapitre 3, qui conserve les extrêmes.

- Étendue : les trois variantes amorcées couvrent une région plus large que *W-CMOLS*, en moyenne 1,61 (G), 1,60 (L) et 1,61 (U), contre 1,31.
- Espacement : celui de *W-CMOLS* est plus faible (0,007 contre 0,021 à 0,025). Son front, plus étroit, est aussi plus régulier.
- Entre variantes amorcées, les écarts sont faibles : *SW-CMOLS-L* est la plus régulière (0,021), devant G (0,023) et U (0,025).

Contenu des clouds. Les trois clouds diffèrent davantage avant la recherche qu'après :

- solutions distinctes : 67 à 302 pour le cloud glouton, 1 047 à 1 500 pour *SW-CMOLS-L*, 587 à 1 054 pour l'union,
- archive de départ (partie non dominée) : plus grande pour L que pour G sur sept instances, par exemple 106 contre 39 sur 250.2,
- exceptions : 500.2 (43 contre 61) et 750.2 (64 contre 98), précisément les deux instances où L reste derrière G. L'union y part de 64 et 90 solutions, ce qui est cohérent avec sa supériorité sur ces instances.

L'hypervolume ne rend compte ni du nombre de solutions remises au décideur, ni du temps nécessaire pour les obtenir. Le tableau 4.5 donne ces deux grandeurs.

Instance	W-CMOLS	SW-CMOLS-G	SW-CMOLS-L	SW-CMOLS-U	W-CMOLS (s)	SW-CMOLS-G (s)	SW-CMOLS-L (s)	SW-CMOLS-U (s)
250.2	140	111	125	124	**0,50**	0,54	1,04	0,59
250.3	843	888	1041	981	**1,21**	1,32	1,75	1,64
250.4	1104	1339	1665	1634	**2,30**	3,37	5,21	5,54
500.2	230	144	158	137	**1,38**	1,66	1,51	1,60
500.3	1518	1638	1777	1748	**3,30**	3,77	4,66	4,64
500.4	2029	2086	2620	2443	**7,56**	7,82	12,45	11,18
750.2	301	304	334	306	**2,81**	3,45	3,18	3,07
750.3	2012	2091	2247	2177	**6,10**	6,70	7,74	7,66
750.4	2836	3010	3506	3327	**14,42**	16,40	22,42	20,72
Moyenne	1224	1290	1497	1431	**4,40**	5,00	6,66	6,29

Tab. 4.5 : Cardinalité moyenne du front rendu par exécution et temps moyen par exécution, 50 répétitions. Le gras indique le temps le plus court de la ligne. La cardinalité n'est pas une mesure de qualité et n'est donc pas mise en gras.

Cardinalité. Les variantes amorcées rendent davantage de solutions que *W-CMOLS* sur les instances à trois et quatre objectifs. À deux objectifs, elles en rendent moins sur 250.2 et 500.2, et autant ou un peu plus sur 750.2.

Temps. Partant d'une archive plus riche, le solveur conserve et explore davantage de solutions, et chaque exécution dure plus longtemps :

- par exécution : 5,00 s (G), 6,66 s (L) et 6,29 s (U), contre 4,40 s pour *W-CMOLS*,
- sur les 450 exécutions du protocole : 37,5 min (G), 50,0 min (L) et 47,2 min (U), contre 33,0 min.

Ce surcoût est le prix de l'amorçage. Le chapitre 5 le supprime : *SW-CMOLS-UQ* garde la qualité de *SW-CMOLS-U* et devient plus rapide que *W-CMOLS* sur les neuf instances. Les temps excluent la génération du cloud.

Coût de génération. Le cloud glouton se génère en quelques secondes par instance. Le cloud *SW-CMOLS-L* demande le même ordre de temps pour ses graines et ses voisinages, plus les décisions du modèle : 4 min 43 s pour les 45 décisions des neuf instances. *SW-CMOLS-U* hérite de ce coût. Chaque cloud est produit une seule fois par instance puis réutilisé pendant les 50 répétitions.

4.5 Conclusion

Ce chapitre a proposé *SW-CMOLS*, qui préremplit l'archive initialement vide de *W-CMOLS* par un cloud de solutions candidates, sans rien modifier d'autre dans le solveur. Trois instanciations ont été étudiées : un balayage glouton (*SW-CMOLS-G*), un cloud construit sur des directions réparties dont les graines centrales sont améliorées par un modèle de langage (*SW-CMOLS-L*), et l'union des deux à parts égales (*SW-CMOLS-U*).

L'amorçage informé améliore l'hypervolume sur les neuf instances pour les trois variantes, de +48,0 % à +52,0 % en moyenne. *SW-CMOLS-L* dépasse le cloud glouton sur 7 instances sur 9, et *SW-CMOLS-U* sur les neuf, y compris les deux instances à deux objectifs où *SW-CMOLS-L* seul reste en retrait. À dispositif identique, les décisions du modèle de langage apportent un gain faible mais systématique : $+0{,}34$ % en moyenne, significatif sur huit instances sur neuf.

Deux leviers expliquent ces résultats. Le premier est la couverture : des directions réparties sur tout le simplexe, en nombre suffisant, et des voisinages denses autour de chacune. Le second, plus modeste, est la décision du modèle : en raisonnant sur le sac réellement saturé, il améliore des graines centrales que le tri glouton, qui traite tous les sacs comme également rares, ne peut pas améliorer.

La principale limite reste le coût d'exécution : partant d'une archive plus riche, le solveur conserve davantage de solutions et chaque exécution est plus longue. L'amorçage reste par ailleurs statique : les hyperparamètres du solveur demeurent fixes. Le chapitre suivant choisit une configuration propre à chaque instance, ce qui permet de réduire ce coût à qualité comparable.

Chapitre 5

Configuration par instance des hyperparamètres via un agent DQN

5.1 Introduction

Ce chapitre présente la deuxième contribution de ce mémoire : *SW-CMOLS-Q*, qui utilise un agent d'apprentissage par renforcement profond, un DQN (section 2.8.2), pour choisir les hyperparamètres de *W-CMOLS* avant chaque exécution. Trois variantes l'associent aux clouds glouton (*SW-CMOLS-GQ*), LLM (*SW-CMOLS-LQ*) et union (*SW-CMOLS-UQ*) du chapitre 4.

Le chapitre précédent a étudié l'amorçage de *W-CMOLS*, dont les hyperparamètres restaient fixes : $\alpha = 10$, $\mathrm{NBL} = 100$, $L = 5$ et $\kappa = 0{,}05$. Une configuration unique convient pourtant différemment aux instances de 250 à 750 objets et de deux à quatre objectifs. Avec un cloud, le solveur part en outre de bonnes solutions et n'a plus forcément besoin de toute la recherche prévue par défaut.

L'agent choisit ces quatre hyperparamètres à partir de caractéristiques de l'instance. Il ne les modifie pas pendant la recherche : il s'agit d'une configuration par instance, et non d'un contrôle dynamique.

Le processus suit cinq étapes :

1. décrire l'instance par quatre nombres (section 5.2),
2. prédire le temps de chaque configuration par une formule de coût (section 5.3.1),
3. interdire les configurations dont le temps prédit dépasse un budget (section 5.3.2),
4. choisir, parmi les configurations autorisées, celle que le réseau juge la meilleure (section 5.3.3), après un entraînement réalisé une seule fois (section 5.3.4),
5. lancer une seule exécution de *W-CMOLS* avec cette configuration.

L'objectif est de conserver la qualité apportée par l'amorçage tout en réduisant le temps d'exécution.

5.2 Formulation : ce que l'agent observe, choisit et comment il est noté

Le chapitre 2 (section 2.8.1) a présenté les processus de décision markoviens et le *Q-learning*. Ici, un épisode correspond à une exécution complète du solveur : l'agent observe l'instance, choisit une configuration, puis reçoit une note, appelée récompense. Comme chaque épisode ne comporte qu'une décision, le problème relève formellement d'un bandit contextuel : la récompense sert directement de cible d'apprentissage, sans étape suivante à anticiper. Le réseau DQN reste adapté à ce cadre : ses quatre têtes traitent les 432 configurations paramètre par paramètre (section 5.3.3). La configuration dépend de l'instance, et non de l'avancement de la recherche, contrairement aux méthodes qui interviennent à chaque itération (section 2.8.3).

Espace d'état. L'état décrit l'instance par quatre caractéristiques :

$$s = \left(\frac{m}{4},\ \frac{\log(n/250)}{\log 3},\ \mathrm{HV}_{\mathrm{base}},\ \frac{\log(\max(t_{\mathrm{base}}, 0{,}4)/0{,}4)}{\log 40} \right) \in \mathbb{R}^4 \qquad (5.1)$$

où m est le nombre d'objectifs, n le nombre d'objets, $\mathrm{HV}_{\mathrm{base}} \in [0,1]$ l'hypervolume normalisé atteint par *W-CMOLS* et t_{base} son temps de référence. Les deux premières composantes viennent du fichier de l'instance, les deux dernières de quelques exécutions de *W-CMOLS* seul, faites une seule fois par instance (six dans notre dispositif). La taille et le temps sont pris en logarithme, car ils agissent en « combien de fois plus ». L'hypervolume est déjà compris entre 0 et 1. Par exemple, l'instance 500.3 (trois objectifs, 500 objets) est décrite par $s = (0{,}75;\ 0{,}63;\ 0{,}40;\ 1{,}01)$. L'état ne contient pas le nom de l'instance : l'agent apprend à réagir à un type d'instance.

Espace d'action. L'action fixe les quatre paramètres, chacun choisi dans une grille :

$$\begin{aligned} a \ &= \ (\alpha, \mathrm{NBL}, L, \kappa), \quad \text{avec} \\ \alpha &\in \{10, 15, 20, 25, 30, 40\}, \qquad \mathrm{NBL} \in \{30, 50, 70, 100, 130, 160\}, \\ L &\in \{3, 5, 8, 10\}, \qquad \kappa \in \{0{,}05,\ 0{,}1,\ 0{,}2\} \end{aligned} \qquad (5.2)$$

soit $6 \times 6 \times 4 \times 3 = 432$ configurations. α fixe la population conservée, NBL le nombre d'itérations (noté T au chapitre 3), L la profondeur du voisinage et κ la pression de sélection à base d'indicateur, apparentée au principe d'*IBEA* [11].

Les bornes des grilles proviennent d'une étude de sensibilité paramètre par paramètre :

- NBL va jusqu'à 160, car la qualité des instances à quatre objectifs progresse encore au-delà de 100. La valeur 200 devient défavorable une fois le temps pénalisé,
- α s'arrête à 40 : de 30 à 40, l'hypervolume ne progresse plus, alors que le temps augmente d'environ moitié. La valeur de référence $\alpha = 10$ est conservée,
- $L = 10$ est ajouté pour les instances à quatre objectifs,
- la grille de κ est maintenue, sa réponse étant peu sensible sur la plage testée.

Sur 250.4, dans cette étude, l'hypervolume reste à 0,189 de 50 à 100 itérations, puis passe à 0,267 à $\mathrm{NBL} = 130$, soit $+41\ \%$. Ce saut est cohérent avec l'ordre des vecteurs de poids décrit à la section 3.5 : pour $m = 4$, le premier vecteur donnant un poids non nul à l'objectif 1 se situe après l'indice 99 et n'est donc pas visité avec 100 itérations.

Fonction de récompense. Soit $\mathrm{HV}_{\mathrm{med}}$ l'hypervolume médian des exécutions du solveur avec la configuration choisie, et t_{moy} leur temps moyen. La récompense combine le gain relatif d'hypervolume et une pénalité de temps :

$$r = \underbrace{\frac{\mathrm{HV}_{\mathrm{med}} - \mathrm{HV}_{\mathrm{base}}}{\mathrm{HV}_{\mathrm{base}}}}_{\text{gain relatif}} - \underbrace{\lambda \cdot \max\left(0,\ \frac{t_{\mathrm{moy}}}{t_{\mathrm{base}}} - 1\right)}_{\text{pénalité de coût}}, \qquad \lambda = 0{,}10 \qquad (5.3)$$

λ désigne ici le poids de la pénalité, sans lien avec les vecteurs de poids des chapitres 3 et 4.

Le gain est relatif afin d'avoir le même sens sur toutes les instances. La pénalité ne s'applique que si la configuration est plus lente que *W-CMOLS*. Son poids est le même pour deux, trois et quatre objectifs, ce qui évite un réglage propre à chaque dimension. Un terme de cardinalité s'y ajoute :

$$r' = r + \nu \cdot \frac{|\mathrm{archive}| - |\mathrm{archive}_{\mathrm{base}}|}{|\mathrm{archive}_{\mathrm{base}}|}, \qquad \nu = 0{,}2 \qquad (5.4)$$

où $|\mathrm{archive}|$ est la cardinalité médiane de l'archive finale et $|\mathrm{archive}_{\mathrm{base}}|$ celle de *W-CMOLS*. Ce terme encourage à rendre un nombre suffisant de compromis, sans le confondre avec une amélioration de qualité.

5.3 Approche proposée : SW-CMOLS-Q

SW-CMOLS-Q associe un réseau de type DQN, avec une architecture *Dueling* et un rejeu d'expérience priorisé (section 2.8.2), à un masque de budget. Le masque repose sur une formule qui prédit le temps de chaque configuration (section 5.3.1) et sur un budget propre à chaque instance (section 5.3.2). Le réseau choisit ensuite parmi les configurations autorisées (sections 5.3.3 et 5.3.4).

5.3.1 Prédire le temps d'une configuration

Pour écarter une configuration trop lente avant de l'essayer, il faut connaître son temps sans lancer *W-CMOLS*. Chaque paramètre multiplie le temps, et chaque instance fixe un niveau de départ c_{instance} :

$$\hat{t}(\alpha, \text{NBL}, L, \kappa \mid \text{instance}) = \exp\Big(c_{\text{instance}} + b_N \ln \text{NBL} + b_L \ln L + b_\alpha \ln \alpha + b_\kappa \kappa\Big) \tag{5.5}$$

En logarithme, cette formule devient une somme. Ses coefficients sont donc obtenus par régression linéaire, sur 900 exécutions de *W-CMOLS* dont le temps a été mesuré. Les 5 % de mesures les plus éloignées de la formule, dues à une machine ralentie ou à des exécutions anormalement courtes, sont retirées avant l'ajustement final. Sur ces données, $R^2 = 0{,}993$ et l'erreur multiplicative médiane vaut 6,9 %. Il ne s'agit pas d'une validation externe.

Les coefficients valent $b_N = 1{,}573$, $b_\alpha = 1{,}613$, $b_L = 0{,}168$ et $b_\kappa = 0{,}397$. Doubler NBL ou α multiplie le temps prédit par environ trois, tandis que L et κ pèsent peu. Ces coefficients sont communs à toutes les instances. Le terme constant c_{instance} de chaque instance et de chaque cloud est étalonné sur des temps mesurés avec le dispositif d'évaluation. Sur 500.4, avec le cloud union, $c_{\text{instance}} = -8{,}729$: la configuration $(10, 50, 8, 0{,}05)$ est prédite à $e^{1{,}506} = 4{,}51$ s, pour 4,32 s mesurées.

5.3.2 Budget et configurations autorisées

Le budget B d'une instance vaut 0,8 fois le temps mesuré de *W-CMOLS* sur cette instance : 6,05 s sur 500.4, par exemple, pour 7,56 s. Un budget par instance impose d'être plus rapide que *W-CMOLS* sur chacune, et non seulement en moyenne. Le facteur 0,8 a été retenu par un criblage à dix répétitions : 0,7 ne réduit pas davantage le temps (27,4 s contre 26,9 s pour 0,8, sur les neuf instances) et perd un peu d'hypervolume ($-0{,}07$ %).

Les paramètres sont choisis l'un après l'autre (section 5.3.3). Une valeur v du paramètre d n'est autorisée que si, complétée par les valeurs les moins coûteuses des paramètres restants, elle reste sous le budget :

$$\text{autorisé}(d, v \mid a_{<d}) \iff \hat{t}\Big(a_{<d},\, d{=}v,\, \text{valeurs restantes les moins coûteuses}\Big) \le B \tag{5.6}$$

Comme le temps croît avec chaque paramètre, ce test suffit : si la complétion la moins coûteuse dépasse B, toutes les autres aussi. Sur 500.4, $\alpha = 25$ est interdit : même complété par $\text{NBL} = 30$, $L = 3$ et $\kappa = 0{,}05$, il est prédit à 7,51 s. $\alpha = 20$ reste autorisé (5,24 s). Au total, 41 configurations sur 432 restent autorisées sur 500.4, et de 32 à 77 selon l'instance.

La configuration de référence $(10, 100, 5, 0{,}05)$ n'est autorisée que si elle tient dans le budget. Avec un cloud, elle coûte plus que *W-CMOLS* seul (4,64 s contre 3,30 s sur 500.3, cloud union, 50 répétitions). Elle est donc écartée sur les neuf instances. Si aucune valeur ne passe le test, la moins coûteuse est conservée.

Le masque s'applique pendant l'entraînement comme à l'utilisation : même lorsqu'il explore au hasard, l'agent ne choisit qu'une configuration autorisée. Il porte sur un temps prédit et ne garantit donc pas le temps mesuré (section 5.4.2).

5.3.3 Architecture du réseau

Une sortie par configuration demanderait 432 sorties. Le réseau choisit plutôt les paramètres l'un après l'autre, dans l'ordre $(\alpha, \text{NBL}, L, \kappa)$, chaque choix tenant compte des précédents [57]. Il comporte un tronc commun et quatre têtes.

Le tronc comporte deux couches de 128 neurones avec activation ReLU et produit une représentation partagée $h \in \mathbb{R}^{128}$. Pour le paramètre d, la tête reçoit h et les choix déjà faits, encodés sous forme *one-hot* :

$$z_d = \text{ReLU}\Big(W_d \cdot [h\,;\, \text{onehot}(a_{<d})] + b_d\Big) \in \mathbb{R}^{64} \tag{5.7}$$

Chaque tête applique ensuite la décomposition *Dueling* (section 2.8.2) :

$$Q_d(s, \cdot \mid a_{<d}) = V_d(z_d) + \Big(A_d(z_d) - \tfrac{1}{|\mathcal{G}_d|} \textstyle\sum_{a'} A_d(z_d)[a']\Big) \tag{5.8}$$

où $\mathcal{G}_d$ est la grille du paramètre. V_d estime la valeur de la situation et A_d l'avantage de chaque valeur. La section 5.5.3 évalue cette décomposition par ablation.

Les quatre têtes produisent au total 19 scores, au lieu de 432. Un score estime la récompense attendue pour une valeur. À la décision, chaque tête retient la valeur autorisée de meilleur score et la transmet à la tête suivante (figure 5.1). Par exemple, sur 500.3, le meilleur score de la tête NBL correspond à 130, qui dépasse le budget : la tête retient 50.

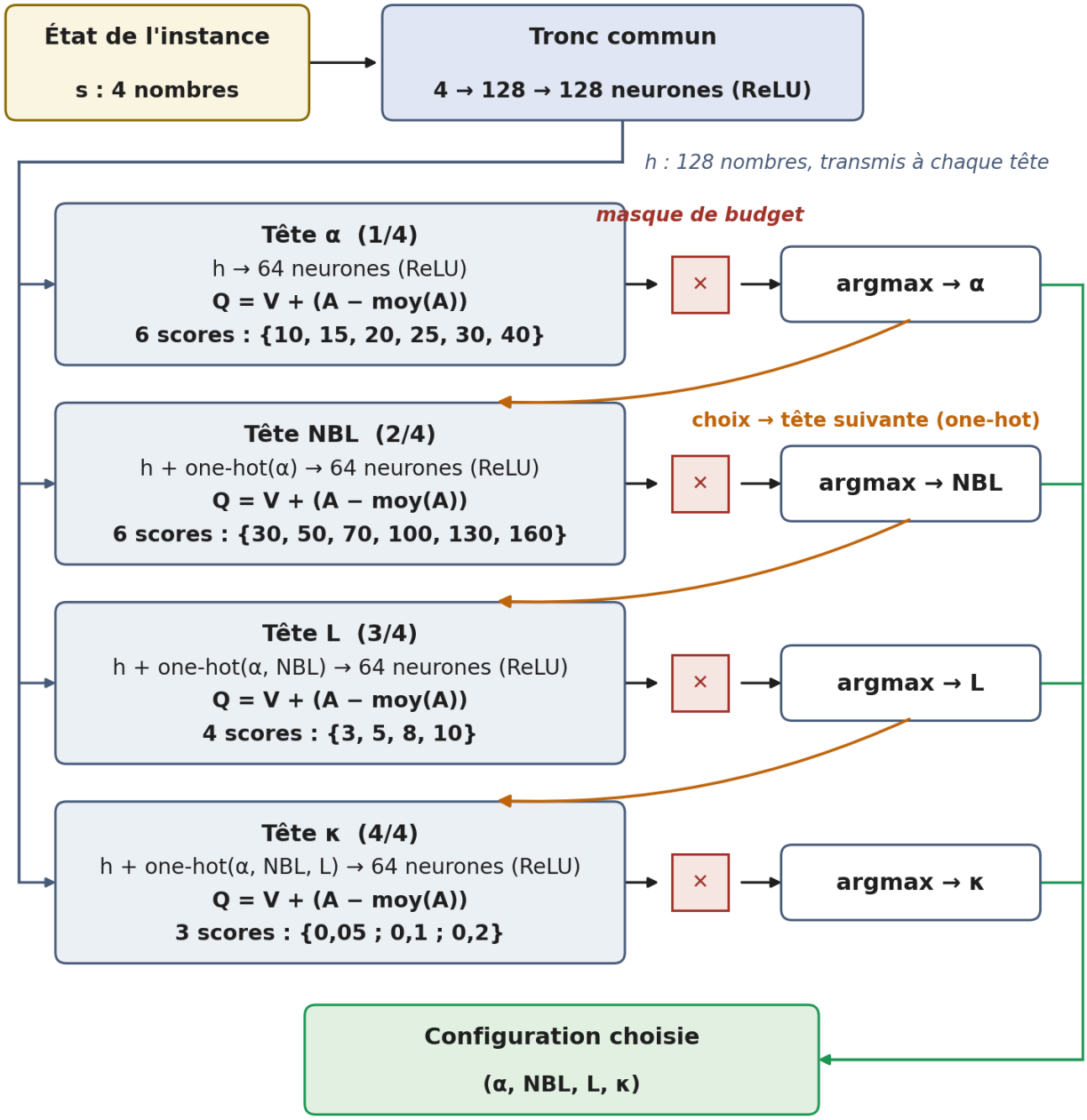


Fig. 5.1 : Architecture du réseau de l'agent : un tronc commun, puis quatre têtes *Dueling*, une par paramètre, qui choisissent les valeurs l'une après l'autre.

5.3.4 Entraînement

L'entraînement répète la boucle suivante :

1. prendre une instance, tour à tour parmi les neuf, et lire son état,
2. choisir une configuration autorisée : au hasard avec la probabilité ε (exploration), sinon celle de meilleurs scores,
3. lancer *W-CMOLS* trois fois avec cette configuration, le solveur étant stochastique, et calculer la récompense (équation 5.4) sur la médiane,
4. ranger l'état, la configuration et la récompense en mémoire,

5. dès que la mémoire contient 32 lignes, en tirer un lot de 32 et rapprocher les scores prédits des récompenses observées.

Le lot privilégie les lignes où le réseau s'est le plus trompé (rejeu priorisé, *PER*). Des poids corrigent ce déséquilibre. Les pertes de Huber des quatre têtes sont regroupées en une seule rétropropagation. L'implémentation conserve un réseau cible, selon le schéma *Double DQN*, mais comme chaque épisode ne comporte qu'une décision, la partie « future » de la cible est nulle : la cible est simplement la récompense. L'algorithme 5.1 détaille une mise à jour.

Algorithme 5.1 (Mise à jour autorégressive avec PER)

```
Entrées : mémoire D des essais (état s, configuration a, récompense r),
          réseau principal Q, taille de lot b = 32
Sortie  : réseau Q mis à jour

si D contient moins de b essais alors
    terminer sans mise à jour
fin si
tirer b essais dans D, en privilégiant ceux où l'erreur passée est forte (PER)
pour chaque essai (s, a, r) du lot faire
    pour chaque paramètre d dans (α, NBL, L, κ) faire
        prédiction_d ← score de la tête d pour la valeur a_d,
                       les valeurs a_<d déjà choisies étant connues
        erreur_d ← r − prédiction_d          // cible = récompense : une seule décision
    fin pour
fin pour
perte ← somme des pertes de Huber des quatre têtes,
        pondérées pour corriger le tirage privilégié
ajuster les poids de Q par rétropropagation de la perte (optimiseur Adam)
mettre à jour la priorité de chaque essai tiré selon son erreur moyenne
rapprocher lentement le réseau cible de Q   // schéma Double DQN, sans effet ici
```

Une fois entraîné, l'agent choisit la configuration d'une instance sans exploration ni nouvel apprentissage. L'algorithme 5.2 détaille ce choix.

Algorithme 5.2 (Choix de la configuration d'une instance par l'agent entraîné)

```
Entrées : instance (n objets, m objectifs), cloud, réseau Q entraîné,
          références de W-CMOLS sur l'instance (HV_base, t_base)
Sortie  : configuration a = (α, NBL, L, κ), puis front rendu par W-CMOLS

s ← état de l'instance (équation 5.1)
B ← 0,8 × t_base                          // budget de l'instance
h ← tronc commun du réseau appliqué à s
pour chaque paramètre d dans (α, NBL, L, κ) faire
    pour chaque valeur v de la grille de d faire
        autorisée(v) ← temps prédit (équation 5.5) de la configuration
                       (a_<d, d = v, paramètres restants à leur valeur
                       la moins coûteuse) ≤ B
    fin pour
    si aucune valeur n'est autorisée alors
        autoriser la moins coûteuse
    fin si
    a_d ← valeur autorisée de meilleur score pour la tête d (h, a_<d)
fin pour
exécuter W-CMOLS avec le cloud et la configuration a
```

Chaque politique est entraînée pendant 25 épisodes par instance, soit 225 épisodes et 675 exécutions du solveur, environ 52 minutes. Elle part d'une politique déjà préentraînée avec le cloud glouton, dont les scores ont déjà un sens : l'exploration ε part donc de 0,4 et non de 1, puis décroît jusqu'à 0,05 par un facteur de 0,99 par épisode.

L'optimiseur est *Adam*, avec un taux d'apprentissage de $5 \cdot 10^{-4}$. Ce nombre d'épisodes suffit à l'affinage : la perte d'entraînement passe d'environ 0,02 à la première mise à jour à moins de $5 \cdot 10^{-4}$ sur le dernier tiers des épisodes. Les politiques sont entraînées avec les clouds du chapitre 4 et les temps de référence de la campagne d'évaluation, c'est-à-dire dans les conditions de leur évaluation.

Trois variantes sont évaluées :

- *SW-CMOLS-GQ* emploie la politique entraînée avec le cloud glouton, sur ce cloud (section 4.3.3),
- *SW-CMOLS-UQ* emploie la politique entraînée avec le cloud union, sur ce cloud (section 4.3.4),

- *SW-CMOLS-LQ* applique la politique de *UQ* au cloud *SW-CMOLS-L* (section 4.3.2) : l'état décrit l'instance et non le cloud, si bien qu'une politique s'applique sans nouvel entraînement à un autre cloud.

5.4 Expérimentations

5.4.1 Protocole expérimental

Instances. Les variantes sont évaluées sur les neuf instances de référence décrites à la section 3.6, les mêmes qu'au chapitre 4.

Paramètres. *W-CMOLS* et les variantes statiques du chapitre 4 gardent les valeurs par défaut : $\alpha = 10$, $\mathrm{NBL} = 100$, $L = 5$ et $\kappa = 0{,}05$ (tableau 3.1). Les variantes contrôlées utilisent, pour chaque instance, les valeurs choisies par l'agent dans les grilles de la section 5.2. L'entraînement utilise un budget de 0,8 fois le temps de *W-CMOLS*, $\lambda = 0{,}10$ et $\nu = 0{,}2$ (sections 5.2 et 5.3).

Métriques. L'hypervolume, calculé comme au chapitre 4, est l'indicateur principal. S'y ajoutent la cardinalité du front, c'est-à-dire le nombre de solutions non dominées rendues, et le temps d'exécution en secondes.

Évaluation. Une fois entraîné, chaque agent est évalué sans exploration sur chacune des neuf instances : le choix de la configuration est déterministe, tandis que le solveur reste stochastique. Chaque combinaison (système, instance) est répétée 50 fois selon le protocole du chapitre 4 : même solveur, mêmes clouds, mêmes germes aléatoires. Seuls les quatre paramètres changent entre une variante statique et sa variante contrôlée, ce qui isole l'effet de la configuration apprise. Les temps de *W-CMOLS* utilisés pour le budget et la pénalité sont ceux de cette campagne.

Tests statistiques. Chaque variante est comparée à *W-CMOLS* par un test de Mann-Whitney U unilatéral, au seuil de 0,05, sans correction pour comparaisons multiples. Chaque variante contrôlée est aussi comparée à sa variante statique, sur le même cloud, dans les deux sens. Le test de Wilcoxon apparié bilatéral sur les neuf moyennes complète ces comparaisons.

5.4.2 Résultats, cardinalité et temps d'exécution

Le tableau 5.1 rapporte l'hypervolume moyen obtenu sur 50 répétitions. Il place côte à côte le solveur de référence, les trois variantes d'amorçage du chapitre 4, et ces mêmes trois variantes dont les quatre paramètres sont choisis par l'agent.

Instance	W-CMOLS	SW-CMOLS-G	SW-CMOLS-GQ	SW-CMOLS-L	SW-CMOLS-LQ	SW-CMOLS-U	SW-CMOLS-UQ
250.2	0,7895	0,8107	0,8097*	**0,8144**	**0,8144***	0,8143	**0,8144***
250.3	0,4311	0,5530	0,5457*	0,5575	0,5549*	**0,5609**	0,5573*
250.4	0,1818	0,3185	0,3213*	0,3245	0,3251*	0,3292	**0,3305***
500.2	0,7261	0,7855	0,7845*	0,7822	0,7819*	**0,7862**	0,7857*
500.3	0,3847	0,5106	0,5051*	**0,5260**	0,5216*	0,5251	0,5220*
500.4	0,1330	0,2922	0,2959*	0,2992	0,2995*	0,3065	**0,3079***
750.2	0,6418	0,7714	0,7674*	0,7708	0,7685*	**0,7728**	0,7711*
750.3	0,3871	0,4991	0,4986*	0,5065	0,5050*	**0,5091**	**0,5091***
750.4	0,1280	0,2771	0,2774*	0,2815	0,2838*	0,2898	**0,2919***
Moyenne	0,4226	0,5354	0,5340	0,5403	0,5394	**0,5438**	0,5433

Tab. 5.1 : Hypervolume moyen sur 50 répétitions.

Le gras indique le meilleur hypervolume de la ligne. (*) : amélioration significative de la variante contrôlée par rapport à *W-CMOLS*, test de Mann-Whitney U, $p < 0{,}05$.

Face à *W-CMOLS*, les trois variantes contrôlées gagnent sur les neuf instances, avec un test significatif dans chaque cas. La moyenne des gains relatifs calculés instance par instance vaut +48,1 % pour *SW-CMOLS-GQ*, +50,1 % pour *SW-CMOLS-LQ* et +52,1 % pour *SW-CMOLS-UQ*. Ce dernier se place au niveau de *SW-CMOLS-U* : son hypervolume moyen (0,5433) reste à 0,1 % de celui de *SW-CMOLS-U* (0,5438), le meilleur de ce mémoire.

Effet propre de la configuration apprise. Chaque variante contrôlée partage son cloud avec une variante statique. Seule la configuration diffère. Sur ces trois paires, l'hypervolume moyen est presque inchangé : $-0{,}26$ % pour *GQ* face à *G*, $-0{,}16$ % pour *LQ* face à *L* et $-0{,}08$ % pour *UQ* face à *U*.

Instance par instance, la variante contrôlée est significativement meilleure sur 2, 2 et 3 instances respectivement, dont 250.4 pour les trois paires. Elle est significativement moins bonne sur 5, 5 et 4 instances, dont 250.3, 500.2, 500.3 et 750.2 pour les trois paires. Le test de Wilcoxon apparié bilatéral ne détecte de différence systématique pour aucune des trois paires (p de 0,25 à 0,73).

La configuration apprise ne vise donc pas à augmenter l'hypervolume : elle réduit surtout les deux paramètres les plus coûteux. Dans les trois variantes, NBL vaut de 30 à 70 selon l'instance, jamais 100 ou plus, et α de 10 à 20. Le réglage retenu varie donc d'une instance à l'autre : c'est ce choix, sous un budget propre à chaque instance, que l'agent automatise. Son effet porte sur le temps d'exécution, présenté plus bas.

Les figures 5.2 à 5.10 illustrent ce résultat sur les neuf instances, dans l'ordre du tableau 5.1. Elles superposent *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées. Les quatre systèmes amorcés produisent des fronts visuellement proches. L'écart principal les sépare du solveur de référence.

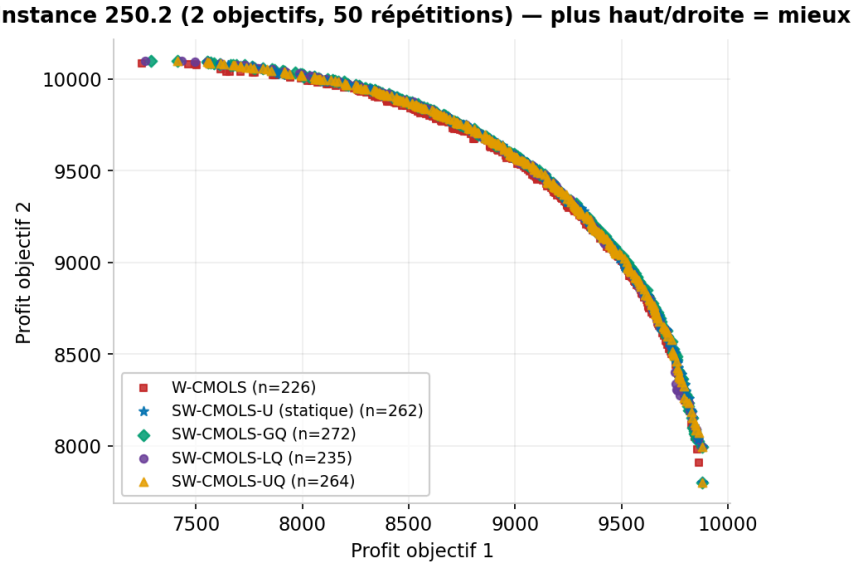


Fig. 5.2 : Instance 250.2 (2 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

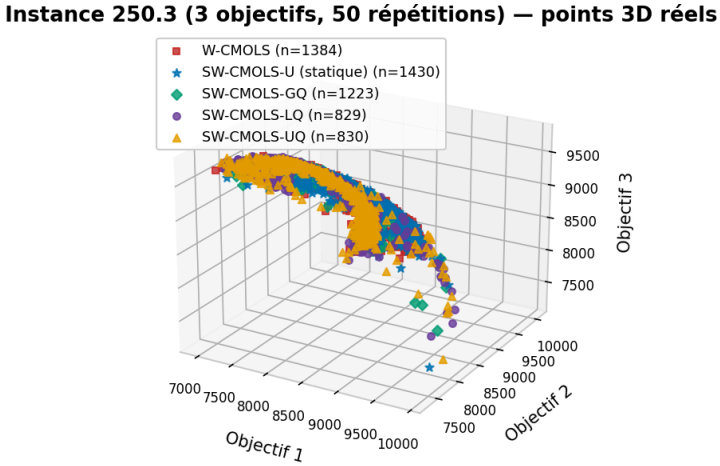


Fig. 5.3 : Instance 250.3 (3 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

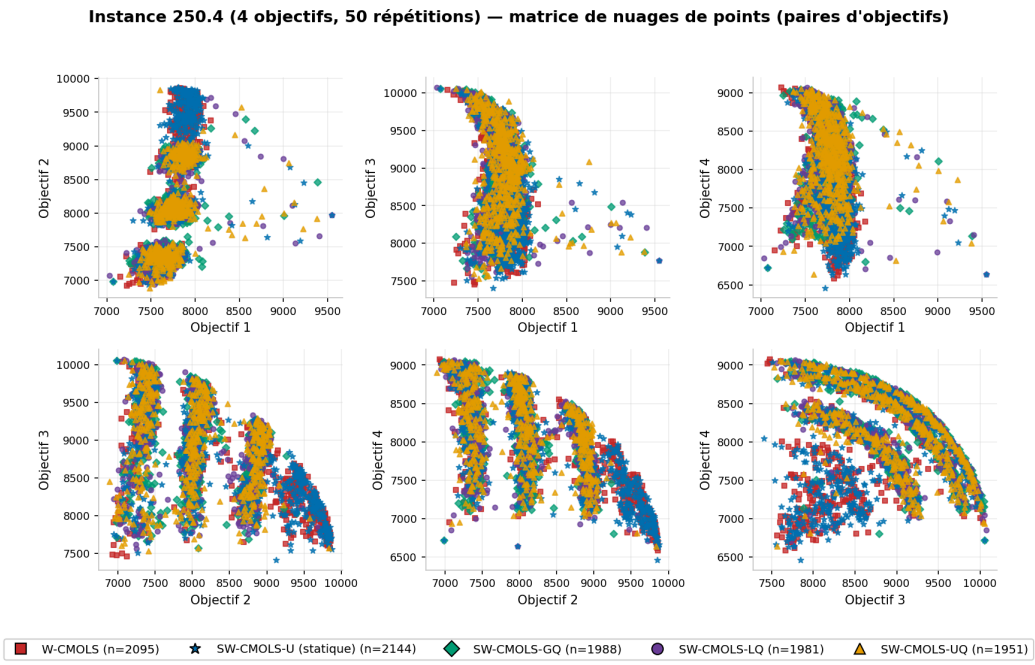


Fig. 5.4 : Instance 250.4 (4 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

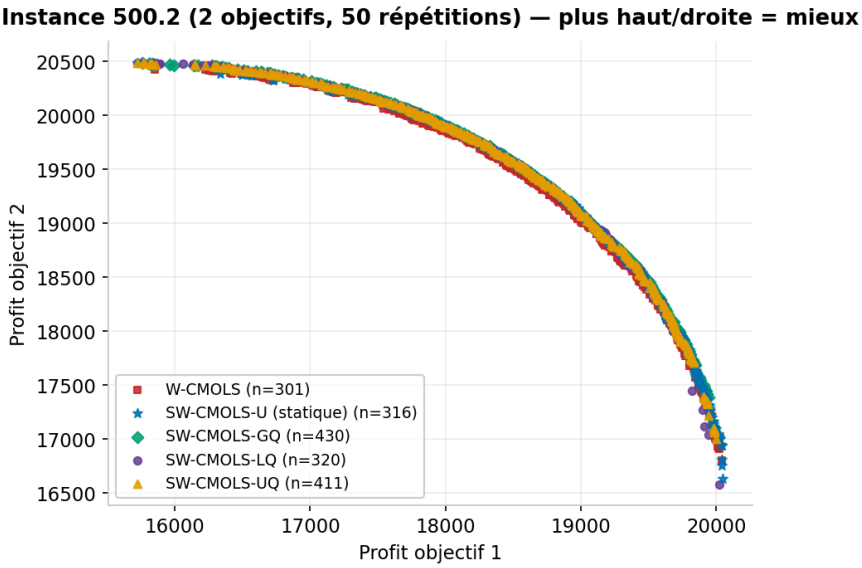


Fig. 5.5 : Instance 500.2 (2 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

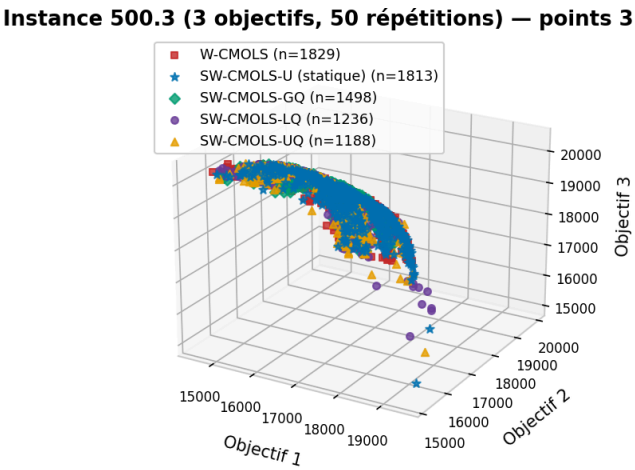


Fig. 5.6 : Instance 500.3 (3 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

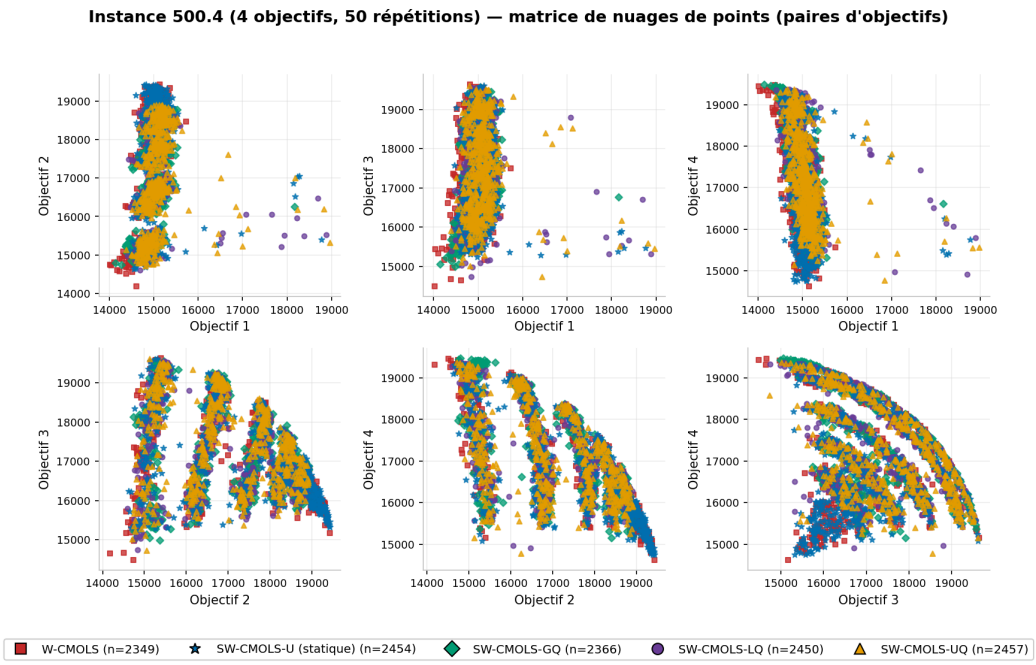


Fig. 5.7 : Instance 500.4 (4 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

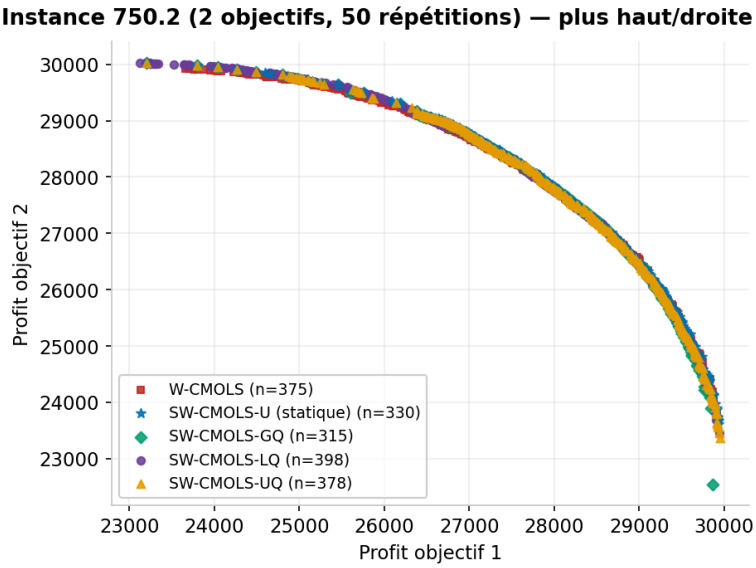


Fig. 5.8 : Instance 750.2 (2 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

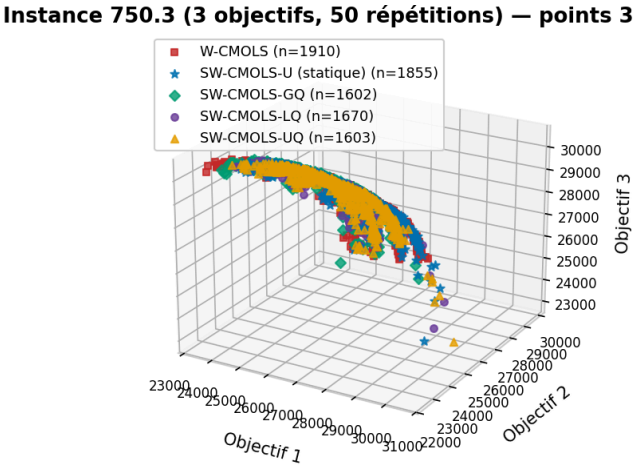


Fig. 5.9 : Instance 750.3 (3 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

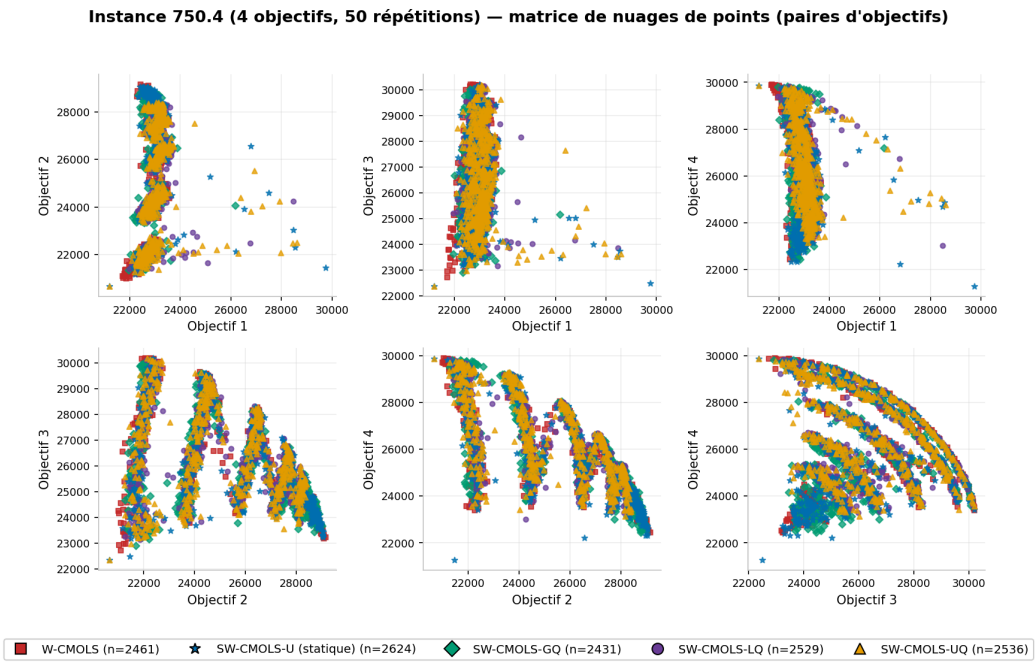


Fig. 5.10 : Instance 750.4 (4 objectifs) : *W-CMOLS*, *SW-CMOLS-U* et les trois variantes contrôlées.

Comme au chapitre 4 (section 4.4.2), l'hypervolume ne dit rien du nombre de solutions remises au décideur ni du temps de calcul. Le tableau 5.2 les rapporte pour les trois variantes contrôlées.

Instance	W-CMOLS	SW-CMOLS-GQ	SW-CMOLS-LQ	SW-CMOLS-UQ	W-CMOLS (s)	SW-CMOLS-GQ (s)	SW-CMOLS-LQ (s)	SW-CMOLS-UQ (s)
250.2	140	105	126	129	0,50	0,41	**0,38**	**0,38**
250.3	843	684	785	710	1,21	0,74	0,87	**0,68**
250.4	1104	1219	1576	1555	2,30	**1,32**	1,79	1,53
500.2	230	146	163	142	1,38	1,15	1,33	**1,06**
500.3	1518	1235	1189	1142	3,30	2,20	1,93	**1,70**
500.4	2029	1802	2343	2150	7,56	**3,21**	5,43	4,32
750.2	301	290	306	292	2,81	**2,01**	2,45	2,72
750.3	2012	1574	1786	1680	6,10	**3,99**	4,83	4,68
750.4	2836	2820	3078	2969	14,42	10,28	9,31	**8,76**
Moyenne	1224	1097	1261	1196	4,40	**2,81**	3,15	2,87

Tab. 5.2 : Cardinalité moyenne du front rendu par exécution et temps moyen par exécution, 50 répétitions. Le gras indique le temps le plus court de la ligne. La cardinalité n'est pas une mesure de qualité et n'est donc pas mise en gras.

En moyenne, les variantes contrôlées rendent de 1 097 à 1 261 solutions par exécution, contre 1 224 pour *W-CMOLS* et de 1 290 à 1 497 pour les variantes statiques. *SW-CMOLS-UQ* en rend davantage que *W-CMOLS* sur les trois instances à quatre objectifs et moins sur les six autres, ce qui reflète un nombre d'itérations plus faible. Les temps sont mesurés sur la même machine, sur les 50 répétitions. Ils excluent la génération du cloud et le choix du réseau, de l'ordre de la milliseconde.

C'est sur le temps que porte l'effet de la configuration apprise. Sur le même cloud, elle réduit le temps cumulé des neuf instances de 44 % pour *GQ* face à *G* (25,3 s contre 45,0 s par répétition), de 53 % pour *LQ* face à *L* (28,3 s contre 60,0 s) et de 54 % pour *UQ* face à *U* (25,8 s contre 56,6 s), pour un hypervolume moyen presque inchangé.

Les trois variantes contrôlées sont aussi plus rapides que *W-CMOLS* sur chacune des neuf instances. *SW-CMOLS-UQ* demande 25,8 s par répétition contre 39,6 s (-35 %) : sur l'ensemble du protocole, ses 450 exécutions durent 21,5 minutes, contre 33,0 pour *W-CMOLS* et 47,2 pour *SW-CMOLS-U*.

Les temps d'une même machine varient d'un jour à l'autre. Les sept systèmes ont donc été remesurés le même jour, en alternance instance par instance, sur 50 répétitions : les hypervolumes sont identiques à la décimale près, et les réductions de temps égales ou supérieures, -47 % pour *GQ* face à *G*, -54 % pour *LQ* face à *L*, -55 % pour *UQ* face à *U* et -36 % pour *UQ* face à *W-CMOLS*. Les trois variantes contrôlées restent plus rapides que *W-CMOLS* sur les neuf instances.

Le masque impose le budget au temps prédit, et l'erreur de prédiction subsiste. Pour *SW-CMOLS-UQ*, le temps mesuré reste sous le budget sur huit instances sur neuf. Les rapports vont de 0,64 (500.3) à 1,21 (750.2), seule instance au-dessus. Le temps total mesuré, 25,8 s, reste sous le budget total de 31,7 s.

Contrôle sans cloud. Pour vérifier que le gain de qualité ne vient pas de la configuration, la politique de *SW-CMOLS-GQ* a aussi été appliquée à *W-CMOLS* sans aucun cloud : l'archive part vide, seuls les paramètres changent. Sur 50 répétitions, l'hypervolume moyen passe de 0,4226 à 0,4004, soit $-6{,}6$ % en moyenne, loin des +48,1 % de *SW-CMOLS-GQ*. La variante est significativement moins bonne sur 8 instances et meilleure sur aucune. Seule 750.2, où le solveur de référence est bimodal (section 4.4.2), voit sa moyenne augmenter, ce qui rend non significatif le test de Wilcoxon apparié sur les neuf moyennes ($p = 0{,}16$). Le temps cumulé baisse de 39,6 s à 18,6 s par répétition (-53 %).

Ce contrôle confirme la répartition des rôles : le cloud apporte la qualité, la configuration apprise réduit le temps. Les réglages appris dépendent du cloud : avec 30 à 70 itérations, une archive partie de zéro n'a pas le temps de rattraper *W-CMOLS*. Une archive déjà amorcée, si.

5.5 Validation de l'agent

Quatre campagnes évaluent l'écart à un oracle empirique, le transfert vers une instance exclue, l'apport des extensions architecturales et la comparaison à *SMAC3*. Elles reprennent le dispositif du tableau 5.1 : mêmes clouds, même solveur, même budget de 0,8 fois le temps de *W-CMOLS* et même récompense. L'oracle et *SMAC3* portent sur les politiques de la section 5.4. L'exclusion d'une instance et l'ablation, sur des agents réentraînés dans ces conditions.

5.5.1 Écart à un oracle empirique

Cette campagne crible les 432 configurations sous le masque de budget : chaque configuration autorisée est exécutée trois fois. Pour chaque instance, les douze meilleures au premier passage et le choix de l'agent sont ensuite réévalués sur dix répétitions. Nous appelons *oracle empirique* la meilleure configuration issue de ce protocole. Ce n'est pas un majorant théorique : le premier criblage reste stochastique. Le tableau 5.3 compare, pour chaque instance, le choix de l'agent à cet oracle.

Instance	Configurations faisables	HV oracle	HV de l'agent	Écart
250.2	51 / 432	0,8148	0,8145	+0,04 %
250.3	53 / 432	0,5595	0,5576	+0,34 %
250.4	32 / 432	0,3332	0,3301	+0,92 %
500.2	64 / 432	0,7873	0,7862	+0,14 %
500.3	51 / 432	0,5228	0,5221	+0,14 %
500.4	41 / 432	0,3119	0,3077	+1,33 %
750.2	77 / 432	0,7749	0,7716	+0,43 %
750.3	57 / 432	0,5117	0,5089	+0,54 %
750.4	43 / 432	0,2953	0,2921	+1,08 %

Tab. 5.3 : Écart entre la configuration choisie par *SW-CMOLS-UQ* et l'oracle empirique issu du criblage des 432 combinaisons, amorçage par cloud union.

La colonne « configurations faisables » compte les combinaisons prédites sous le budget de l'instance, sans le réglage de référence.

L'écart médian à l'oracle empirique est de 0,43 % (moyenne : 0,55 %) et reste inférieur à 1 % sur sept instances. L'agent ne retrouve la configuration exacte de l'oracle sur aucune instance. L'écart maximal, +1,33 %, concerne 500.4.

Cet écart doit être lu avec l'étendue des configurations autorisées. Sous ce budget, elles restent proches les unes des autres : au premier passage, la configuration autorisée médiane se situe à 0,50 % de la meilleure (médiane sur les neuf instances). L'agent fait donc un peu mieux qu'un choix quelconque parmi les configurations autorisées, sans retrouver l'optimum. L'essentiel de la qualité est fixé par le cloud et par le masque.

5.5.2 Généralisation à une instance nouvelle

L'agent du tableau 5.1 est entraîné et évalué sur les mêmes neuf instances. Pour tester le transfert, neuf agents supplémentaires sont entraînés depuis zéro sur huit instances, pendant 60 épisodes par instance, puis évalués sans nouvel entraînement sur la neuvième.

L'instance exclue ne fournit aucune donnée d'entraînement, mais son état contient son hypervolume et son temps de référence, et son budget et le terme constant du modèle de coût sont connus. Cette validation mesure donc un transfert une fois ces références mesurées, et non une prédiction sans aucune information sur l'instance.

Le protocole d'évaluation reste celui de la section 5.4.1. La statistique A_{12} estime la probabilité qu'une exécution contrôlée dépasse une exécution du solveur de référence. Le tableau 5.4 en donne les résultats.

Instance exclue	Gain d'HV	p	A_{12}	Configuration choisie	Verdict
250.2	+3,10 %	3,5e−18	1,000	α=15, NBL=30, L=5, κ=0,1	gagné
250.3	+29,26 %	3,5e−18	1,000	α=20, NBL=30, L=3, κ=0,05	gagné
250.4	+80,97 %	3,5e−18	1,000	α=15, NBL=30, L=3, κ=0,05	gagné
500.2	+8,20 %	3,5e−18	1,000	α=25, NBL=30, L=3, κ=0,2	gagné
500.3	+35,04 %	3,5e−18	1,000	α=20, NBL=30, L=3, κ=0,1	gagné
500.4	+127,76 %	3,5e−18	1,000	α=20, NBL=30, L=5, κ=0,05	gagné
750.2	+18,65 %	1,0	0,327	α=25, NBL=30, L=3, κ=0,1	non conclu
750.3	+31,65 %	3,5e−18	1,000	α=20, NBL=30, L=8, κ=0,2	gagné
750.4	+125,75 %	3,5e−18	1,000	α=15, NBL=30, L=5, κ=0,1	gagné

Tab. 5.4 : Validation croisée par exclusion d'une instance : neuf agents réentraînés depuis zéro sur huit instances, évalués sur la neuvième.

Le gain est mesuré face à *W-CMOLS*, test de Mann-Whitney U unilatéral.

Huit instances exclues sur neuf donnent une amélioration significative avec $A_{12} = 1{,}000$: l'agent transfère ce qu'il a appris à des instances qu'il n'a pas vues, une fois leurs références mesurées.

Le seul cas non concluant est 750.2, dont le solveur de référence a une distribution bimodale (chapitre 4). Malgré une moyenne supérieure, le test de rangs ne détecte pas d'amélioration ($p \simeq 1$, $A_{12} = 0{,}327$). *W-CMOLS* s'effondre sur 10 exécutions sur 50, l'agent sur aucune. Mais hors effondrements, *W-CMOLS* reste légèrement au-dessus en moyenne (0,7643 contre 0,7615). Entraîné sans cette instance, l'agent y choisit 30 itérations, là où la politique entraînée sur les neuf instances en retient 70. Ce cas impose de lire ensemble moyenne et distribution.

Dans cette campagne, le temps mesuré reste sous le budget sur les neuf instances exclues, au plus 0,92 fois le budget (500.2). Le temps cumulé des 50 répétitions est de 1 022 s, contre 1 583 s pour *W-CMOLS* mesuré le même jour (-35 %).

5.5.3 Apport de chaque mécanisme et comparaison à SMAC3

Le tableau 5.5 retire séparément l'architecture *Dueling* et le rejeu priorisé, avec réentraînement dans les mêmes conditions.

Variante	Gain moyen d'HV vs W-CMOLS	Instances gagnées
Agent complet (Dueling + rejeu priorisé)	**+46,79 %**	9 / 9
Sans Dueling	+46,23 %	9 / 9
Sans rejeu priorisé	+46,60 %	9 / 9

Tab. 5.5 : Retrait, un par un, des deux extensions architecturales. Amorçage par cloud glouton, agents entraînés depuis zéro, protocole d'entraînement et d'évaluation inchangé par ailleurs.

Les trois variantes gagnent les neuf instances. L'agent complet devance la variante sans *Dueling* sur sept instances sur neuf, de 0,37 % en moyenne, et la variante sans rejeu priorisé sur quatre seulement, de 0,15 %. Ces écarts sont faibles et une seule graine d'entraînement a été utilisée par architecture : l'ablation ne montre pas d'apport mesurable des deux extensions. La qualité tient d'abord au cloud et au masque. Entraîné depuis zéro, l'agent complet atteint +46,79 %, contre +48,09 % pour *SW-CMOLS-GQ*, dont l'entraînement part d'une politique préentraînée (section 5.3.4).

SMAC3 [60], un outil d'optimisation bayésienne et non d'apprentissage par renforcement, est exécuté séparément sur chaque instance, dans les conditions de *SW-CMOLS-GQ* : cloud glouton, mêmes 432 configurations,

même modèle de faisabilité. Il évalue 25 configurations autorisées sur dix répétitions chacune. Une configuration hors budget est pénalisée sans être exécutée ni compter parmi les 25. Il bat le solveur de référence sur les neuf instances, avec +49,75 % en moyenne, contre +48,09 % pour *SW-CMOLS-GQ*, et dépasse l'hypervolume de l'agent sur les neuf instances (+0,89 % en moyenne, Wilcoxon apparié sur les neuf moyennes, $p = 0{,}004$).

Les deux méthodes n'optimisent pas la même chose. *SMAC3* maximise ici le seul hypervolume. Ses configurations rendent 24 % de solutions en moins que celles de l'agent (7 481 contre 9 875 par exécution, sommées sur les neuf instances), dont la récompense valorise aussi la cardinalité. Exécuté avec cette même récompense, *SMAC3* obtient une récompense légèrement supérieure sur huit instances, la neuvième à égalité, pour un hypervolume identique en moyenne (0,5330 contre 0,5340, $p = 1{,}0$) et 11 % de solutions en plus, avec des configurations plus lentes (23,0 s contre 18,7 s par répétition, mesurées le même jour).

En cherchant sur chaque instance, *SMAC3* gagne donc un peu sur le critère qu'on lui fixe. L'agent reste proche sur tous les critères à la fois. La différence principale porte sur le coût d'utilisation. *SMAC3* recommence sa recherche pour chaque instance : 2 250 exécutions du solveur au total, soit 109 minutes et jusqu'à 36,4 minutes sur 750.4. Une politique entraînée choisit sa configuration en une milliseconde environ, une fois les références de l'instance mesurées. Son entraînement, à partir de la politique préentraînée, a coûté 675 exécutions, une seule fois pour les neuf instances.

5.6 Conclusion

Ce chapitre a présenté *SW-CMOLS-Q*, qui choisit $(\alpha, \mathrm{NBL}, L, \kappa)$ avant l'exécution de *W-CMOLS*. L'agent décrit l'instance par quatre nombres, écarte les configurations dont le temps prédit dépasse 0,8 fois celui de *W-CMOLS*, puis choisit la meilleure configuration autorisée à l'aide d'un réseau DQN à têtes *Dueling*, entraîné avec un rejeu priorisé. La décision étant unique, le problème relève formellement d'un bandit contextuel.

Les trois variantes contrôlées améliorent significativement *W-CMOLS* sur les neuf instances. *SW-CMOLS-UQ* gagne +52,1 % en moyenne. Sur un même cloud, la configuration apprise ne modifie pas l'hypervolume de façon systématique ($-0{,}08$ à $-0{,}26$ %), mais réduit le temps d'exécution de 44 à 54 %. *SW-CMOLS-UQ* conserve ainsi la qualité de *SW-CMOLS-U* en 54 % de temps en moins, et reste plus rapide que *W-CMOLS* sur les neuf instances (soit -35 % au total).

L'écart médian à l'oracle empirique est de 0,43 %, et le transfert vers une instance exclue réussit sur huit instances sur neuf, une fois ses références mesurées. *SMAC3*, qui recommence sa recherche pour chaque instance, obtient un hypervolume légèrement supérieur (+0,9 %), au prix de jusqu'à 36 minutes de recherche par instance.

Ces résultats complètent l'amorçage du chapitre 4 : le cloud apporte la qualité, la configuration apprise le temps. Le chapitre 6 compare les deux contributions aux algorithmes concurrents de la littérature.

Chapitre 6

Comparaison avec les algorithmes de l'état de l'art

6.1 Introduction

Les chapitres 4 et 5 ont montré que l'amorçage par cloud de solutions (*SW-CMOLS*) et la configuration par instance (*SW-CMOLS-Q*) améliorent le solveur de référence *W-CMOLS*. Ce résultat ne suffit pas à situer les contributions par rapport à la littérature. Ce chapitre compare donc *SW-CMOLS-U* et *SW-CMOLS-UQ* à treize algorithmes externes.

Le protocole conserve les opérateurs, la sélection et le traitement natif des contraintes de chaque concurrent. Les seules adaptations autorisées concernent la lecture des instances, le format des sorties et, si nécessaire, le critère d'arrêt. Un algorithme qui exigerait une nouvelle réparation conçue pour cette étude n'est pas retenu.

Les concurrents retenus couvrent la décomposition, la scalarisation, la dominance de Pareto, les indicateurs, l'apprentissage par renforcement et la recherche locale. Tous sont exécutables sur le problème étudié : m objectifs, m contraintes de capacité et des décisions binaires.

6.2 Protocole expérimental

Le protocole vise le même problème, le même nombre de répétitions et un budget commun lorsque les interfaces le permettent. Les exceptions sont indiquées explicitement.

Instances. Les treize concurrents sont évalués sur les neuf instances de Zitzler et Thiele (1999) utilisées aux chapitres 4 et 5. Tous lisent les mêmes poids, profits et capacités. Les bornes de normalisation de l'hypervolume sont également identiques à celles du chapitre 3.

Répétitions. Chaque couple algorithme-instance est évalué sur 20 exécutions. Les chapitres 4 et 5 en contiennent 50 pour nos systèmes. Seules les vingt premières sont utilisées ici afin de conserver la même taille d'échantillon.

Le passage de 50 à 20 exécutions modifie la moyenne de *SW-CMOLS-U* de moins de 0,3 % sur les neuf instances. L'écart est plus marqué pour *W-CMOLS* sur 500.2 ($+2{,}2$ %) et 750.2 ($-5{,}2$ %), où le tableau 4.3 signale déjà une instabilité. Les comparaisons proches sur ces deux instances doivent donc être interprétées avec prudence.

Budget. Les concurrents PlatEMO [67] et PISA reçoivent au plus 200 000 évaluations, avec une population de 100 lorsque leur implémentation l'autorise. *MOEA/D* conserve ses tailles natives, de 150 à 350 selon l'instance. *EAG-MOEA/D* et *GWASF-GA* utilisent les mêmes tailles. *mobkp/PLS* ne proposant pas d'arrêt par évaluations, son budget est temporel.

Pour PlatEMO, le plafond retenu vaut vingt fois la valeur par défaut de 10 000 évaluations. Il est commun aux concurrents concernés, mais ne correspond pas nécessairement au budget recommandé dans chaque article.

Le budget de *W-CMOLS* n'a pas d'équivalent exact. Une insertion ou un retrait met à jour les objectifs en $O(m)$, alors qu'une évaluation complète coûte $O(n \cdot m)$. Un même compteur ne représenterait donc pas le même travail.

Le plafond de 200 000 évaluations sert à harmoniser les concurrents compatibles. Les exceptions sont précisées ci-dessous.

Adaptations et exceptions.

1. L'implémentation de référence de *MOEA/D* utilise un arrêt temporel. Celui-ci est remplacé par le compteur d'évaluations déjà présent dans le code. Les opérateurs restent inchangés. Selon la taille de population, les exécutions s'arrêtent à 100 200, 125 250 ou 175 851 évaluations, toujours sous le plafond.
2. *mobkp/PLS* utilise une limite égale au temps moyen de *W-CMOLS*. Un contrôle prolongé à 180 secondes est présenté en section 6.4.1. Le budget de cette méthode reste donc différent de celui exprimé en évaluations.
3. Les temps sont seulement indicatifs : les concurrents PlatEMO sont exécutés sous MATLAB, tandis que notre solveur est compilé avec Cython. Ils ne servent pas à établir le classement de qualité.
4. *EAG-MOEA/D* et *GWASF-GA* ne gèrent pas les contraintes explicites de PlatEMO 4 et ne produisent alors aucune solution faisable. Ils utilisent donc la formulation du MOKP de PlatEMO 3.5. Pour chaque objet i, la réparation calcule $\max_k(p_{ki}/w_{ki})$, puis retire les objets dans l'ordre croissant de cette valeur jusqu'à satisfaire les capacités. La logique est celle de la plateforme. Seules les capacités proviennent de nos instances. Cette formulation diffère de celle des autres concurrents PlatEMO. Les résultats de ces deux algorithmes décrivent donc leur fonctionnement natif, pas un traitement uniforme des contraintes.

Le tableau 6.1 résume la configuration de chaque concurrent : implémentation, traitement des contraintes, population et critère d'arrêt.

Algorithmes	Implémentation	Traitement des contraintes	Population	Critère d'arrêt
MOEA/D, MOEA/D-GR	C++ des auteurs	Réparation native	150 à 350	Au plus 200 000 évaluations
EAG-MOEA/D, GWASF-GA	PlatEMO, MATLAB	Réparation MOKP de PlatEMO 3.5	150 à 350	200 000 évaluations
C-MOEA-D, MOEA-D-2WA, I-DBEA	PlatEMO, MATLAB	Contraintes explicites	100	200 000 évaluations
DCNSGA-III, CMODRL	PlatEMO, MATLAB	Contraintes explicites	100	200 000 évaluations
mobkp/PLS	C++	Mécanisme natif	-	Temps moyen de W-CMOLS
NSGA-II, SPEA2, IBEA	PISA, C	Réparation native du variateur	100	200 000 évaluations

Tab. 6.1 : Configuration expérimentale des treize concurrents.

Test statistique. Comme aux chapitres 4 et 5, un test de Mann-Whitney U unilatéral est appliqué instance par instance, au seuil de 0,05. Les vingt hypervolumes de chaque concurrent sont recalculés avec les mêmes bornes. Aucune correction pour comparaisons multiples n'est appliquée. La significativité rapportée correspond donc au seuil nominal.

6.3 Algorithmes comparés

6.3.1 Décomposition et scalarisation

Les méthodes de cette section transforment le problème multi-objectif en sous-problèmes scalaires associés à des vecteurs de poids.

MOEA/D : décomposition de référence. *MOEA/D* (*Multiobjective Evolutionary Algorithm based on Decomposition*, Zhang et Li, 2007) [34] associe chaque vecteur de poids à un sous-problème scalaire et l'optimise avec ses voisins. Ce principe est proche de celui de *W-CMOLS*, ce qui en fait le concurrent principal. L'expérience utilise le code C++ original des auteurs.

MOEA/D-GR : remplacement global. *MOEA/D-GR* (*Global Replacement*, Wang, Zhang, Zhou, Gong et Jiao, 2016) [68] utilise le même code, les mêmes opérateurs et la même réparation que *MOEA/D*. La différence porte sur le remplacement : la nouvelle solution est affectée au sous-problème auquel elle convient le mieux, puis elle met à jour son voisinage.

L'article teste six tailles de voisinage sans fixer de valeur par défaut. Les deux meilleures ont été évaluées sur les neuf instances. La meilleure est retenue. La comparaison avec *MOEA/D* mesure donc principalement l'effet de la règle de remplacement.

EAG-MOEA/D : décomposition guidée par archive. *EAG-MOEA/D* (*External Archive Guided MOEA/D*, Cai, Li, Fan et Zhang, 2015) [69] combine la décomposition avec une archive externe. Un sous-problème est sélectionné plus souvent lorsque ses descendants ont récemment intégré l'archive. Le code PlatEMO est conservé. La réparation et les tailles de population sont celles indiquées en section 6.2.

MOEA-D-2WA : deux types de vecteurs de poids. *MOEA-D-2WA* (*Two-type Weight Adjustments*, Jiao, Zeng, Li et Ong, 2021) [70] ajoute la violation des contraintes comme objectif. Des vecteurs de poids distincts guident les solutions faisables et infaisables. Les poids associés aux solutions infaisables sont progressivement supprimés. Le code PlatEMO est utilisé sans modification algorithmique, avec une population de 100. Après filtrage, le front final contient en moyenne 51 à 68 solutions. L'effet de cette taille limitée est contrôlé en section 6.4.5.

GWASF-GA : scalarisation. *GWASF-GA* (*Global Weighting Achievement Scalarizing Function Genetic Algorithm*, Saborido, Ruiz et Luque, 2017) [71] classe la population avec une fonction de type Tchebychev, des vecteurs de poids et deux points de référence, utopique et nadir. Il utilise la réparation de PlatEMO 3.5 et les tailles de population de *MOEA/D*. Il apporte une méthode de scalarisation indépendante de la lignée directe de *MOEA/D*.

C-MOEA-D : dominance sous contraintes. *C-MOEA-D* [72] reprend la décomposition de *MOEA/D* et ajoute une règle de dominance qui favorise les solutions faisables, puis les solutions ayant la plus faible violation. Il ne possède pas de réparation spécifique au sac à dos.

I-DBEA : décomposition à nombreux objectifs. *I-DBEA* (*Improved Decomposition-Based Evolutionary Algorithm*, Asafuddoula, Ray et Sarker, 2015) [73] a été conçu pour l'optimisation continue à nombreux objectifs. Il vérifie si la décomposition seule reste efficace sur ce problème binaire contraint.

6.3.2 Contraintes et apprentissage par renforcement

Cette section regroupe deux autres méthodes qui traitent explicitement les contraintes dans PlatEMO.

DCNSGA-III : transformation dynamique des contraintes. *DCNSGA-III* [74] transforme le problème contraint au cours de la recherche. Cette transformation permet à la population d'explorer temporairement des régions infaisables avant de revenir vers le domaine faisable.

CMODRL : réglage par apprentissage par renforcement. *CMODRL* [63] ajuste les hyperparamètres d'un algorithme évolutionnaire contraint pendant son exécution. *SW-CMOLS-Q* choisit au contraire la configuration avant chaque exécution complète du solveur.

6.3.3 Recherche locale dédiée : mobkp/PLS

Contrairement aux méthodes PlatEMO généralistes, *mobkp* traite spécifiquement le sac à dos binaire multi-objectif.

mobkp [37] est une bibliothèque C++20 qui propose plusieurs algorithmes interruptibles à tout moment. Nous utilisons la recherche locale de Pareto PLS, variante `flip_exchange`, avec sa configuration par défaut : solution initiale vide, archive non ordonnée, file FIFO et objets dans l'ordre du fichier.

Le code algorithmique reste inchangé. Le chargeur accepte le format de Zitzler et Thiele. Seule la mise en forme des neuf fichiers d'instance est adaptée.

Ce concurrent joue un rôle particulier. PLS est une recherche locale, comme *W-CMOLS* (chapitre 3), mais elle part d'un point unique, la solution vide, là où *W-CMOLS* part d'une population diversifiée. Leur écart renseigne sur l'intérêt potentiel d'un amorçage diversifié, sans constituer une ablation stricte puisque les deux implémentations diffèrent aussi par leurs voisinages et leur gestion d'archive.

6.3.4 Références classiques : NSGA-II, SPEA2 et IBEA

NSGA-II, *SPEA2* et *IBEA* sont trois références classiques de l'optimisation multi-objectif. Les expériences utilisent les sélecteurs C distribués avec PISA, et non une réimplémentation récente.

PISA [75] sépare deux programmes qui communiquent par fichiers. Le *variateur* contient le problème, les opérateurs et le traitement des contraintes. Le *sélecteur* contient l'algorithme de sélection.

Le variateur utilisé est le module sac à dos d'origine, avec sa réparation gloutonne par ratio profit/poids. Les trois sélecteurs restent inchangés.

NSGA-II (Deb, Pratap, Agarwal et Meyarivan, 2002) [32] repose sur le tri par fronts de non-domination et la distance de surpeuplement. *SPEA2* (Zitzler, Laumanns et Thiele, 2001) [33] calcule une force fondée sur le nombre de solutions dominées et garde une archive externe. Il succède à SPEA, l'algorithme de l'article qui a introduit ce jeu d'instances. *IBEA* (Zitzler et Künzli, 2004) [11] remplace la dominance par un indicateur binaire de qualité, et fonde toute sa sélection sur cet indicateur.

Ces trois méthodes apportent aussi un environnement C compilé, indépendant de PlatEMO et du code C++ de *MOEA/D*.

Le variateur d'origine génère une instance aléatoire. L'adaptation effectuée remplace cette génération par la lecture des neuf instances du mémoire. Les opérateurs, la réparation et les sélecteurs ne changent pas.

6.4 Résultats

Règle de lecture. Les tests sont effectués instance par instance au seuil nominal de 0,05, sans correction pour les 117 comparaisons. Ils sont donc exploratoires. Le cas le plus serré est 250.2 face à *MOEA/D* : l'écart y est faible (+0,43 % pour *SW-CMOLS-U*), mais régulier sur les vingt répétitions ($p < 10^{-7}$ pour *SW-CMOLS-U* comme pour *SW-CMOLS-UQ*). Les vingt valeurs individuelles restent disponibles dans le dépôt de code.

6.4.1 Comparaison avec SW-CMOLS-U

Le tableau 6.2 compare *SW-CMOLS-U* à *W-CMOLS* et aux treize concurrents externes, classés par hypervolume moyen décroissant.

Système	250.2	250.3	250.4	500.2	500.3	500.4	750.2	750.3	750.4	Moyenne	Sign.
W-CMOLS	0,7895	0,4301	0,1825	0,7421	0,3830	0,1315	0,6085	0,3859	0,1291	0,4202	-
SW-CMOLS-U	**0,8143**	**0,5610**	**0,3293**	**0,7861**	**0,5247**	**0,3065**	**0,7729**	**0,5092**	**0,2905**	**0,5438**	-
MOEA/D	0,8108	0,5263	0,2606	0,7616	0,4503	0,1737	0,7296	0,3714	0,1319	0,4685	9/9
EAG-MOEA/D	0,7530	0,4913	0,2649	0,6973	0,4402	0,2159	0,6773	0,3824	0,1964	0,4576	9/9
C-MOEA-D	0,7848	0,4608	0,2239	0,7362	0,4104	0,1702	0,6950	0,3538	0,1415	0,4418	9/9
MOEA-D-2WA	0,7824	0,4574	0,2101	0,7388	0,4123	0,1613	0,7067	0,3533	0,1433	0,4406	9/9
GWASF-GA	0,7824	0,4669	0,2451	0,7058	0,4131	0,1704	0,6771	0,3402	0,1461	0,4386	9/9
MOEA/D-GR	0,8007	0,4906	0,2435	0,7324	0,3955	0,1390	0,6876	0,2918	0,1023	0,4315	9/9
IBEA	0,7407	0,4421	0,2318	0,6702	0,3730	0,1792	0,6368	0,3186	0,1546	0,4163	9/9
SPEA2	0,7364	0,4250	0,1848	0,6566	0,3663	0,1385	0,6218	0,3130	0,1188	0,3957	9/9
NSGA-II	0,7502	0,3705	0,1296	0,6877	0,3297	0,0868	0,6496	0,2846	0,0754	0,3738	9/9
mobkp/PLS	0,7159	0,3475	0,1523	0,6700	0,3370	0,1053	0,6628	0,2754	0,0777	0,3716	9/9
DCNSGA-III	0,7088	0,3960	0,1723	0,6314	0,3127	0,1291	0,5872	0,2696	0,1080	0,3683	9/9
CMODRL	0,6229	0,3626	0,1889	0,5208	0,2789	0,1194	0,4734	0,2278	0,0931	0,3209	9/9
I-DBEA	0,6276	0,2436	0,1001	0,4501	0,1665	0,0506	0,3611	0,1041	0,0306	0,2371	9/9

Tab. 6.2 : Hypervolume moyen (*SW-CMOLS-U*, *W-CMOLS* et treize concurrents : 20 répétitions, sans exception).

Concurrents ordonnés par hypervolume moyen décroissant. La colonne « Sign. » donne le nombre d'instances, sur neuf, où *SW-CMOLS-U* est supérieur selon le test de Mann-Whitney U unilatéral au seuil nominal $p < 0{,}05$. Les 117 comparaisons franchissent ce seuil sans correction de multiplicité.

SW-CMOLS-U obtient une moyenne supérieure dans les 117 couples instance-concurrent, avec un test significatif au seuil nominal dans chaque cas. L'écart le plus faible concerne *MOEA/D* sur 250.2 : 0,8143 contre 0,8108, soit $+0{,}43$ %. Face au même concurrent, il atteint $+120{,}3$ % sur 750.4. Cette progression est décrite selon la taille et le nombre d'objectifs en section 6.4.4.

Six concurrents dépassent *W-CMOLS* en moyenne : *MOEA/D*, *EAG-MOEA/D*, *C-MOEA-D*, *MOEA-D-2WA*, *GWASF-GA* et *MOEA/D-GR*. Tous reposent sur la décomposition ou la scalarisation. *EAG-MOEA/D* est le meilleur concurrent sur 750.3 et sur les trois instances à quatre objectifs. Parmi les autres méthodes, *IBEA* dépasse *W-CMOLS* sur cinq instances, mais sa moyenne reste légèrement inférieure (0,4163 contre 0,4202).

Deux réserves complètent cette lecture. Sur 500.2, la moyenne de *W-CMOLS* dépend du sous-échantillon de vingt répétitions à cause de sa bimodalité (tableau 4.3). Pour *mobkp/PLS*, un contrôle porté à 180 secondes termine sur sept instances sans combler l'écart. Les instances 500.4 et 750.4 restent limitées par le temps. Ces résultats ne permettent pas d'isoler l'effet de l'initialisation, car les voisinages et les archives diffèrent aussi.

6.4.2 Comparaison avec SW-CMOLS-UQ

Le tableau 6.3 reprend la même comparaison, dans le même ordre, pour *SW-CMOLS-UQ* (chapitre 5), la variante de *SW-CMOLS-Q* combinant configuration par instance et cloud union.

Système	250.2	250.3	250.4	500.2	500.3	500.4	750.2	750.3	750.4	Moyenne	Sign.
W-CMOLS	0,7895	0,4301	0,1825	0,7421	0,3830	0,1315	0,6085	0,3859	0,1291	0,4202	-
SW-CMOLS-UQ	**0,8144**	**0,5575**	**0,3306**	**0,7857**	**0,5223**	**0,3080**	**0,7716**	**0,5084**	**0,2919**	**0,5434**	-
MOEA/D	0,8108	0,5263	0,2606	0,7616	0,4503	0,1737	0,7296	0,3714	0,1319	0,4685	9/9
EAG-MOEA/D	0,7530	0,4913	0,2649	0,6973	0,4402	0,2159	0,6773	0,3824	0,1964	0,4576	9/9
C-MOEA-D	0,7848	0,4608	0,2239	0,7362	0,4104	0,1702	0,6950	0,3538	0,1415	0,4418	9/9
MOEA-D-2WA	0,7824	0,4574	0,2101	0,7388	0,4123	0,1613	0,7067	0,3533	0,1433	0,4406	9/9
GWASF-GA	0,7824	0,4669	0,2451	0,7058	0,4131	0,1704	0,6771	0,3402	0,1461	0,4386	9/9
MOEA/D-GR	0,8007	0,4906	0,2435	0,7324	0,3955	0,1390	0,6876	0,2918	0,1023	0,4315	9/9
IBEA	0,7407	0,4421	0,2318	0,6702	0,3730	0,1792	0,6368	0,3186	0,1546	0,4163	9/9
SPEA2	0,7364	0,4250	0,1848	0,6566	0,3663	0,1385	0,6218	0,3130	0,1188	0,3957	9/9
NSGA-II	0,7502	0,3705	0,1296	0,6877	0,3297	0,0868	0,6496	0,2846	0,0754	0,3738	9/9
mobkp/PLS	0,7159	0,3475	0,1523	0,6700	0,3370	0,1053	0,6628	0,2754	0,0777	0,3716	9/9
DCNSGA-III	0,7088	0,3960	0,1723	0,6314	0,3127	0,1291	0,5872	0,2696	0,1080	0,3683	9/9
CMODRL	0,6229	0,3626	0,1889	0,5208	0,2789	0,1194	0,4734	0,2278	0,0931	0,3209	9/9
I-DBEA	0,6276	0,2436	0,1001	0,4501	0,1665	0,0506	0,3611	0,1041	0,0306	0,2371	9/9

Tab. 6.3 : Hypervolume moyen (*SW-CMOLS-UQ*, *W-CMOLS* et treize concurrents : 20 répétitions, sans exception).

Concurrents ordonnés par hypervolume moyen décroissant. La colonne « Sign. » donne le nombre d'instances, sur neuf, où *SW-CMOLS-UQ* est supérieur au seuil nominal $p < 0{,}05$. Les 117 comparaisons franchissent ce seuil sans correction de multiplicité.

SW-CMOLS-UQ présente une moyenne supérieure à *W-CMOLS* et aux treize concurrents externes sur les neuf instances, avec un test significatif dans chacune des 117 comparaisons. Sur 250.2, l'écart avec *MOEA/D* reste le plus faible (0,8144 contre 0,8108). La configuration par instance ne modifie donc pas le classement observé pour *SW-CMOLS-U*.

Contrôle global centré sur UQ. Les neuf instances sont également traitées comme neuf blocs. Sur *SW-CMOLS-UQ* et les treize concurrents, le test de Friedman rejette l'égalité des rangs ($\chi^2(13) = 100{,}02$, $p = 1{,}64 \times 10^{-15}$). Les comparaisons postérieures utilisent ensuite le test exact de Wilcoxon unilatéral sur les neuf moyennes d'instance. La valeur brute est $p = 0{,}001953$ face à chacun des treize concurrents. Après correction de Holm sur cette famille de treize tests, les treize valeurs ajustées valent $0{,}02539$. Ce contrôle porte sur les instances comme blocs. Il complète, sans remplacer, les tests de Mann-Whitney réalisés sur les répétitions de chaque instance. Il ne concerne pas la comparaison interne entre *SW-CMOLS-UQ* et *SW-CMOLS-U*.

Les figures 6.1 à 6.9 illustrent les tableaux sur les neuf instances. Elles superposent *W-CMOLS*, *SW-CMOLS-UQ* et quatre concurrents : *MOEA/D* (2007) et *EAG-MOEA/D* (2015), les deux premiers du classement, *GWASF-GA* (2017) pour la scalarisation, et *mobkp/PLS* (2022), le seul dédié au sac à dos. *SW-CMOLS-U* et les autres concurrents sont omis pour préserver la lisibilité, mais restent présents dans les tableaux. *MOEA-D-2WA* est tracé aux figures 6.10 à 6.18.

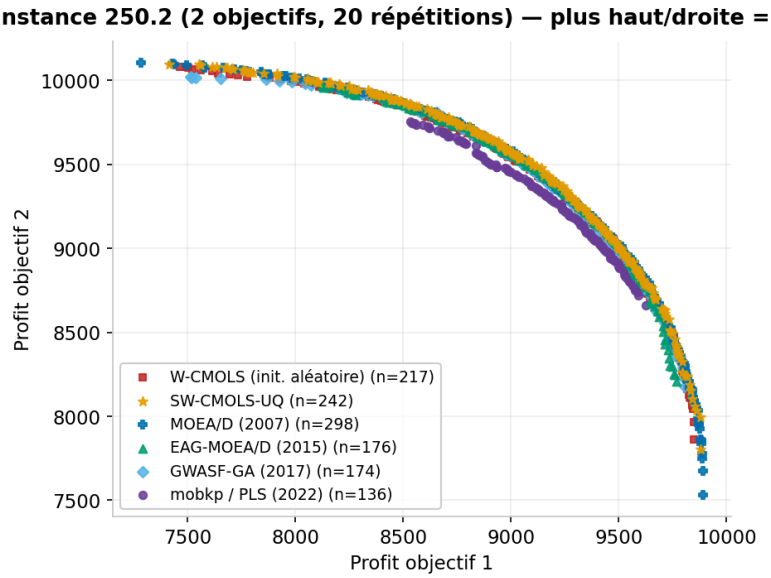


Fig. 6.1 : Instance 250.2 (2 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

Sur 250.2, les fronts de *MOEA/D*, *SW-CMOLS-UQ* et *W-CMOLS* sont proches. L'écart entre *SW-CMOLS-UQ* et *MOEA/D* est le plus faible du tableau 6.3 (0,8144 contre 0,8108), mais il est significatif. *EAG-MOEA/D* et *GWASF-GA* restent sous *W-CMOLS* : 0,7530 et 0,7824 contre 0,7895.

Le front de *mobkp/PLS* décroche dans la région centrale. Le contrôle prolongé montre que ce résultat est stable sur cette instance, sans permettre d'attribuer l'écart au seul amorçage.

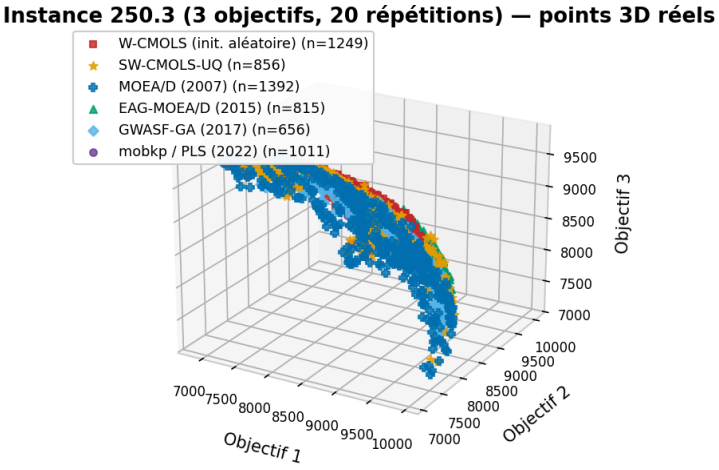


Fig. 6.2 : Instance 250.3 (3 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

Le passage à trois objectifs sépare ce que 250.2 confondait : *SW-CMOLS-UQ* (0,5575) devance *MOEA/D* (0,5263), puis *EAG-MOEA/D* (0,4913) et *GWASF-GA* (0,4669), tous trois au-dessus de *W-CMOLS* (0,4301), tandis que *mobkp/PLS* (0,3475) reste en deçà. La projection en trois dimensions masque une partie des nuages. Les valeurs du tableau 6.3 font foi.

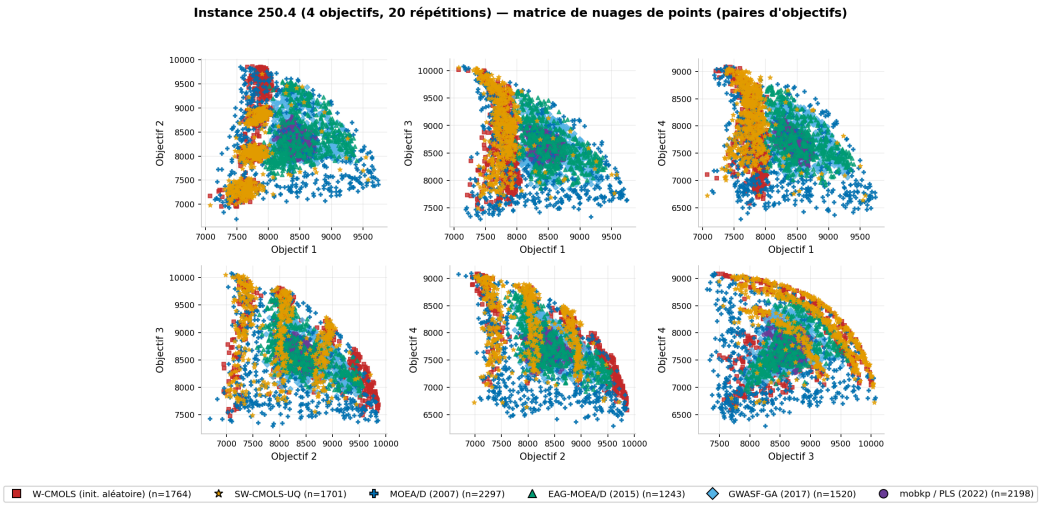


Fig. 6.3 : Instance 250.4 (4 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

Première des trois instances à quatre objectifs, *EAG-MOEA/D* y devient le meilleur concurrent (0,2649 contre 0,2606 pour *MOEA/D*), loin toutefois de *SW-CMOLS-UQ* (0,3306). Les panneaux montrent deux comportements distincts : *EAG-MOEA/D* et *GWASF-GA* concentrent leurs solutions dans un nuage central compact, tandis que *MOEA/D* les disperse largement, y compris vers l'intérieur de l'enveloppe. *SW-CMOLS-UQ* et *W-CMOLS* occupent les bords extérieurs, en particulier sur la paire formée par les objectifs 3 et 4.

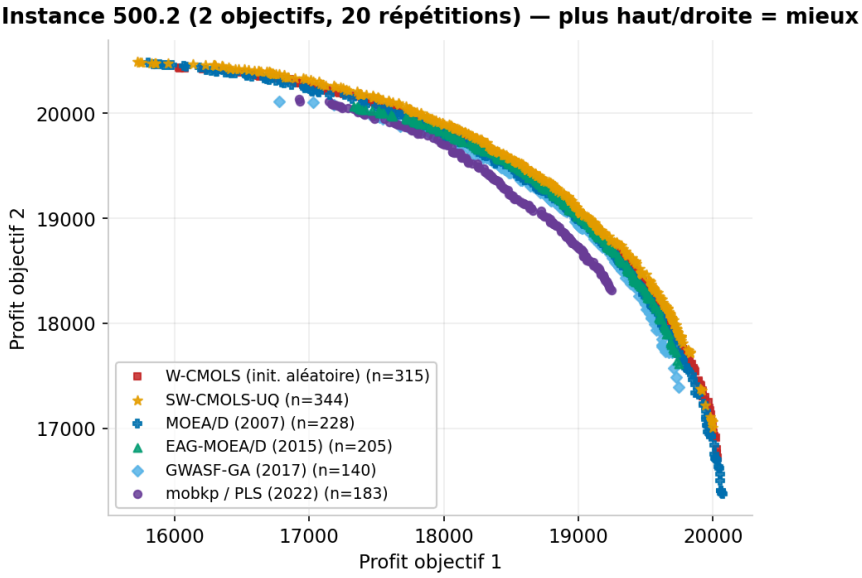


Fig. 6.4 : Instance 500.2 (2 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

Même configuration qu'en 250.2, mais les écarts se creusent avec la taille : *SW-CMOLS-UQ* tient le bord extérieur du front (0,7857 contre 0,7616 pour *MOEA/D*). *EAG-MOEA/D* et *GWASF-GA* passent sous *W-CMOLS* (0,6973 et 0,7058 contre 0,7421), et *mobkp/PLS* décroche nettement dans la région centrale.

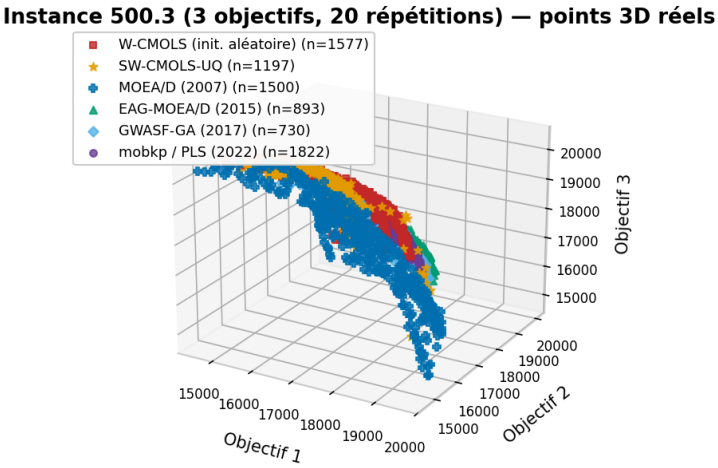


Fig. 6.5 : Instance 500.3 (3 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

SW-CMOLS-UQ (0,5223) conserve une avance nette sur *MOEA/D* (0,4503), *EAG-MOEA/D* (0,4402) et *GWASF-GA* (0,4131), qui dépassent tous trois *W-CMOLS* (0,3830). *mobkp/PLS* (0,3370) est le seul des quatre concurrents tracés à rester en deçà.

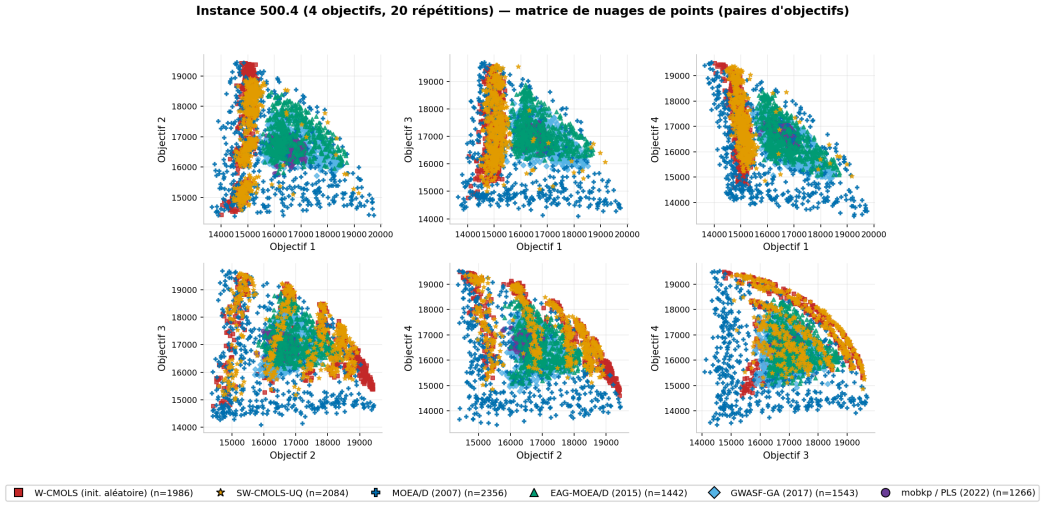


Fig. 6.6 : Instance 500.4 (4 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

EAG-MOEA/D y devance nettement *MOEA/D* (0,2159 contre 0,1737), mais reste loin de *SW-CMOLS-UQ* (0,3080). Les panneaux reproduisent les comportements de la figure 6.3 : nuage central compact pour *EAG-MOEA/D* et *GWASF-GA*, large dispersion de *MOEA/D*, bords extérieurs tenus par *SW-CMOLS-UQ* et *W-CMOLS*. Ces projections montrent des compromis différents entre couverture et convergence. Un tracé en deux dimensions ne suffit pas à expliquer l'hypervolume complet.

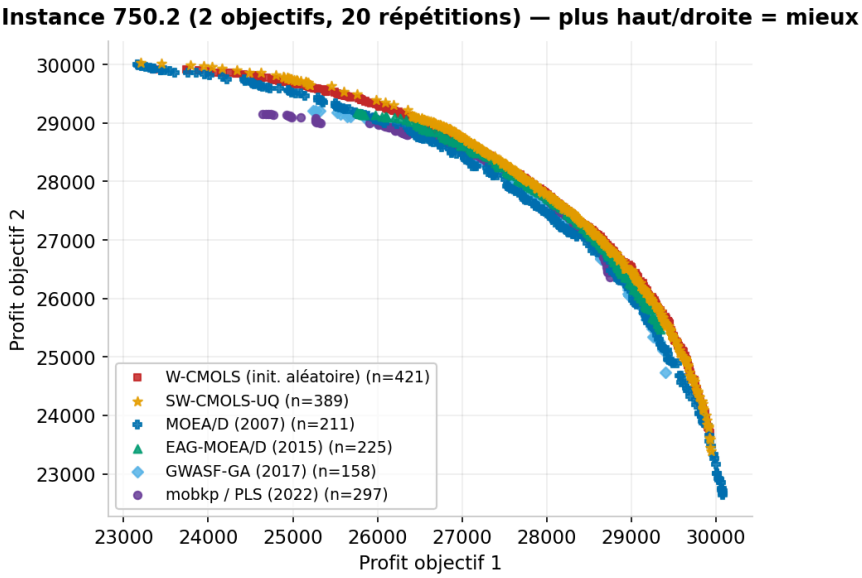


Fig. 6.7 : Instance 750.2 (2 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

Sur la plus grande des instances à deux objectifs, *SW-CMOLS-UQ* tient le bord extérieur du front sur presque toute sa longueur. *MOEA/D* décroche sur la région centrale, bien qu'il atteigne plus loin à l'extrémité inférieure droite. *EAG-MOEA/D* et *GWASF-GA* suivent à l'intérieur, avec des hypervolumes presque identiques (0,6773 et 0,6771). Le nuage de *mobkp/PLS* apparaît détaché en haut à gauche.

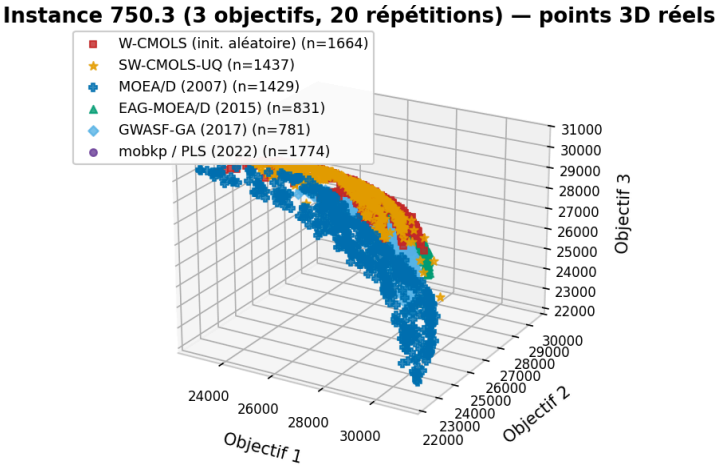


Fig. 6.8 : Instance 750.3 (3 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

Sur 750.3, *MOEA/D* passe sous *W-CMOLS* (0,3714 contre 0,3859). *EAG-MOEA/D* est le meilleur concurrent avec 0,3824, devant *GWASF-GA* (0,3402) et *mobkp/PLS* (0,2754). *SW-CMOLS-UQ* obtient 0,5084.

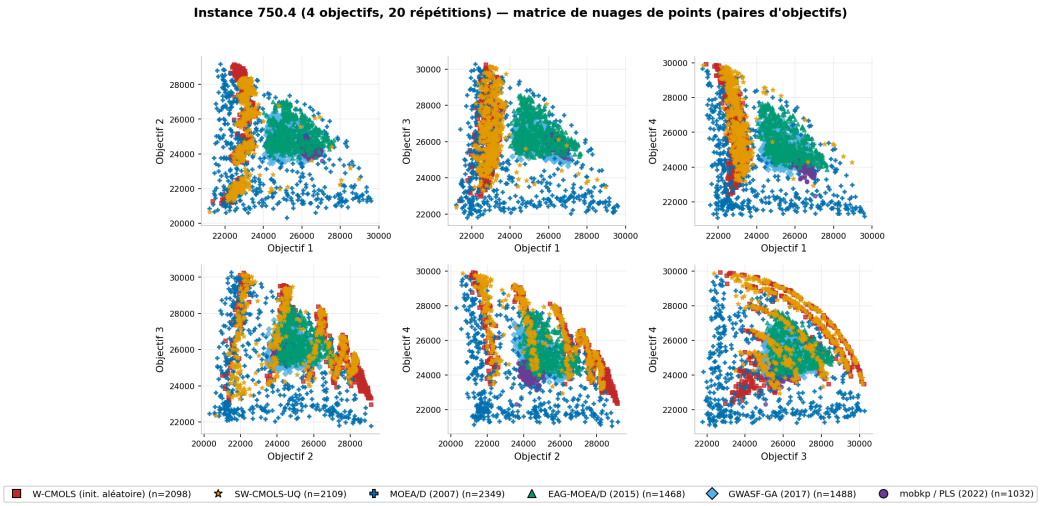


Fig. 6.9 : Instance 750.4 (4 objectifs) : *SW-CMOLS-UQ* face à quatre concurrents externes.

Sur l'instance la plus grande et la plus complexe des neuf, l'écart atteint son maximum : *SW-CMOLS-UQ* (0,2919) obtient plus du double de l'hypervolume de *MOEA/D* (0,1319) et environ 49 % de plus qu'*EAG-MOEA/D* (0,1964), meilleur concurrent de l'instance. Les comportements relevés aux figures 6.3 et 6.6 s'y retrouvent amplifiés.

6.4.3 Fronts de MOEA/D et MOEA-D-2WA

Les figures 6.10 à 6.18 comparent les fronts de *SW-CMOLS-U* et *SW-CMOLS-UQ* à ceux de *MOEA/D* et *MOEA-D-2WA*. *EAG-MOEA/D* et *GWASF-GA* figurent déjà dans la série précédente.

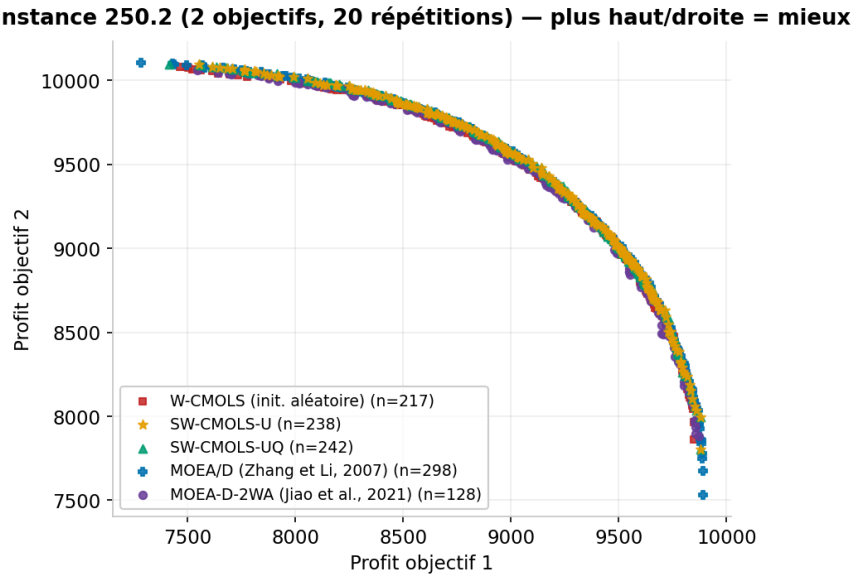


Fig. 6.10 : Instance 250.2 (2 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 250.2, les cinq fronts sont proches. *MOEA-D-2WA* reste légèrement en retrait dans la partie centrale.

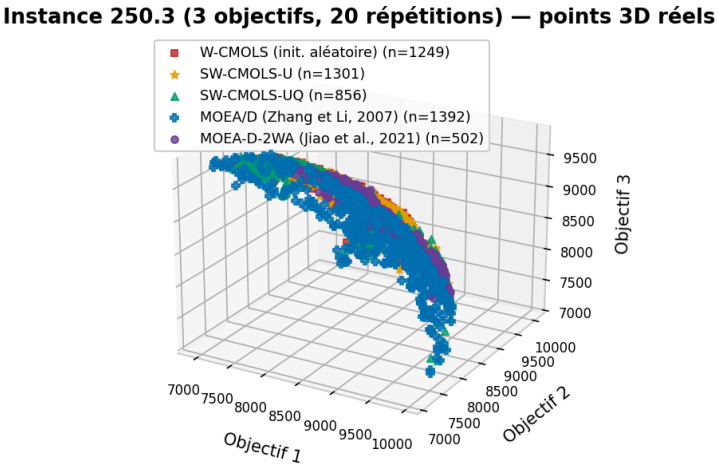


Fig. 6.11 : Instance 250.3 (3 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 250.3, *SW-CMOLS-U* et *SW-CMOLS-UQ* obtiennent les fronts les plus externes, suivis de *MOEA/D* puis de *MOEA-D-2WA*.

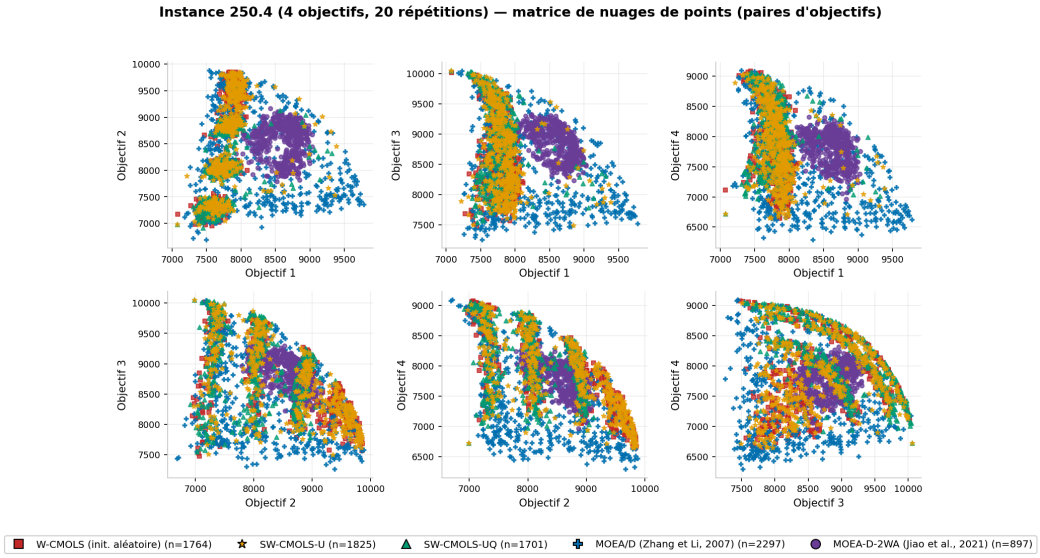


Fig. 6.12 : Instance 250.4 (4 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 250.4, le nuage de *MOEA-D-2WA* est plus compact que celui de nos systèmes, ce qui est cohérent avec sa population fixe de 100 solutions.

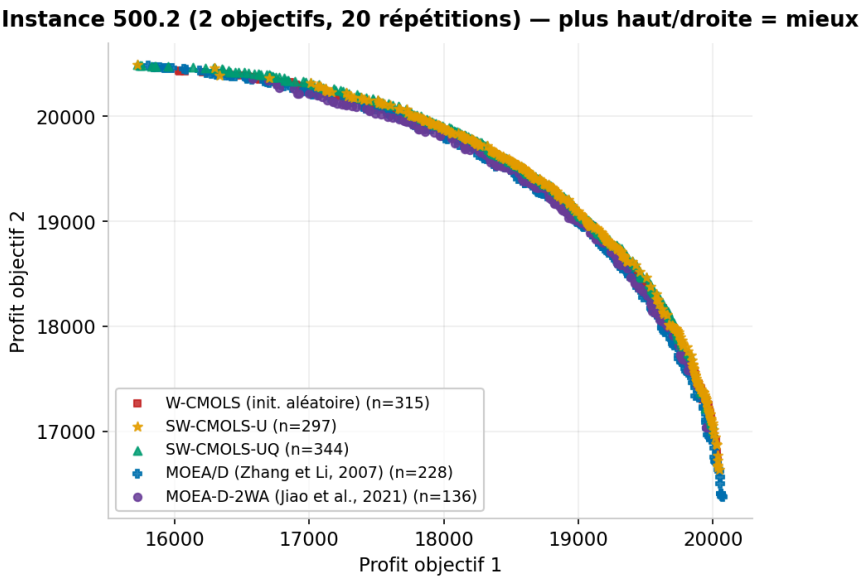


Fig. 6.13 : Instance 500.2 (2 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 500.2, l'écart apparaît surtout dans la partie centrale du front : *SW-CMOLS-UQ* précède *MOEA/D*, puis *MOEA-D-2WA*.

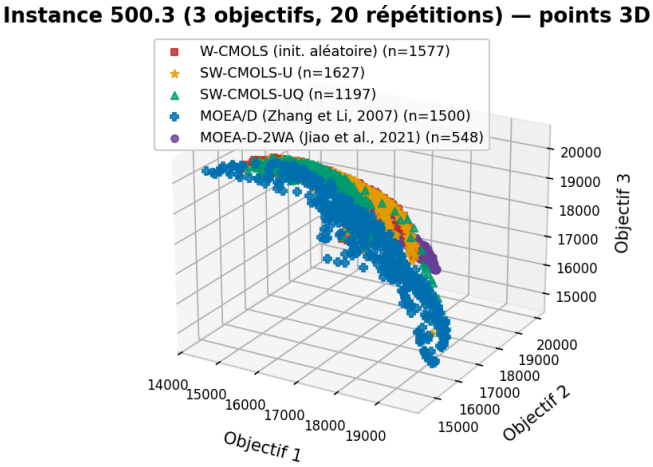


Fig. 6.14 : Instance 500.3 (3 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 500.3, nos deux systèmes précèdent les deux concurrents. *MOEA/D* et *MOEA-D-2WA* restent toutefois au-dessus de *W-CMOLS* dans le tableau 6.3.

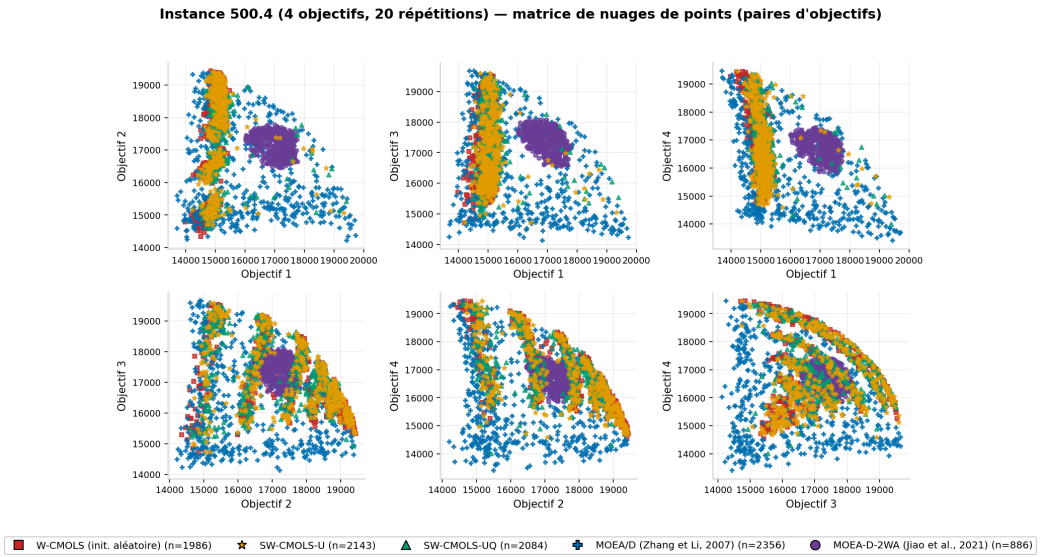


Fig. 6.15 : Instance 500.4 (4 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 500.4, l'hypervolume de *SW-CMOLS-UQ* vaut 0,3080 contre 0,1613 pour *MOEA-D-2WA*, soit un rapport de 1,91.

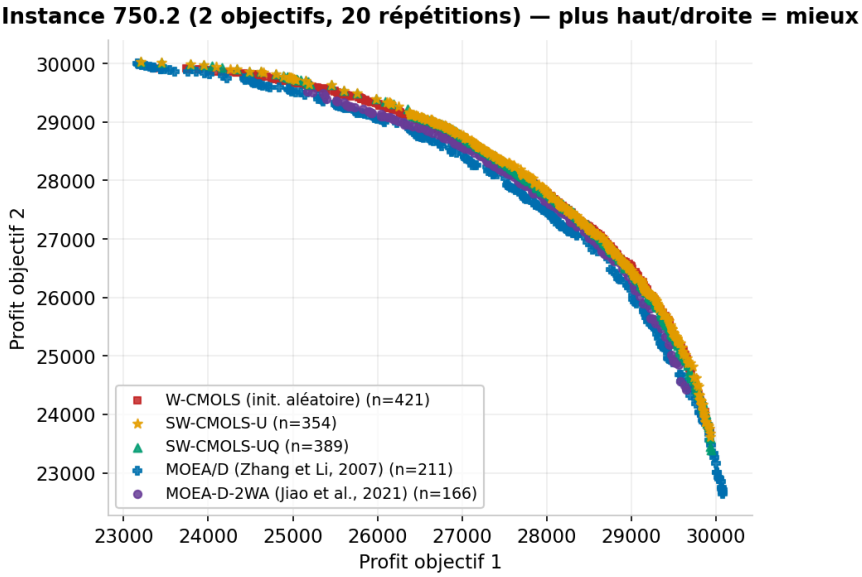


Fig. 6.16 : Instance 750.2 (2 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 750.2, *SW-CMOLS-U* et *SW-CMOLS-UQ* couvrent le bord extérieur. *MOEA-D-2WA* suit à l'intérieur. *MOEA/D* couvre mieux certaines extrémités, mais moins bien la région centrale.

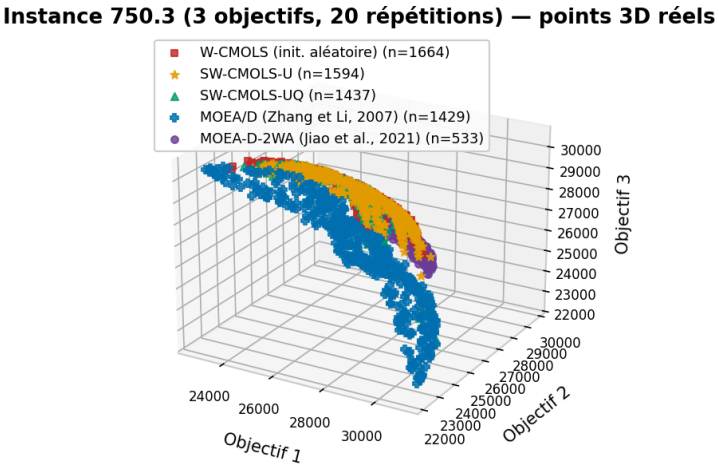


Fig. 6.17 : Instance 750.3 (3 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 750.3, *MOEA/D* et *MOEA-D-2WA* sont en retrait par rapport à *W-CMOLS*, tandis que nos deux systèmes obtiennent les hypervolumes les plus élevés.

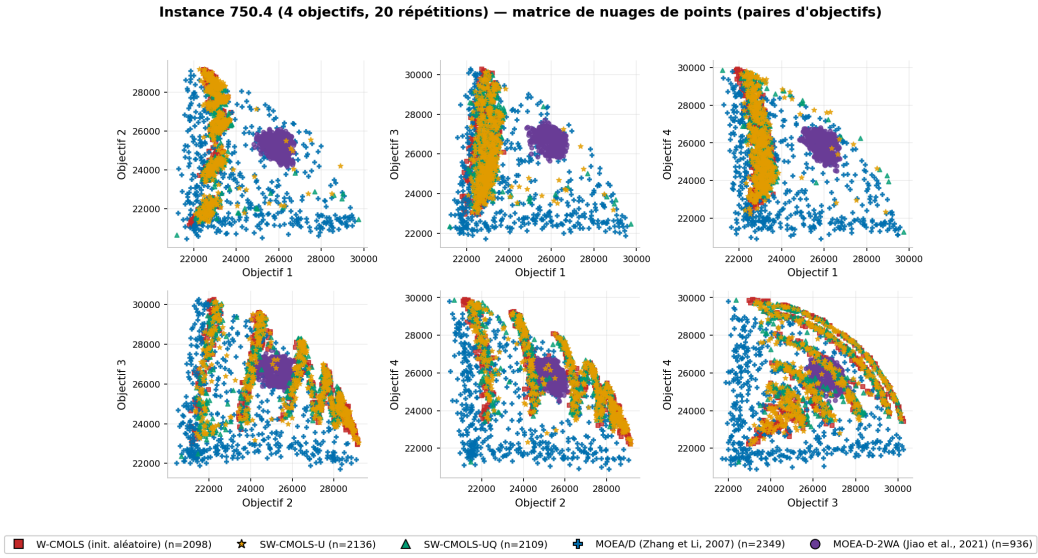


Fig. 6.18 : Instance 750.4 (4 objectifs) : *SW-CMOLS-U* et -UQ face aux deux méthodes par décomposition.

Sur 750.4, *MOEA-D-2WA* forme un groupe compact au centre des projections, alors que *MOEA/D* disperse davantage ses solutions. Nos systèmes couvrent une région plus étendue. Le tableau 6.3 fournit la comparaison dans l'espace complet à quatre objectifs.

6.4.4 Résultats selon le nombre d'objectifs

Le tableau 6.4 regroupe le gain d'hypervolume sur *W-CMOLS* selon le nombre d'objectifs.

Nombre d'objectifs	Instances concernées	Gain SW-CMOLS-U	Gain SW-CMOLS-UQ
2 objectifs	250.2, 500.2, 750.2	+12,0 %	+11,9 %
3 objectifs	250.3, 500.3, 750.3	+33,1 %	+32,6 %
4 objectifs	250.4, 500.4, 750.4	+112,9 %	+113,9 %

Tab. 6.4 : Gain moyen d'hypervolume sur *W-CMOLS*, regroupé par nombre d'objectifs (moyenne des trois instances de chaque groupe).

Le gain moyen passe d'environ +12 % avec deux objectifs à +33 % avec trois, puis à +113 % avec quatre. *SW-CMOLS-U* et *SW-CMOLS-UQ* suivent presque le même profil : l'ajout de la configuration par instance modifie peu

cette tendance déjà présente avec l'amorçage seul. Cette observation reste descriptive. Le protocole n'isole pas l'effet propre du nombre d'objectifs.

Les hypervolumes bruts de groupes ayant des nombres d'objectifs différents ne sont pas directement comparables, car leurs espaces et leurs points de référence diffèrent. À deux objectifs, le gain relatif de *SW-CMOLS-U* augmente aussi avec la taille : $+3{,}1\ \%$ sur 250.2, $+5{,}9\ \%$ sur 500.2 et $+27{,}0\ \%$ sur 750.2. Ce dernier résultat reste sensible à l'instabilité de *W-CMOLS* signalée en section 6.2.

6.4.5 Contrôle de la taille des fronts

Nos systèmes retournent de 124 à 3 327 solutions, contre quelques dizaines ou centaines pour les concurrents à population fixe. Comme l'hypervolume dépend de la couverture, ce nombre de solutions peut influencer le résultat.

Pour le contrôler, chaque front de nos systèmes est réduit au nombre moyen de solutions du concurrent, puis son hypervolume est recalculé avec les mêmes bornes. La réduction conserve les points extrêmes. Comme le nombre exact de solutions du concurrent varie entre exécutions, l'appariement repose sur sa moyenne. Le tableau 6.5 détaille le cas de *MOEA-D-2WA*.

Instance	Points	MOEA-D-2WA	SW-CMOLS-U *(réduit)*	SW-CMOLS-UQ *(réduit)*	SW-CMOLS-U *(complet)*
250.2	51	0,7824	0,8056	**0,8058**	0,8143
250.3	55	0,4574	**0,4920**	0,4826	0,5610
250.4	55	0,2101	0,2345	**0,2368**	0,3293
500.2	57	0,7388	**0,7758**	0,7740	0,7861
500.3	59	0,4123	**0,4454**	0,4415	0,5247
500.4	56	0,1613	0,2071	**0,2125**	0,3065
750.2	68	0,7067	**0,7625**	0,7612	0,7729
750.3	59	0,3533	**0,4337**	0,4283	0,5092
750.4	58	0,1433	0,1909	**0,1931**	0,2905

Tab. 6.5 : Hypervolume après réduction de nos fronts au nombre moyen de points de *MOEA-D-2WA*. Le gras indique la meilleure valeur parmi *MOEA-D-2WA* et les deux fronts réduits. La dernière colonne rappelle le front complet.

Après amincissement, *SW-CMOLS-U* dépasse encore *MOEA-D-2WA* sur les neuf instances, de $+3{,}0\ \%$ sur 250.2 à $+33{,}2\ \%$ sur 750.4. *SW-CMOLS-UQ* aussi. Le même contrôle a été appliqué à *MOEA/D* et *DCNSGA-III* : la moyenne de *SW-CMOLS-U* comme celle de *SW-CMOLS-UQ* reste supérieure dans les vingt-sept couples formés par ces trois concurrents et les neuf instances. Il s'agit d'un contrôle descriptif à cardinalité moyenne, pas d'un nouveau test statistique.

MOEA/D retourne davantage de solutions que *SW-CMOLS-U* sur sept instances. Dans ces cas, nos fronts ne sont pas augmentés. L'écart d'hypervolume ne favorise donc pas systématiquement l'ensemble le plus grand. Inversement, sur 250.4, *MOEA-D-2WA* atteint 0,2101 contre 0,1580 pour *W-CMOLS* réduit à la même taille : son avantage sur le solveur de référence ne vient pas seulement de la cardinalité.

6.4.6 Validation avec l'indicateur epsilon

Les résultats précédents reposent sur l'hypervolume. L'indicateur ε additif (équation 1.7) vérifie si le classement dépend de cette seule mesure.

Référence. Pour chaque instance, R est l'union non dominée des fronts de tous les systèmes évalués dans le mémoire : *W-CMOLS*, nos six variantes, les treize concurrents et quatre variantes d'un algorithme de colonie de fourmis développé au laboratoire, non comparé ici. La référence ne dépend donc pas de nos seuls résultats.

Calcul. Son coût croît avec le produit du nombre de points du front et de la référence. Chaque exécution est donc limitée à 1 500 points et R à 2 000 points par une réduction déterministe. Les objectifs utilisent la même normalisation que l'hypervolume. Le résultat est une approximation commune appliquée à tous les systèmes.

Interprétation. $I_{\varepsilon+}(A, R)$ mesure la translation nécessaire pour que le front A couvre la référence. Une valeur plus faible est meilleure. L'implémentation a été comparée au programme `eps_ind` de PISA sur sept couples de fronts : les valeurs coïncident à 10^{-9} près.

Notre solveur et *IBEA* utilisent une fitness liée à epsilon. Cet alignement peut les favoriser sous cet indicateur. Il ne suffit cependant pas à expliquer le classement : *W-CMOLS* possède le même mécanisme et se classe

quatorzième, tandis qu'*IBEA* est neuvième. L'hypervolume reste donc la mesure principale. Epsilon sert de contrôle complémentaire.

Système	$I_{\varepsilon+}$ moyen	Rang ε	Rang HV	SW-CMOLS-U l'emporte
SW-CMOLS-U	**0,0570**	**1**	**1**	-
MOEA/D	0,1168	2	2	8/9 (8 sign.)
MOEA/D-GR	0,1393	3	7	9/9 (9 sign.)
EAG-MOEA/D	0,1930	4	3	9/9 (9 sign.)
GWASF-GA	0,2099	5	6	9/9 (9 sign.)
C-MOEA-D	0,2174	6	4	9/9 (9 sign.)
MOEA-D-2WA	0,2304	7	5	9/9 (9 sign.)
NSGA-II	0,2323	8	11	9/9 (9 sign.)
IBEA	0,2650	9	9	9/9 (9 sign.)
SPEA2	0,2674	10	10	9/9 (9 sign.)
DCNSGA-III	0,3224	11	13	9/9 (9 sign.)
mobkp/PLS	0,3270	12	12	9/9 (9 sign.)
CMODRL	0,3602	13	14	9/9 (9 sign.)
W-CMOLS	0,4138	14	8	9/9 (9 sign.)
I-DBEA	0,4662	15	15	9/9 (9 sign.)

Tab. 6.6 : Indicateur ε additif moyen sur les neuf instances, plus petit étant meilleur. Le rang HV est celui du tableau 6.2. « Sign. » : test de Mann-Whitney U unilatéral, $p < 0{,}05$.

SW-CMOLS-U obtient la meilleure moyenne ε et une valeur inférieure dans 125 comparaisons sur 126, toutes significatives au seuil nominal. L'exception est *MOEA/D* sur 250.2, où ε est légèrement plus faible pour *MOEA/D* (0,0137 contre 0,0139) : l'instance où l'écart d'hypervolume est le plus faible est aussi celle où ε ne nous départage pas favorablement.

Avec la même référence, la moyenne de *SW-CMOLS-UQ* vaut 0,0582, juste derrière *SW-CMOLS-U* (0,0570) et loin devant *MOEA/D* (0,1168). La configuration par instance conserve donc la tête sous ε face aux concurrents, avec un léger recul par rapport à la variante statique, cohérent avec ses fronts moins fournis (section 5.4.2). Face aux concurrents, le seul cas où la moyenne de *SW-CMOLS-UQ* n'est pas meilleure reste *MOEA/D* sur 250.2. Ces valeurs utilisent la même référence et les mêmes plafonds d'amincissement que le tableau 6.6.

Les six meilleurs concurrents sous l'hypervolume occupent aussi les rangs 2 à 7 sous ε, dans un ordre différent. *W-CMOLS* passe du huitième au quatorzième rang : l'hypervolume mesure la couverture globale, tandis que ε dépend de la région la moins bien couverte. Les deux indicateurs donnent donc des classements compatibles, mais ils reposent sur les mêmes exécutions.

6.4.7 Temps d'exécution indicatifs

Le tableau 6.7 rapporte le temps moyen d'une exécution, instance par instance, pour les sept variantes de ce mémoire et les treize concurrents, ces derniers dans l'ordre du tableau 6.2. Ces temps sont indicatifs (section 6.2). Les systèmes tournent dans des environnements différents, donnés dans la dernière colonne. Un écart mesuré ne vient donc pas du seul algorithme.

Système	250.2	250.3	250.4	500.2	500.3	500.4	750.2	750.3	750.4	Moyenne	Env.
W-CMOLS	0,50	1,21	2,30	1,38	3,30	7,56	2,81	6,10	14,42	4,40	Cython
SW-CMOLS-G	0,54	1,32	3,37	1,66	3,77	7,82	3,45	6,70	16,40	5,00	Cython
SW-CMOLS-L	1,04	1,75	5,21	1,51	4,66	12,45	3,18	7,74	22,42	6,66	Cython
SW-CMOLS-U	0,59	1,64	5,54	1,60	4,64	11,18	3,07	7,66	20,72	6,29	Cython
SW-CMOLS-GQ	0,41	0,74	1,32	1,15	2,20	3,21	2,01	3,99	10,28	2,81	Cython
SW-CMOLS-LQ	0,38	0,87	1,79	1,33	1,93	5,43	2,45	4,83	9,31	3,15	Cython
SW-CMOLS-UQ	0,38	0,68	1,53	1,06	1,70	4,32	2,72	4,68	8,76	2,87	Cython
MOEA/D	3,13	4,61	10,96	7,16	11,27	36,16	11,52	17,55	67,45	18,87	C++
EAG-MOEA/D	14,18	17,49	17,17	20,90	23,11	25,44	27,29	30,40	31,75	23,08	MATLAB
C-MOEA-D	43,05	35,27	33,88	38,57	43,43	43,09	59,69	63,11	58,43	46,50	MATLAB
MOEA-D-2WA	96,77	123,36	99,45	166,87	160,23	172,85	168,69	182,44	179,76	150,05	MATLAB
GWASF-GA	37,28	72,02	84,22	49,16	85,27	108,43	55,43	62,99	110,25	73,89	MATLAB
MOEA/D-GR	4,26	6,64	14,00	7,60	13,31	40,02	11,91	18,17	66,25	20,24	C++
IBEA	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	C
SPEA2	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	C
NSGA-II	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	n.d.	C
mobkp/PLS	0,31	0,80	2,21	0,56	3,41	7,77	2,36	6,60	15,52	4,39	C++
DCNSGA-III	14,18	18,31	17,84	24,69	21,70	40,69	21,89	19,02	22,64	22,33	MATLAB
CMODRL	113,77	72,50	59,60	65,86	61,02	69,31	77,36	73,26	80,51	74,80	MATLAB
I-DBEA	46,90	50,73	49,95	51,52	58,79	57,89	71,93	89,07	162,01	70,98	MATLAB

Tab. 6.7 : Temps moyen par exécution, en secondes. « Env. » : environnement d'exécution. « n.d. » : temps enregistré mais non rapporté, car il mesure surtout l'attente du protocole PISA (voir le texte ci-dessous).

Les temps de *W-CMOLS* et de chaque variante *SW-CMOLS* portent sur 50 répétitions, mesurées sur la même machine. Ils excluent la génération du cloud et l'inférence du réseau. Les concurrents reposent sur 20 répétitions, sauf *DCNSGA-III*, *CMODRL* et *MOEA-D-2WA*, chronométrés sur une seule exécution par instance. Pour ces trois méthodes, le tableau donne donc seulement un ordre de grandeur. Enfin, le temps de *mobkp/PLS* est son critère d'arrêt, fixé à partir du temps moyen de *W-CMOLS*.

Les temps des trois références PISA ont été enregistrés mais ne sont pas rapportés. Dans cette architecture, l'algorithme et le problème sont deux programmes qui échangent des fichiers. Chacun vérifie toutes les 0,02 seconde si l'autre a répondu. Sur 2 000 générations, cette attente atteint à elle seule 80 secondes environ. Les durées enregistrées, de 74 à 145 secondes, varient d'ailleurs peu avec la taille de l'instance. Elles mesurent ce protocole plutôt que le coût de *IBEA*, *SPEA2* ou *NSGA-II*.

Sous ces réserves, les variantes configurées par instance prennent en moyenne de 2,81 à 3,15 secondes, contre 4,40 pour *W-CMOLS*, et sont plus rapides que lui sur chaque instance. Les variantes statiques atteignent 6,66 secondes. Parmi les concurrents à budget en évaluations, *MOEA/D* prend 18,87 secondes et *MOEA-D-2WA* 150,05 secondes en moyenne. L'avantage de qualité de nos systèmes ne repose donc pas sur un temps de calcul plus long : *SW-CMOLS-UQ* prend 2,87 secondes, contre 18,87 pour *MOEA/D*, le meilleur concurrent.

6.5 Conclusion

Ce chapitre a comparé *SW-CMOLS-U* et *SW-CMOLS-UQ* à treize algorithmes publiés entre 2001 et 2025. Les mêmes instances, les mêmes bornes d'hypervolume et 20 exécutions sont utilisées. Les différences de budget et de traitement des contraintes sont signalées dans le protocole.

Trois conclusions se dégagent.

Premièrement, *SW-CMOLS-U* et *SW-CMOLS-UQ* obtiennent chacun un hypervolume moyen supérieur dans les cent dix-sept comparaisons, toutes significatives au seuil nominal non corrigé. L'écart le plus faible reste 250.2 face à *MOEA/D* (+0,43 % et +0,44 %). Le contrôle par blocs sur les neuf instances reste significatif après correction de Holm pour chacun des treize concurrents ($p_{\mathrm{aj}} = 0{,}02539$).

Deuxièmement, les six concurrents qui dépassent *W-CMOLS* en moyenne relèvent de la décomposition ou de la scalarisation : *MOEA/D*, *EAG-MOEA/D* [69], *C-MOEA-D*, *MOEA-D-2WA* [70], *GWASF-GA* [71] et *MOEA/D-GR* [68]. Leur ancienneté varie de 2007 à 2021. La récence n'explique donc pas leur rang. Le dernier rang d'*I-DBEA*, également fondé sur la décomposition, montre aussi que cette famille n'est pas suffisante à elle seule.

Troisièmement, l'avantage moyen subsiste après réduction de nos fronts au nombre moyen de solutions des concurrents. Le classement de tête est aussi confirmé par l'indicateur epsilon. La taille des archives et le choix de l'hypervolume n'expliquent donc pas, à eux seuls, les écarts observés.

Ces conclusions valent pour ce banc, avec les budgets hétérogènes documentés et des tests non corrigés. Dans ces limites, le chapitre soutient la conception spécifique au problème développée aux chapitres 4 et 5.

Conclusion générale

Ce mémoire est parti d'un solveur métaheuristique éprouvé pour le sac à dos multi-objectif, *W-CMOLS*, et s'est attaché à deux décisions souvent fixées à la main : d'où la recherche part, et avec quels réglages elle s'exécute.

Remplacer l'archive initialement vide par un cloud de solutions informées, sans rien modifier d'autre dans le solveur, améliore l'hypervolume sur les neuf instances, avec un test significatif dans chaque cas. Le gain moyen, calculé instance par instance, vaut +48,0 % pour le cloud glouton, +50,1 % pour le cloud construit avec le modèle de langage et +52,0 % pour leur union. Pour cette dernière, il dépasse +120 % sur 500.4 et 750.4 (chapitre 4, section 4.4.2).

Deux leviers expliquent ce résultat. Le premier est la couverture du front : des directions de recherche en nombre suffisant, réparties sur tout le simplexe des poids, et des voisinages denses autour de chacune. Avec dix directions seulement, un criblage sans modèle de langage restait jusqu'à 32 % derrière le cloud glouton sur les instances à quatre objectifs. Avec 56, il le dépasse. Le second levier est la décision du modèle de langage. Sur les cinq graines centrales de chaque instance, le modèle repère le sac réellement saturé et remplace les objets qui le gaspillent, un échange que le tri glouton, qui traite tous les sacs comme également rares, ne propose pas. Sur les 45 décisions, 39 modifient ainsi leur graine, les six autres concluant qu'aucun échange n'est avantageux. À dispositif identique, ces 45 décisions améliorent l'hypervolume sur les neuf instances, de façon significative sur huit (+0,34 % en moyenne). L'effet est faible, mais il est mesuré contre un amorçage glouton déjà très fort.

La variante avec modèle de langage dépasse le cloud glouton sur sept instances sur neuf. L'union des deux le dépasse sur les neuf, y compris les deux instances à deux objectifs où la variante avec modèle seule reste en retrait : le cloud glouton apporte des solutions précises sur 302 directions, le cloud du modèle des voisinages denses et des graines centrales améliorées.

Sur le réglage, la configuration par instance (chapitre 5) tient un rôle précis : sur un même cloud, elle ne modifie pas l'hypervolume de façon systématique, mais réduit le temps d'exécution de 44 à 54 %. Le système final *SW-CMOLS-UQ* conserve ainsi la qualité de *SW-CMOLS-U* (+52,1 % sur *W-CMOLS*) tout en étant plus rapide que le solveur de référence sur les neuf instances : 21,5 minutes pour les 450 exécutions du protocole, contre 33,0 minutes. Deux validations complètent ce résultat (section 5.5) : réentraînée sans une instance donnée, la politique obtient une amélioration significative sur huit des neuf instances exclues, une fois leurs références mesurées. Son écart médian à l'oracle empirique vaut 0,43 %. *SMAC3* obtient un hypervolume légèrement supérieur (+0,9 %), mais doit recommencer sa recherche pour chaque instance, jusqu'à 36 minutes, là où une politique entraînée décide en une milliseconde, une fois les références de l'instance mesurées.

Enfin, la comparaison externe (chapitre 6) place ces contributions dans le paysage de la littérature. Face à treize algorithmes publiés entre 2001 et 2025, exécutés avec leurs opérateurs et leur traitement des contraintes d'origine, *SW-CMOLS-U* et *SW-CMOLS-UQ* obtiennent chacun un hypervolume moyen supérieur dans les 117 comparaisons instance-concurrent, toutes significatives au seuil nominal. L'écart le plus faible, +0,4 %, concerne *MOEA/D* sur 250.2. Ce classement résiste à la réduction de nos fronts à la taille de ceux des concurrents et à l'indicateur epsilon, qui les place en tête (0,0570 et 0,0582, contre 0,1168 pour *MOEA/D*). Le chapitre dégage aussi une régularité qui dépasse le classement : les six concurrents dont l'hypervolume moyen dépasse celui de *W-CMOLS* relèvent tous de la décomposition ou de la scalarisation, assorties d'un traitement des contraintes adapté au problème.

S'ajoute un résultat de mécanisme qui traverse plusieurs chapitres : l'étude du solveur révèle que le balayage des vecteurs de poids est tronqué dans sa configuration de référence, au point que le premier objectif ne reçoit directement aucun poids sur les instances à quatre objectifs. Plusieurs observations sont cohérentes avec ce mécanisme : le gain de l'amorçage croît avec le nombre d'objectifs (+12 %, +33 %, +113 %), et, sur 250.4, l'hypervolume stagne de 50 à 100 itérations puis gagne 41 % à 130 itérations (section 5.2). Elles ne suffisent toutefois pas à isoler causalement son effet de celui du budget de recherche.

Quatre limites doivent être énoncées clairement.

Le périmètre expérimental est étroit. Tous les résultats portent sur un problème (le MOKP), un jeu d'instances [35] et un solveur (*W-CMOLS*). Rien n'établit que les conclusions se transposent à d'autres problèmes combinatoires ou à d'autres métaheuristiques, et la propriété de balayage tronqué de la section 3.5 est une propriété de *cette* configuration. Les choix de conception ont en outre été fixés sur ces neuf instances, avec un seul cloud par instance, alors que les concurrents du chapitre 6 sont employés avec leurs réglages par défaut.

L'apport propre du modèle de langage reste modeste et dépend d'un modèle distant. Les 45 décisions ont été prises par un seul modèle. Elles sont consignées et rejouables, mais leur reproduction avec un autre modèle n'a pas été mesurée. L'essentiel du gain sur le cloud glouton vient de la couverture du front, qui ne demande aucun modèle de langage.

L'amorçage allonge chaque exécution. Partant d'une archive plus riche, le solveur conserve davantage de solutions : les variantes statiques demandent de 37,5 à 50,0 minutes pour le protocole, contre 33,0 pour *W-CMOLS*. La configuration par instance compense ce surcoût, mais sans garantie instance par instance : le temps mesuré dépasse le budget individuel sur une instance (750.2, 1,21 fois le budget).

La configuration par instance reste en deçà de l'oracle empirique et de *SMAC3*. Le criblage des 432 configurations (section 5.5.1) situe la politique apprise à 0,43 % de la meilleure configuration réévaluée en médiane, et jusqu'à 1,33 % sur 500.4. *SMAC3*, qui cherche sur chaque instance, la dépasse de 0,9 % en hypervolume. Les extensions *Dueling* et rejeu priorisé n'apportent pas de gain mesurable dans l'ablation. L'agent suppose enfin quelques exécutions préalables de *W-CMOLS* sur chaque instance (six dans notre dispositif) pour mesurer les références de l'état.

Quatre directions prolongent naturellement ce travail.

Mieux exploiter le modèle de langage. La sélection des cinq graines centrales est un choix de conception qui n'a pas été comparé à d'autres. Confier au modèle des graines réparties sur tout le front, ou davantage de graines sur les instances à quatre objectifs, permettrait de mesurer jusqu'où son apport propre peut croître.

Exploiter la propriété de balayage tronqué. Deux corrections directes peuvent être étudiées : entrelacer l'ordre des vecteurs de poids, ou augmenter le budget d'itérations jusqu'à couvrir la liste, au prix d'un temps supérieur.

Étendre le périmètre. Transposer l'amorçage informé à d'autres problèmes combinatoires multi-objectif, et à d'autres métaheuristiques que la recherche locale à base d'indicateur, permettrait de distinguer ce qui relève du principe général de ce qui relève de cette configuration particulière.

Rendre le dispositif plus autonome et plus comparable. L'état de l'agent demande aujourd'hui quelques exécutions de *W-CMOLS* sur l'instance. Le décrire à partir des seules données de l'instance supprimerait ce préalable. Un protocole de temps commun à tous les concurrents compléterait par ailleurs la comparaison du chapitre 6.

Le titre de ce mémoire annonce les modèles de langage, et le résultat principal les inclut sans s'y réduire : sur les neuf instances étudiées, un amorçage informé qui couvre tout le front améliore nettement la qualité de la recherche locale, et un modèle de langage qui raisonne sur les capacités réellement saturées y ajoute un gain faible mais mesurable. Associé à une configuration apprise par apprentissage par renforcement profond, ce système conserve sa qualité en moins de temps que le solveur de référence.

Bibliographie

[1] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, et V. Grunert da Fonseca. Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2):117-132, 2003. DOI : 10.1109/TEVC.2003.810758.

[2] H. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77-91, 1952.

[3] R. M. Karp. Reducibility among combinatorial problems. Dans *Complexity of Computer Computations*, Plenum Press, p. 85-103, 1972.

[4] I. Ben Mansour, M. Basseur, et F. Saubion. A multi-population algorithm for multi-objective knapsack problem. *Applied Soft Computing*, 70:814-825, 2018. DOI : 10.1016/j.asoc.2018.06.024.

[5] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, et X. Chen. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*, 2024.

[6] M. Ehrgott. *Multicriteria optimization*. Springer, 2005.

[7] K. Deb. *Multi-objective optimization using evolutionary algorithms*. John Wiley & Sons, 2001.

[8] C. A. C. Coello, G. B. Lamont, et D. A. Van Veldhuizen. *Evolutionary algorithms for solving multi-objective problems*. Springer, 2007.

[9] A. Auger, J. Bader, D. Brockhoff, et E. Zitzler. Theory of the hypervolume indicator: Optimal μ-distributions and the choice of the reference point. Dans *Proceedings of the tenth ACM SIGEVO workshop on Foundations of Genetic Algorithms*, ACM, p. 87-102, 2009. DOI : 10.1145/1527125.1527138.

[10] J. Bader, et E. Zitzler. HypE: An algorithm for fast hypervolume-based many-objective optimization. *Evolutionary Computation*, 19(1):45-76, 2011. DOI : 10.1162/EVCO_a_00009.

[11] E. Zitzler, et S. Künzli. Indicator-based selection in multiobjective search. Dans *Parallel Problem Solving from Nature - PPSN VIII*, Springer, p. 832-842, 2004. DOI : 10.1007/978-3-540-30217-9_84.

[12] J. R. Schott. *Fault Tolerant Design Using Single and Multicriteria Genetic Algorithm Optimization*. mémoire de master, Massachusetts Institute of Technology, 1995.

[13] M. R. Garey, et D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[14] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[15] H. Kellerer, U. Pferschy, et D. Pisinger. *Knapsack Problems*. Springer, 2004.

[16] C. Blum, et A. Roli. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys*, 35(3):268-308, 2003.

[17] S. Kirkpatrick, C. D. Gelatt, et M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671-680, 1983. DOI : 10.1126/science.220.4598.671.

[18] F. Glover. Tabu search—Part I. *ORSA Journal on Computing*, 1(3):190-206, 1989.

[19] T. A. Feo, et M. G. C. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2):109-133, 1995.

[20] M. Dorigo, V. Maniezzo, et A. Colorni. The ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1):29-41, 1996.

[21] J. Kennedy, et R. Eberhart. Particle swarm optimization. Dans *Proceedings of ICNN'95 - International Conference on Neural Networks*, p. 1942-1948, 1995.

[22] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.

[23] H. R. Lourenço, O. C. Martin, et T. Stützle. Iterated Local Search. Dans *Handbook of Metaheuristics*, Springer, 2003.

[24] N. Mladenović, et P. Hansen. Variable neighborhood search. *Computers & Operations Research*, 24(11):1097-1100, 1997.

[25] C. Voudouris, et E. Tsang. Guided local search and its application to the travelling salesman problem. *European Journal of Operational Research*, 113(2):469-499, 1999.
[26] L. Paquete, M. Chiarandini, et T. Stützle. Pareto Local Optimum Sets in the Biobjective Traveling Salesman Problem: An Experimental Study. Dans *Metaheuristics for Multiobjective Optimisation*, Springer, p. 177-199, 2004.
[27] M. Basseur, et E. K. Burke. Indicator-based multi-objective local search. Dans *IEEE Congress on Evolutionary Computation (CEC 2007)*, p. 3100-3107, 2007.
[28] A. Zhou, B.-Y. Qu, H. Li, S.-Z. Zhao, P. N. Suganthan, et Q. Zhang. Multiobjective evolutionary algorithms: A survey of the state of the art. *Swarm and Evolutionary Computation*, 1(1):32-49, 2011.
[29] J. D. Schaffer. *Multiple objective optimization with vector evaluated genetic algorithms*. Carnegie Mellon University, 1985.
[30] C. M. Fonseca, et P. J. Fleming. Genetic algorithms for multiobjective optimization: Formulation, discussion and generalization. Dans *Proceedings of the 5th International Conference on Genetic Algorithms*, p. 416-423, 1993.
[31] J. Horn, N. Nafpliotis, et D. E. Goldberg. A niched Pareto genetic algorithm for multiobjective optimization. Dans *Proceedings of the First IEEE Conference on Evolutionary Computation*, p. 82-87, 1994.
[32] K. Deb, A. Pratap, S. Agarwal, et T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182-197, 2002. DOI : 10.1109/4235.996017.
[33] E. Zitzler, M. Laumanns, et L. Thiele. SPEA2: Improving the strength Pareto evolutionary algorithm. *TIK-Report*, 103, 2001.
[34] Q. Zhang, et H. Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712-731, 2007. DOI : 10.1109/TEVC.2007.892759.
[35] E. Zitzler, et L. Thiele. Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257-271, 1999. DOI : 10.1109/4235.797969.
[36] I. Ben Mansour. An Effective Hybrid Ant Colony Optimization for the Knapsack Problem Using Multi-Directional Search. *SN Computer Science*, 2023. DOI : 10.1007/s42979-022-01564-5.
[37] A. D. Jesus. *mobkp: (Anytime) algorithms for the multi-objective binary knapsack problem*. 2022. DOI : 10.5281/zenodo.6857685.
[38] O. Vinyals, M. Fortunato, et N. Jaitly. Pointer Networks. Dans *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
[39] I. Bello, H. Pham, Q. V. Le, M. Norouzi, et S. Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.
[40] W. Kool, H. van Hoof, et M. Welling. Attention, learn to solve routing problems! Dans *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.
[41] I. Goodfellow, Y. Bengio, et A. Courville. *Deep Learning*. MIT Press, 2016.
[42] D. P. Kingma, et J. Ba. Adam: A method for stochastic optimization. Dans *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.
[43] P. J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73-101, 1964. DOI : 10.1214/aoms/1177703732.
[44] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, et I. Polosukhin. Attention is all you need. Dans *Advances in Neural Information Processing Systems 30 (NeurIPS)*, p. 5998-6008, 2017.
[45] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, et others. Training language models to follow instructions with human feedback. Dans *Advances in Neural Information Processing Systems 35 (NeurIPS)*, p. 27730-27744, 2022.
[46] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et others. Language models are few-shot learners. Dans *Advances in Neural Information Processing Systems 33 (NeurIPS)*, p. 1877-1901, 2020.
[47] Anthropic. Claude: large language model (version Opus 5.5). 2026. https://www.anthropic.com/claude.
[48] R. Zhong, A. G. Hussien, J. Yu, et M. Munetomo. LLMOA: A novel large language model assisted hyper-heuristic optimization algorithm. *Advanced Engineering Informatics*, 64:103042, 2025. DOI : 10.1016/j.aei.2024.103042.
[49] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, et G. Song. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. *arXiv preprint arXiv:2402.01145*, 2024.
[50] K.-H. Lee, I. Fischer, Y.-H. Wu, D. Marwood, S. Baluja, D. Schuurmans, et X. Chen. Evolving Deeper LLM Thinking. *arXiv preprint arXiv:2501.09891*, 2025.
[51] R. S. Sutton, et A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
[52] C. J. C. H. Watkins, et P. Dayan. Q-learning. *Machine Learning*, 8(3):279-292, 1992.

[53] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, et D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529-533, 2015. DOI : 10.1038/nature14236.

[54] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, et N. Freitas. Dueling network architectures for deep reinforcement learning. Dans *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, p. 1995-2003, 2016.

[55] T. Schaul, J. Quan, I. Antonoglou, et D. Silver. Prioritized experience replay. Dans *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

[56] H. van Hasselt, A. Guez, et D. Silver. Deep reinforcement learning with double Q-learning. Dans *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.

[57] L. Metz, J. Ibarz, N. Jaitly, et J. Davidson. Discrete sequential prediction of continuous actions for deep RL. *arXiv preprint arXiv:1705.05035*, 2017.

[58] Á. E. Eiben, R. Hinterding, et Z. Michalewicz. Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 3(2):124-141, 1999.

[59] G. Karafotias, M. Hoogendoorn, et Á. E. Eiben. Parameter control in evolutionary algorithms: Trends and challenges. *IEEE Transactions on Evolutionary Computation*, 19(2):167-187, 2015.

[60] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, et F. Hutter. SMAC3: A Versatile Bayesian Optimization Package for Hyperparameter Optimization. *Journal of Machine Learning Research*, 23(54):1-9, 2022.

[61] J. Zheng, J. Zhong, M. Chen, et K. He. A reinforced hybrid genetic algorithm for the traveling salesman problem. *Computers & Operations Research*, 157:106249, 2023. DOI : 10.1016/j.cor.2023.106249.

[62] F. Ming, W. Gong, L. Wang, et Y. Jin. Constrained Multi-objective Optimization with Deep Reinforcement Learning Assisted Operator Selection. *IEEE/CAA Journal of Automatica Sinica*, 2024.

[63] F. Ming, W. Gong, B. Xue, M. Zhang, et Y. Jin. Automated Configuration of Evolutionary Algorithms via Deep Reinforcement Learning for Constrained Multiobjective Optimization. *IEEE Transactions on Cybernetics*, 55(12):5877-5890, 2025.

[64] H. B. Mann, et D. R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1):50-60, 1947. DOI : 10.1214/aoms/1177730491.

[65] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, et K. Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2):31-39, 2011. DOI : 10.1109/MCSE.2010.118.

[66] J. Blank, et K. Deb. pymoo: Multi-Objective Optimization in Python. *IEEE Access*, 8:89497-89509, 2020. DOI : 10.1109/ACCESS.2020.2990567.

[67] Y. Tian, R. Cheng, X. Zhang, et Y. Jin. PlatEMO: A MATLAB Platform for Evolutionary Multi-Objective Optimization. *IEEE Computational Intelligence Magazine*, 12(4):73-87, 2017. DOI : 10.1109/MCI.2017.2742868.

[68] Z. Wang, Q. Zhang, A. Zhou, M. Gong, et L. Jiao. Adaptive replacement strategies for MOEA/D. *IEEE Transactions on Cybernetics*, 46(2):474-486, 2016.

[69] X. Cai, Y. Li, Z. Fan, et Q. Zhang. An External Archive Guided Multiobjective Evolutionary Algorithm Based on Decomposition for Combinatorial Optimization. *IEEE Transactions on Evolutionary Computation*, 19(4):508-523, 2015.

[70] R. Jiao, S. Zeng, C. Li, et Y.-S. Ong. Two-type weight adjustments in MOEA/D for highly constrained many-objective optimization. *Information Sciences*, 578:592-614, 2021.

[71] R. Saborido, A. B. Ruiz, et M. Luque. Global WASF-GA: An Evolutionary Algorithm in Multiobjective Optimization to Approximate the Whole Pareto Optimal Front. *Evolutionary Computation*, 25(2):309-349, 2017.

[72] H. Jain, et K. Deb. An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point Based Nondominated Sorting Approach, Part II: Handling Constraints and Extending to an Adaptive Approach. *IEEE Transactions on Evolutionary Computation*, 18(4):602-622, 2014. DOI : 10.1109/TEVC.2013.2281534.

[73] M. Asafuddoula, T. Ray, et R. Sarker. A Decomposition-Based Evolutionary Algorithm for Many Objective Optimization. *IEEE Transactions on Evolutionary Computation*, 19(3):445-460, 2015. DOI : 10.1109/TEVC.2014.2339823.

[74] R. Jiao, S. Zeng, C. Li, S. Yang, et Y.-S. Ong. Handling Constrained Many-Objective Optimization Problems via Problem Transformation. *IEEE Transactions on Cybernetics*, 51(10):4834-4847, 2021. DOI : 10.1109/TCYB.2020.3031632.

[75] S. Bleuler, M. Laumanns, L. Thiele, et E. Zitzler. PISA — a platform and programming language independent interface for search algorithms. Dans *Evolutionary Multi-Criterion Optimization (EMO 2003)*, Springer, p. 494-508, 2003.